



Addis Ababa Science and Technology University

College of Engineering

Department of Software Engineering

Enhanced Advanced Adaptive RAG Pipeline



Addis Ababa Science and Technology University
College of Engineering
Department of Software Engineering

Title: Enhanced Advanced Adaptive RAG Pipeline

Group Members:

No.	Name	ID
1.	Nahom Ketsela	ETS2501/14
2.	Niyat Hanibbal	ETS1305/14
3.	Saad Musema	ETS1389/14
4.	Samiya Yusuf	ETS1404/14
5.	Samrawit Gebremaryam	ETS1406/14

Advisor Name: Instructor Felix Edesa

Signature:

MAY, 2026

ACKNOWLEDGEMENT

First and foremost, we would like to express our deepest professional gratitude to our advisor, Mr. Felix Edessa, for his invaluable guidance, technical insights, and consistent encouragement throughout the development of this project. His expertise in software architecture and commitment to academic excellence have been pivotal in shaping the direction of "Grounded AI"

We also wish to thank the Addis Ababa Science and Technology University (AASTU), specifically the Department of Software Engineering, for providing the scholarly environment and resources necessary to undertake this research. We are indebted to the faculty members who have shared their knowledge throughout our undergraduate journey.

Finally, we acknowledge the global open-source community and the researchers in the field of Large Language Models whose work provided the foundation for our adaptive pipeline. This project is a culmination of collective effort, curiosity, and the unwavering support of our families and peers.

Table of Contents

ACKNOWLEDGEMENT	i
LIST OF FIGURES	viii
LIST OF TABLES	ix
LIST OF ABBREVIATIONS, SYMBOLS, AND NOMENCLATURE	x
ABSTRACT	1
Chapter One: Introduction	2
1.1 Statement of the Problem	3
1.2 Objectives	4
1.2.1 General Objective	4
1.2.2 Specific Objectives	4
1.3 Scope and Limitation	5
1.3.1 Scope	5
1.3.2 Limitation	5
1.4 Methodology	6
1.4.1 Research and Analysis Approach	6
1.4.2 System Design Approach	6
1.4.3 Development and Implementation Approach	7
1.4.4 Testing and Evaluation Approach	7
1.5 Plan of Activities	7
1.6 Budget Required	9
1.7 Significance of the Study	10
1.8 Outline of the Study	11
Chapter One: Introduction	11
Chapter Two: Literature Review	11
Chapter Three: Problem Analysis and Modeling	11
Chapter Four: System Design	12
Chapter Five: System Implementation	12
Chapter Six: System Evaluation	12
Chapter Seven: Conclusions and Recommendations	12
Chapter Two: Literature Review	13
2.1. Study Related Works	14
2.1.1 Large Language Models	14

2.1.2 Retrieval-Augmented Generation	14
2.1.3 Basic RAG Architecture	14
2.1.4 Limitations of Conventional RAG Systems	15
Pipeline Complexity	15
Unreliable Retrieval	15
Query Ambiguity	15
Hallucination and Trust Deficit	16
2.1.5 Hybrid Retrieval	16
2.1.6 BM25 and Lexical Retrieval	16
2.1.7 Semantic Retrieval	16
2.1.8 Reciprocal Rank Fusion	17
2.1.9 Reranking Techniques	17
2.1.10 Query Transformation	17
2.1.11 Semantic and Adaptive Chunking	17
2.1.12 Temporal Ranking and Freshness-Aware Retrieval	17
2.1.13 Corrective Retrieval-Augmented Generation	18
2.1.14 Verification-Oriented Generation	18
2.1.15 Structured Output and Source Attribution	18
2.1.16 Agent-Oriented Grounded AI Systems	18
2.1.17 Productized Document Intelligence Platforms	18
2.2. Identifying Milestones and Finding the Gaps	19
2.2.1 Emergence of Standalone LLM Interfaces	19
2.2.2 Emergence of Retrieval-Augmented Generation	19
2.2.3 Hybrid Retrieval and Reranking	19
2.2.4 Adaptive and Corrective Retrieval Architectures	20
2.2.5 Trustworthy and Productized Grounded AI Systems	20
2.2.6 Research Gaps Identified from the Literature	20
2.3. Lessons Learned from the Literature	21
Chapter Three: Problem Analysis and Modeling	23
3.1 Existing System and Its Problems	23
3.2 Specifying the Requirements of the Proposed Solution	24
3.3. System Modeling	25
3.3.1. Functional Requirements (FR)	25

3.3.2. Non-Functional Requirements (NFR)	27
Summary of Requirements Mapping	28
3.3.3. Usecase	28
3.3.4. Dynamic Models of a System	42
3.3.4.1. Sequence Diagram: Document Upload and Processing Flow	42
3.3.4.2. Sequence Diagram: Query Execution Flow	42
3.3.4.3. State Machine Diagram: Document Processing States	43
3.3.4.4. State Machine Diagram: Query Execution States	44
3.3.4.5. Activity Diagram: Complete Query Processing Workflow	46
3.3.4.6. Collaboration Diagram: Query Processing Components	47
3.4 Model Validation	47
Chapter Four: System Design	49
4.1. Overview	49
4.2. Specifying the Design Goals	49
4.3. System Design	51
4.3.1. Proposed Software Architecture	51
4.3.2. Subsystem Decomposition	53
4.3.2.1. User Interface Subsystem	53
4.3.2.2. Authentication and Authorization Subsystem	54
4.3.2.3. Document Processing Subsystem	55
4.3.2.4. Retrieval Subsystem	57
4.3.2.5. Query Orchestration Subsystem	58
4.3.2.6. Generation and Verification Subsystem	59
4.3.2.7. Monitoring and Logging Subsystem	60
4.3.3. Database Design	61
4.3.4. Deployment Diagram	64
4.3.5. User Interface Design	65
4.3.6. System Integration	66
4.3.7. Security Design	67
4.4. Verifying the Requirements in the Design	67
Chapter Five: System Implementation	69
5.1 Reviewing the Design Solution	69
5.1.1 RAG Context and the Implemented Response	69

5.1.2 Rival Approaches Considered	70
5.1.3 Implementation Story and Roadmap	70
5.1.4 Requirement Prioritization Using MoSCoW	70
MoSCoW Prioritization Summary	71
5.1.4 Three-Tier Execution Model (Standard, Enterprise, Critical)	71
Standard tier flow	72
Enterprise tier flow (additions on top of Standard)	72
Critical tier flow (additions on top of Enterprise)	72
Tier gating in data	73
5.1.5 User Journey	73
5.1.6 User Manual	74
Step 1: Review the Workspace Overview	74
Step 2: Choose an Agent Setup Path	76
Step 3: Configure the Agent	77
Step 4: Select Knowledge Sources and Launch the Agent	78
Step 5: Ask Questions and Review Grounded Answers	79
5.2 Deciding on the Development Tools	80
5.2.1 Programming Languages	80
5.2.2 Backend Framework and Tools	81
5.2.3 Frontend Framework and Tools	81
5.2.4 Storage and Retrieval Tools	82
5.2.5 Development Environment and Infrastructure Tools	82
5.2.6 Version Control Practice	82
5.2.7 Documented Product Capabilities	83
5.3 Developing the Solution	86
5.3.1 Backend Application Entry and API Routing	99
5.3.2 Authentication and Tenant Context Implementation	100
5.3.3 Workspace and Dataset Implementation	100
5.3.4 Document Upload and Ingestion Implementation	101
5.3.5 Extraction and Normalization Implementation	101
5.3.6 Structure-Aware Chunking Implementation	101
5.3.7 Sparse and Dense Indexing Implementation	102
5.3.8 Query Analysis and Transformation Implementation	102

5.3.9 Hybrid Retrieval Implementation	103
5.3.10 Ranking and Evidence Selection Implementation	103
5.3.11 Grounded Query Orchestration Implementation	103
5.3.12 Response Shaping and Structured Output Implementation	104
5.3.13 Verification Implementation	104
5.3.14 Agent Implementation	104
5.3.15 Conversation and Chat Implementation	104
5.3.16 Run Trace and Inspection Implementation	105
5.3.17 Dashboard and Product Visibility Implementation	105
5.3.18 API Key Management Implementation	105
5.3.19 Frontend Product Implementation	106
5.3.20 Implementation of the Research Contribution and Product Contribution	107
Research Contribution in Implementation	107
Product Contribution in Implementation	107
Chapter Six: System Evaluation	109
6.1 Preparing Sample Test Plans	109
6.1.1 Evaluation Objectives	109
6.1.2 Types of Tests Used	110
Functional Testing	110
Integration Testing	110
Grounding and Retrieval Evaluation	110
Product Workflow Evaluation	110
Traceability and Trust Evaluation	111
6.1.3 Test Preparation Strategy	111
6.1.4 Success Criteria	111
6.2 Evaluating the Proposed Design and Solutions	112
6.2.1 Functional Test Cases	112
Interpretation notes for selected cases	119
6.2.2 Pipeline-Oriented Evaluation	119
Query Ambiguity Handling Evaluation	119
Retrieval Reliability Evaluation	119
6.2.2 Execution-tier evaluation	120
Groundedness and Trust Evaluation	120

6.2.3 Integration Evaluation	121
Integration Scenario 1: Upload to Searchable Knowledge	121
Integration Scenario 2: Dataset to Agent to Chat	121
Integration Scenario 3: Chat to Run Inspection	122
6.2.4 Trust and Traceability Evaluation	122
6.2.5 Evaluation of the Product Contribution	123
6.3 Result Analysis	123
6.3.1 Research-Side Analysis	124
6.3.2 Product-Side Analysis	124
6.3.3 Overall Analysis	124
Chapter Seven: Conclusions and Recommendations	126
7.1 Conclusion	126
7.2 Recommendations	127
References	129

LIST OF FIGURES

Figure 1.1. Gantt Chart	9
Figure 3.3.1. Use Case Diagram	25
Figure 3.3.4.1. Sequence Diagram: Document Upload and Processing Flow	26
Figure 3.3.4.2. Sequence Diagram: Query Execution Flow	27
Figure 3.3.4.3. State Machine Diagram: Document Processing States	27
Figure 3.3.4.4. State Machine Diagram: Query Execution States	28
Figure 3.3.4.5. Activity Diagram: Complete Query Processing Workflow	30
Figure 3.3.4.6. Collaboration Diagram: Query Processing Components	31
Figure 4.3.2.1. User Interface Subsystem	37
Figure 4.3.2.2. Authentication and Authorization Subsystem	38
Figure 4.3.2.3. Document Processing Subsystem	39
Figure 4.3.2.4. Retrieval Subsystem	41
Figure 4.3.2.5. Query Orchestration Subsystem	42
Figure 4.3.2.6. Generation and Verification Subsystem	43
Figure 4.3.2.7. Monitoring and Logging Subsystem	44
Figure 4.3.3.1. ER Diagram	45
Figure 4.3.4. Deployment Diagram	48
Figure 4.3.5. User Interface Diagram	49
Figure 5.1. User Journey of the Grounded AI Platform	57
Figure 5.2. System Architecture of Grounded AI	58
Figure 5.3. Account Registration Interface for Creating a New Grounded AI User and Organization Profile	59
Figure 5.4. Implemented Grounding Pipeline (Shared Across Tiers)	103
Figure 5.5. Dataset-to-Agent-to-Grounded-Chat Workflow	104
Figure 5.6. Entity-Relationship Model for the Implemented Data Schema	106
Figure 5.7. Activity Diagram – Document Upload and Ingestion	107
Figure 5.8. Activity Diagram – Dataset, Agent, and Grounded Chat	107
Figure 5.9. Activity Diagram – Run Inspection and Trust Review	107
Figure 5.10. Collaboration Diagram – Frontend, API, Services, Stores, and Retrieval	
Indexes	108
Figure 6.1. Evaluation Workflow Mapped to the Product Journey	122

LIST OF TABLES

Table 1.1. Plan of Activities	9
Table 1.2. Budget Required	10
Table 3.1. Requirement Mapping	27
Table 3.2. Use Case Identification	29
Table 3.3. Use Case 01: Submit Query	30
Table 3.4. Use Case 02: Agentic Query Expansion and Multi-Step Decomposition	31
Table 3.5. Use Case 03: Execute Recursive Semantic Chunking	31
Table 3.6. Use Case 04: Hybrid Retrieval (BM25 + Dense Search)	32
Table 3.7. Use Case 05: BGE Neural Reranking	33
Table 3.8. Use Case 06: Apply Temporal Scoring and Time-Decay Logic	33
Table 3.9. Use Case 07: Agentic Verification (Generator–Critic Loop)	34
Table 3.10. Use Case 08: Generate Grounded Response	35
Table 3.11. Use Case 09: Provide Final Output to User	35
Table 3.12. Use Case 10: Manage Data Sources	36
Table 3.13. Use Case 11: Configure System Parameters	36
Table 3.14. Use Case 12: Monitor System Logs and Pipeline Performance	37
Table 3.15. Use Case 13: Obtain API Key and Integrate RAG API	38
Table 5.1. MoSCoW Prioritization	55
Table 5.2. User-Facing Modes and Backing Tiers	57
Table 5.3. Development Tools and Their Purpose	66
Table 5.4. Frontend Mapping (Page → API → Function)	70
Table 5.5. Grounded AI Pipeline Implementation	103
Table 5.6. Research Contribution vs. Software Artifact (Summary)	117
Table 6.1. Functional Test Cases	123
Table 6.2. Backend Integration Test Inventory	125
Table 6.3. Quantitative Evaluation Indicators	126
Table 6.4. Detailed Evaluation Mapping	127
Table 6.5. Supplementary Benchmark Results	128

LIST OF ABBREVIATIONS, SYMBOLS, AND NOMENCLATURE

Abbreviation	Full Term / Description
AI	Artificial Intelligence
LLM	Large Language Model (e.g., GPT-4, Llama 3)
RAG	Retrieval-Augmented Generation: A framework for retrieving facts from an external knowledge base to ground LLM responses
CRAG	Corrective RAG: A self-correcting mechanism that triggers web-search if internal data is insufficient
RRF	Reciprocal Rank Fusion: An algorithm used to merge and rank multiple search result lists
BGE	BAAI General Embedding: A high-performance reranking model used for factual distillation
Vector DB	Vector Database: A specialized database (like Qdrant or Pinecone) that stores data as mathematical vectors for semantic search
NLP	Natural Language Processing
API	Application Programming Interface
Pydantic	A data validation library used here to enforce structured citations and verifiable quotes
BM25	Best Matching 25: A ranking function used by search engines to estimate the relevance of documents to a search query
UML	Unified Modeling Language
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case

CRUD	Create, Read, Update, Delete
RBAC	Role-Based Access Control
OAuth	Open Authorization
REST	Representational State Transfer
HTTPS	Hypertext Transfer Protocol Secure
TLS	Transport Layer Security
JSON	JavaScript Object Notation
JSONB	JSON Binary (PostgreSQL type)
ORM	Object-Relational Mapping
SQL	Structured Query Language
ER	Entity-Relationship
ETB	Ethiopian Birr
UUID	Universally Unique Identifier
SHA-256	Secure Hash Algorithm 256-bit
nDCG	Normalized Discounted Cumulative Gain
MRR	Mean Reciprocal Rank
BEIR	Benchmarking IR (Information Retrieval)
RAGAS	RAG Assessment (evaluation framework)

MoSCoW	Must/Should/Could/Won't
UI	User Interface
E2E	End-to-End
CI	Continuous Integration
CPU	Central Processing Unit

ABSTRACT

Large Language Models (LLMs) have redefined the capabilities of artificial intelligence, yet their reliance on static training data leads to the "Hallucination" crisis, the generation of factually incorrect information. While Retrieval-Augmented Generation (RAG) was introduced to mitigate this by providing real-time context, standard implementations suffer from an "Expertise-Reliability Gap." Building a production-ready RAG system requires complex engineering of chunking, vector indexing, and reranking, which remains inaccessible to many developers and prone to error in high-stakes fields such as law and medicine. [1]

This project introduces Grounded, an advanced, multi-tiered Advanced Adaptive RAG Pipeline designed to automate high-precision AI construction. Grounded utilizes an 11-stage optimization suite, including Agentic Query Transformation, Recursive Semantic Chunking, and a secondary "Critic Agent" for post-generation verification. The system features an adaptive router that classifies queries into three tiers (Standard, Enterprise, and Critical) to balance latency with extreme verifiability. Delivered as both a zero-friction API and a Docker-native environment for total data sovereignty, Grounded allows users to simply provide their data and receive a production-grade, citation-backed AI system.

Keywords: *Retrieval-Augmented Generation (RAG), Large Language Models, Hallucination Mitigation, Adaptive Pipelines, Data Sovereignty. [1]*

Chapter One: Introduction

The rapid advancement of Artificial Intelligence (AI), particularly Large Language Models (LLMs) such as GPT-4, Claude, and Llama, significantly transformed the way users interacted with digital information. These models demonstrated capabilities in natural language understanding, human-like text generation, content summarization, and decision-support processes across multiple domains including healthcare, education, legal services, customer support, and enterprise knowledge management. As a result, organizations increasingly integrated AI-powered systems into modern operational workflows.

At the same time, organizations continuously generated large volumes of digital documents such as reports, contracts, manuals, policies, technical specifications, meeting records, and operational guidelines. Although these documents contained valuable organizational knowledge, users often faced difficulties accessing the required information efficiently. Traditional approaches such as manual document browsing and keyword-based search remained inefficient, time-consuming, and highly dependent on users knowing the exact wording used within source documents.

Large Language Models introduced a more natural approach to information access by enabling conversational interaction with digital systems. However, standalone LLMs remained insufficient for organizational environments because they primarily relied on parametric knowledge acquired during training. These models could not reliably access private organizational documents, recently updated information, or domain-specific knowledge unless external retrieval mechanisms were integrated into the system. Consequently, LLMs frequently generated fluent but unsupported or factually incorrect responses, a problem commonly referred to as hallucination [1], [12], [26].

Retrieval-Augmented Generation (RAG) emerged as an important solution to this limitation. RAG systems improved response grounding by retrieving relevant information from external knowledge sources and injecting it into the language model before response generation [1]. This approach enabled AI systems to generate responses based on actual organizational documents rather than relying entirely on internal model memory.

Despite these advantages, conventional RAG systems continued to face major challenges in practical organizational environments. Many implementations suffered from pipeline complexity, unreliable retrieval, and query ambiguity. Basic RAG systems often depended heavily on vector similarity retrieval, which retrieved semantically related but irrelevant passages while failing to retrieve exact supporting evidence [5], [6]. Additionally, users frequently submitted vague or multi-intent queries that required contextual interpretation, yet many RAG pipelines forwarded such queries directly into retrieval without sufficient clarification or transformation [16], [17].

These challenges reduced trust in AI-generated outputs and limited the adoption of AI systems in professional environments where accuracy, transparency, and verifiability remained essential. Organizations required systems that not only generated intelligent responses but also provided reliable evidence, traceability, and practical usability.

This project addressed these challenges by proposing the Advanced Adaptive RAG Pipeline integrated into a platform called Grounded AI. The proposed framework integrated advanced mechanisms such as

query transformation, semantic chunking, hybrid retrieval, temporal ranking, reranking, corrective retrieval behavior, verification-oriented generation, structured output enforcement, and source attribution to improve retrieval precision and reduce hallucination.

In addition to the research contribution, the project also focused on practical software engineering and product implementation. The Advanced Adaptive RAG Pipeline, served as the intelligence core of a grounded AI platform that allowed organizations and users to upload documents, organize datasets, create grounded agents, interact through conversational interfaces, inspect outputs, and manage access through dashboards and API workflows. Therefore, this project contributed both an adaptive RAG architecture and a practical software platform intended to make grounded organizational AI systems more reliable, transparent, scalable, and usable.

1.1 Statement of the Problem

Retrieval-Augmented Generation (RAG) systems were introduced to improve the factual grounding of Large Language Models by enabling them to retrieve external information before generating responses [1]. While this approach reduced reliance on parametric memory, many existing RAG implementations still struggle to provide reliable and trustworthy responses in organizational environments where information accuracy and traceability are critical [12], [26].

One major challenge is retrieval reliability. Many conventional systems rely primarily on vector similarity search, which may retrieve semantically related but non-answer-bearing passages while failing to identify exact policy statements, procedures, or domain-specific information required to answer a user's question accurately [5], [6]. This problem becomes more significant when organizational knowledge is distributed across large collections of heterogeneous documents.

Another challenge is query understanding. Users often submit vague, underspecified, or multi-intent questions that require contextual interpretation before retrieval can be performed effectively. Existing RAG pipelines frequently process such queries directly, resulting in retrieval of partially relevant evidence and responses that do not fully address the user's information needs [16], [17].

A further limitation is the lack of transparency and verifiability in many current systems. Although retrieved documents are used during generation, users are often unable to clearly determine which sources support a particular response or whether the generated answer accurately reflects the retrieved evidence. This reduces trust in AI-generated outputs and limits adoption in environments where accountability and evidence-based decision making are required.

In addition, many proposed RAG architectures remain research-oriented and do not provide complete solutions for practical organizational deployment. Organizations require systems that not only retrieve and generate responses, but also support document management, dataset organization, reusable AI agents, access control, monitoring, and traceable interactions within a unified platform.

Therefore, this study addresses the following central problems:

1. How can retrieval reliability, query understanding, grounding accuracy, and response verifiability be improved within Retrieval-Augmented Generation systems for organizational document intelligence?
2. How can these improvements be integrated into a practical grounded AI platform that supports document management, grounded agents, transparent evidence attribution, and secure organizational deployment?

1.2 Objectives

1.2.1 General Objective

The general objective of this study is to design and implement Grounded, an adaptive Retrieval-Augmented Generation platform that improves the reliability and usability of AI-driven document intelligence systems for organizations and users

1.2.2 Specific Objectives

The specific objectives of the project are to:

1. Examine the limitations of traditional RAG systems, particularly pipeline complexity, unreliable retrieval, and query ambiguity.
2. Design an Advanced Adaptive RAG Pipeline that improves retrieval quality, grounding accuracy, and response trustworthiness.
3. Implement query transformation mechanisms to reduce ambiguity before retrieval.
4. Implement semantic chunking techniques to preserve contextual meaning during document indexing.
5. Implement namespace isolation mechanisms to support secure organizational data separation.
6. Implement hybrid retrieval strategies that combine semantic and keyword-based retrieval methods.
7. Implement temporal ranking mechanisms to reduce the impact of outdated information.
8. Implement reranking techniques to improve evidence relevance and retrieval precision.
9. Implement corrective retrieval behavior for low-confidence or weakly supported responses.
10. Implement verification-oriented response generation with structured output enforcement and source attribution.
11. Develop a platform that allows users to upload, organize, and manage organizational documents.
12. Develop grounded agents that can interact with organizational datasets through conversational workflows.
13. Develop dashboard and run-inspection features for monitoring, traceability, and system transparency.
14. Provide developer and organizational access through APIs and API key management workflows.
15. Evaluate both the technical performance of the Advanced Adaptive RAG Pipeline and the usability of the final software platform.

1.3 Scope and Limitation

1.3.1 Scope

This project focused on both the research-oriented adaptive Retrieval-Augmented Generation (RAG) pipeline and the practical grounded AI platform developed around it.

From a technical perspective, the project covered the design and implementation of an Advanced Adaptive RAG Pipeline that incorporated:

- Query analysis and transformation before retrieval
- Structure-aware document chunking that preserves contextual coherence
- Namespace-based tenant and dataset isolation
- Hybrid retrieval combining BM25 sparse search and dense vector search
- Reciprocal rank fusion for combining retrieval results
- Temporal relevance scoring based on document metadata
- Reranking for improved evidence relevance
- Corrective retrieval behavior that triggers additional search
- Internal evidence expansion for supplementing retrieval
- Verification loops for validating generated claims
- Structured response shaping with citations and confidence labels
- Persistent execution traces for post-query inspection
- Source attribution linking generated statements to specific documents

From a product perspective, the project included the development of a grounded AI platform that enabled users and organizations to:

- Create and manage workspaces
- Upload TXT, PDF, and DOCX documents
- Organize uploaded files into datasets
- Process documents into searchable knowledge bases
- Create grounded agents connected to datasets
- Interact with datasets through conversational chat interfaces
- Receive grounded responses with citations and trust indicators
- Inspect execution runs, outputs, and retrieval traces
- Manage API keys and system access

The project therefore addressed both the internal intelligent retrieval architecture and the external software platform that operationalized this architecture in real-world organizational contexts. The design aligned with established RAG system principles and extensions such as corrective retrieval and verification mechanisms to improve grounding and reduce hallucinations in generated responses [1], [8], [9].

1.3.2 Limitation

Although the project delivered a functional adaptive RAG platform within the defined scope, several limitations remained.

- The document ingestion pipeline supported only text-oriented formats such as TXT, PDF, and DOCX.
- The response quality depended heavily on the completeness, consistency, and structure of the uploaded documents.
- Highly ambiguous or underspecified queries sometimes required additional clarification before accurate retrieval and generation.
- Retrieval effectiveness varied based on document formatting, segmentation quality, and writing style.
- Large-scale enterprise optimization, including distributed deployment and high-throughput scaling, remained outside the academic scope of the project.
- Advanced enterprise governance features and extensive third-party integrations were not fully implemented in the current system version.

These limitations reflected both implementation constraints and the academic scope boundaries of the project rather than conceptual shortcomings of the proposed architecture.

1.4 Methodology

This study followed a software engineering research-and-development methodology that combined analytical research with practical system implementation. The methodology focused on identifying limitations in conventional Retrieval-Augmented Generation (RAG) systems, designing an improved adaptive pipeline, and implementing the proposed solution as a complete grounded AI platform.

1.4.1 Research and Analysis Approach

The study began with an investigation of existing Large Language Model (LLM) and Retrieval-Augmented Generation architectures. The analysis identified major limitations in conventional RAG systems, particularly pipeline complexity, unreliable retrieval, hallucination, and query ambiguity. The study reviewed related research works, retrieval techniques, reranking strategies, and verification-oriented approaches to establish the theoretical foundation for the proposed system [1], [7], [8], [9], [26].

Based on this analysis, the study defined functional and non-functional requirements for developing a reliable grounded AI platform capable of supporting organizational document intelligence and trustworthy information retrieval.

1.4.2 System Design Approach

After requirement analysis, the study designed an Advanced Adaptive RAG Pipeline aimed at improving retrieval quality, grounding accuracy, and response trustworthiness. The design integrated multiple components, including query transformation, semantic chunking, hybrid retrieval, temporal ranking, reranking, corrective retrieval behavior, verification mechanisms, structured output enforcement, and source attribution.

The system architecture also supported practical organizational use. It included workspace management, dataset organization, grounded agents, conversational interaction workflows, dashboard monitoring, API integration, and multi-tenant data isolation.

The study used Unified Modeling Language (UML) diagrams and workflow models to represent system structure, component interactions, and operational processes.

1.4.3 Development and Implementation Approach

The implementation phase followed an iterative development approach in which system components were developed, tested, refined, and integrated incrementally.

The backend services were implemented using FastAPI to support scalable API communication and workflow orchestration. The frontend interface was developed using Next.js to provide an interactive and responsive user experience. PostgreSQL was used for structured data storage, while Qdrant supported dense vector indexing and semantic retrieval. MinIO was integrated for object storage and document management. Docker Compose was used to provide reproducible infrastructure and simplified deployment workflows.

The implementation process covered both the Advanced Adaptive RAG Pipeline and the user-facing platform, including datasets, grounded agents, conversational workflows, dashboard features, retrieval tracing, and API interactions.

1.4.4 Testing and Evaluation Approach

System testing and evaluation were conducted throughout the development process to ensure functional correctness, retrieval reliability, and platform usability, consistent with standard software testing documentation practices.

The evaluation focused on:

- retrieval effectiveness
- grounding accuracy
- response relevance
- citation consistency
- workflow correctness
- API functionality
- platform usability

Both pipeline-level behavior and end-user workflows were tested to verify that the system operated reliably within the defined project scope.

1.5 Plan of Activities

The project followed a structured progression from research analysis to final system implementation, testing, and documentation. The activities were organized across a 24-week period beginning with problem identification and literature review, followed by system design and implementation phases, and concluding with testing, evaluation, documentation, and final defense preparation.

No.	Activity	Description	Deliverable	Timeline
1	Problem Identification	Identified weaknesses in traditional RAG and document-access systems	Problem Definition	November 2025
2	Literature Review	Reviewed RAG architectures, retrieval methods, verification techniques, and related works	Literature Review Chapter	November – December 2025
3	Requirement Analysis	Identified system and product requirements	Requirement Specification	December 2025
4	System Modeling	Modeled adaptive pipeline behavior and platform workflows	Analysis and Modeling Chapter	January 2026
5	System Design	Designed adaptive RAG architecture and software platform	Design Chapter	January 2026
6	Backend Implementation	Developed ingestion, retrieval, grounding, and API services	Functional Backend System	February – April 2026
7	Frontend Implementation	Developed datasets, agents, dashboard, and chat interfaces	Functional Frontend System	April – May 2026
8	System Testing and Evaluation	Tested pipeline performance, workflow correctness, and platform usability	Evaluation Report	May 2026
9	Documentation and Finalization	Prepared final thesis documentation and system refinements	Final Report	May 2026

10	Final Submission and Defense Preparation	Finalized report corrections, presentation slides, and defense preparation	Final Submission and Defense	June 3, 2026
----	--	--	------------------------------	--------------

Table 1.1: Plan of Activities

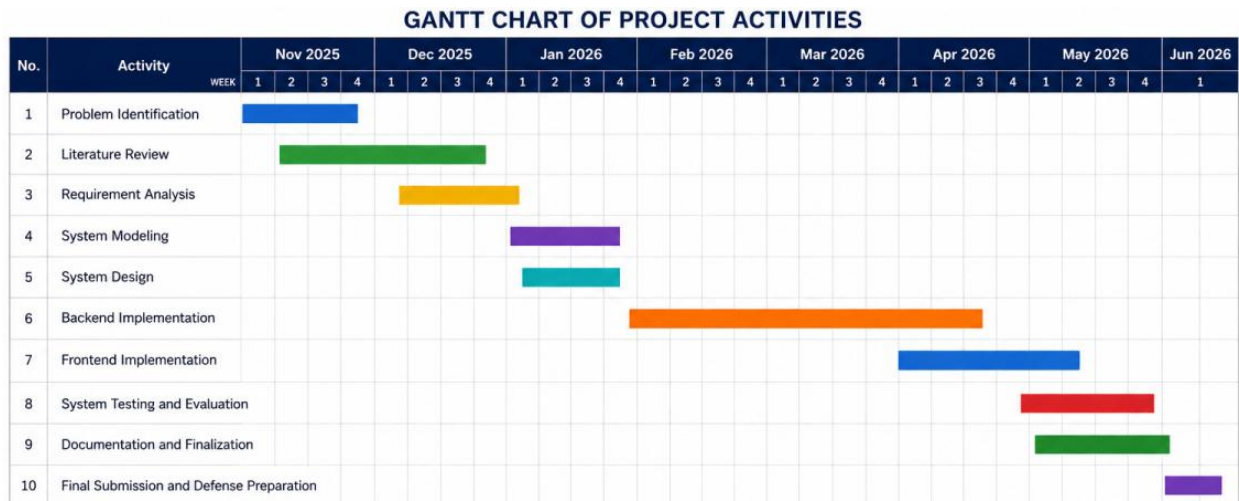


Fig 1. Gantt Chart

1.6 Budget Required

The project was implemented primarily using open-source technologies, local computing resources, and cloud-based testing services.

No.	Item	Purpose	Estimated Cost (ETB)
1	Internet Access	Package downloads, testing, cloud access, and documentation	3,000
2	Computing and Power Usage	Development, model testing, and infrastructure runtime	2,500
3	Printing and Binding	Proposal drafts and final thesis submission	2,000

4	Stationery	Notes, review materials, and documentation support	800
5	Cloud and API Testing	External API usage and evaluation services	2,500
6	Contingency	Miscellaneous technical and reporting expenses	1,200
		Total Estimated Budget	12,000 ETB

Table 1.2: Budget Required

The total budget estimate is approximately 12,000 ETB. This estimate is intended to be sufficient for prototype development, integration testing, and performance evaluation under realistic query workloads.

1.7 Significance of the Study

This study was significant because it contributed both to the advancement of Retrieval-Augmented Generation research and to the practical development of grounded AI systems for organizational use.

From the research perspective, the study addressed important limitations of traditional RAG systems, particularly unreliable retrieval, pipeline complexity, hallucination, and query ambiguity [1], [12], [26]. The proposed adaptive pipeline demonstrated how multiple retrieval, ranking, and verification mechanisms could be integrated to improve grounding accuracy, retrieval quality, and response trustworthiness.

The study also contributed to the growing field of trustworthy AI by emphasizing verification-oriented generation, structured output enforcement, and source attribution. These mechanisms improved transparency and helped users understand how generated responses were supported by retrieved evidence [8], [10].

From the software engineering and product perspective, the study demonstrated how advanced AI research concepts could be transformed into a practical software platform. Instead of requiring organizations to build complex RAG systems independently, the proposed platform provided an integrated environment where users could upload documents, organize datasets, create grounded agents, and interact with organizational knowledge through conversational workflows.

The project also received validation and guidance from external stakeholders, including faculty advisors, software engineering professionals, and organizational reviewers involved during the analysis and implementation phases. Feedback obtained from these stakeholders helped refine the system architecture, improve workflow usability, strengthen retrieval reliability, and align the platform with real-world

organizational requirements. Faculty recommendations particularly influenced the emphasis on verification-oriented response generation, source attribution, and practical usability, while external technical reviewers highlighted the importance of retrieval transparency, grounded citations, and scalable workflow orchestration for enterprise-ready AI systems.

The study was significant in the following ways:

- It identified key weaknesses in traditional RAG systems for real-world organizational use.
- It proposed an adaptive RAG architecture designed to improve grounding reliability and retrieval quality.
- It demonstrated how advanced AI retrieval concepts could be integrated into a practical software platform.
- It improved trust in AI-generated outputs through citations, verification, and source attribution.
- It reduced the technical complexity associated with deploying organizational document-intelligence systems.
- It contributed to the development of more reliable, transparent, and usable grounded AI applications.
- It incorporated stakeholder and faculty feedback to align the system with practical organizational and academic expectations.
- It demonstrated the feasibility of transforming advanced AI research concepts into deployable organizational solutions suitable for real-world environments.

1.8 Outline of the Study

This report is organized into seven chapters, each addressing a specific aspect of the study from problem identification to system evaluation and conclusion.

Chapter One: Introduction

Chapter One presents the background of the study, statement of the problem, objectives, scope and limitations, methodology, plan of activities, budget requirements, significance of the study, and the overall organization of the report.

Chapter Two: Literature Review

Chapter Two examines existing literature related to Large Language Models, Retrieval-Augmented Generation, retrieval strategies, reranking methods, verification-oriented systems, and grounded AI architectures. It establishes the theoretical and conceptual foundation of the study.

Chapter Three: Problem Analysis and Modeling

Chapter Three analyzes the limitations of existing systems and identifies the functional and non-functional requirements of the proposed solution. It also presents system models, workflows, and architectural representations of the proposed platform.

Chapter Four: System Design

Chapter Four presents the design of the Advanced Adaptive RAG Pipeline and the grounded AI platform. It explains the system architecture, major components, database structure, retrieval workflow, and integration mechanisms.

Chapter Five: System Implementation

Chapter Five describes the practical implementation of the proposed system, including backend services, frontend interfaces, retrieval orchestration, dataset management, grounded agents, conversational workflows, and deployment infrastructure.

Chapter Six: System Evaluation

Chapter Six presents the testing procedures and evaluation results of both the Advanced Adaptive RAG Pipeline, and the software platform. It discusses retrieval effectiveness, response grounding, workflow correctness, and overall system usability.

Chapter Seven: Conclusions and Recommendations

Chapter Seven summarizes the major findings of the study, highlights the research and product contributions, and provides recommendations for future improvements and further research directions.

Chapter Two: Literature Review

The literature review established the theoretical, technical, and conceptual foundation of this study by examining the evolution of Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), retrieval strategies, verification-oriented architectures, and grounded AI systems. Rather than describing existing approaches in isolation, this chapter critically analyzed their design assumptions, operational strengths, and practical limitations in organizational document intelligence contexts.

Although Retrieval-Augmented Generation improved factual grounding by integrating external knowledge sources into the generation process [1], subsequent research and implementations demonstrated that this improvement remained constrained by several unresolved challenges. Existing RAG systems often failed to consistently ensure retrieval reliability, particularly under ambiguous queries or domain-specific document structures. In many cases, retrieval mechanisms prioritized semantic similarity over evidential correctness, which introduced irrelevant or partially supportive contexts that weakened response accuracy and trustworthiness [5], [6].

In addition, prior studies revealed that many RAG pipelines treated query processing as a static step rather than an adaptive reasoning process. This design limitation reduced system robustness when handling multi-intent, vague, or under-specified user queries, leading to misaligned retrieval outcomes and incomplete responses [16], [17]. Furthermore, while recent approaches attempted to mitigate hallucination through retrieval augmentation, verification layers, or self-reflective mechanisms, these solutions often introduced additional system complexity without fully resolving the underlying dependency between retrieval quality and generation reliability [8], [9], [26].

The literature also indicated a growing shift toward hybrid retrieval, reranking, and agentic architectures to improve retrieval precision and contextual relevance. However, most existing implementations still operated in loosely coupled pipelines where retrieval, ranking, and generation components functioned independently, limiting their ability to adapt dynamically to query difficulty, document structure, or temporal relevance.

This chapter therefore synthesized existing research contributions in retrieval optimization, semantic indexing, reranking strategies, and verification-driven generation to identify persistent gaps in current systems. In particular, it highlighted the lack of fully adaptive retrieval pipelines that integrate query understanding, hybrid retrieval, reranking, temporal awareness, and verification into a unified and coordinated architecture.

The findings from this literature review directly informed the design of the Advanced Adaptive Retrieval-Augmented Generation (RAG) Pipeline and the development of the Grounded AI Platform, which aimed to address these limitations through a more integrated, adaptive, and verification-aware system architecture.

2.1. Study Related Works

2.1.1 Large Language Models

Large Language Models (LLMs) such as GPT, Claude, Gemini, and Llama significantly advanced natural language processing by enabling systems to perform reasoning, summarization, translation, and conversational interaction at scale. However, despite their apparent versatility, research consistently demonstrated that these models remained fundamentally limited in knowledge reliability within domain-specific and organizational environments.

A core limitation identified across studies was the dependence of LLMs on parametric memory, which constrained their ability to access up-to-date, private, or domain-specific information without external augmentation. As a result, LLMs frequently generated fluent but ungrounded outputs, particularly when queries extended beyond their training distribution. These failures were not merely incidental errors but structural limitations of the generative paradigm, where plausibility often replaced factual correctness [1], [12], [26].

Furthermore, hallucination analysis by Ji et al. (2023) showed that errors in LLM outputs were not uniform; intrinsic hallucinations occurred when models contradicted provided context, while extrinsic hallucinations introduced entirely unsupported information [26]. This distinction highlighted a deeper issue: improving fluency alone did not guarantee factual reliability. Consequently, LLMs alone proved insufficient for high-stakes applications requiring traceability, verification, and organizational accountability.

2.1.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) emerged as a direct response to the limitations of standalone LLMs by integrating external retrieval mechanisms into the generation pipeline. While this approach improved factual grounding by conditioning outputs on retrieved documents, research showed that RAG systems did not eliminate reliability issues; instead, they shifted the failure point toward the retrieval stage [1].

Although RAG improved access to external knowledge, its effectiveness remained heavily dependent on the quality of retrieved context. Weak retrieval produced downstream errors even when generation models were highly capable. This dependency exposed a critical structural weakness: RAG systems replaced one uncertainty source (parametric memory) with another (retrieval quality), rather than eliminating uncertainty altogether.

Additionally, many RAG implementations treated retrieval as a static pre-processing step rather than an adaptive reasoning process, which limited their robustness under ambiguous or complex queries [16], [17].

2.1.3 Basic RAG Architecture

Conventional RAG pipelines typically followed a linear sequence of ingestion, chunking, embedding, indexing, retrieval, and generation. While this architecture provided a practical baseline, it assumed that retrieval quality could be sufficiently ensured through embedding similarity alone.

However, empirical findings in retrieval research demonstrated that similarity-based retrieval often failed to guarantee evidence relevance, particularly in domains requiring precise terminology, structured reasoning, or policy-level accuracy. This architectural assumption created a strong dependency between embedding space proximity and factual correctness, which was not consistently valid in real-world document systems.

As a result, naive RAG systems exhibited predictable failure modes when handling ambiguous queries, long-context documents, or heterogeneous corpora, where semantic similarity did not necessarily align with answer correctness [5], [6].

2.1.4 Limitations of Conventional RAG Systems

Pipeline Complexity

Although RAG systems appeared modular in design, practical implementations revealed significant orchestration complexity across ingestion, indexing, retrieval, reranking, and generation stages. These components were often independently designed, which created integration fragility and operational inefficiency in production environments.

This fragmentation indicated a deeper architectural limitation: RAG systems lacked unified coordination mechanisms that could dynamically adapt processing strategies based on query difficulty or retrieval confidence.

Unreliable Retrieval

Conventional RAG systems frequently relied on vector similarity as the primary retrieval mechanism. While effective for semantic matching, this approach systematically underperformed in cases requiring exact matches, structured identifiers, or policy-specific wording.

Lexically irrelevant but semantically similar passages were often prioritized over precise answer-bearing content, which introduced retrieval noise and reduced downstream answer reliability [5], [6]. This mismatch between semantic proximity and factual relevance remained a persistent challenge in production systems.

Query Ambiguity

Most RAG pipelines assumed well-formed queries and failed to incorporate explicit mechanisms for query understanding or transformation. Consequently, ambiguous or multi-intent queries were directly passed into retrieval modules without decomposition or clarification.

This design limitation resulted in retrieval drift, where systems optimized for similarity rather than intent alignment, producing responses that partially addressed user needs but failed to resolve the underlying information requirement [16], [17].

Hallucination and Trust Deficit

Even when retrieval succeeded, generation models could still produce unsupported or partially inferred statements. This revealed that hallucination in RAG systems was not eliminated but redistributed across retrieval and generation stages.

The absence of structured verification mechanisms and explicit evidence tracing further reduced user trust, particularly in domains requiring accountability and traceability [26].

2.1.5 Hybrid Retrieval

Hybrid retrieval approaches attempted to address the limitations of single-method retrieval by combining lexical and semantic strategies. While this approach improved recall and precision balance, research showed that integration strategies were often simplistic and lacked adaptive weighting mechanisms.

Although hybrid retrieval improved robustness, its effectiveness depended heavily on how well lexical and semantic signals were fused. Without proper ranking coordination, hybrid systems could still suffer from retrieval inconsistency and redundancy.

2.1.6 BM25 and Lexical Retrieval

BM25-based retrieval provided strong performance in exact keyword matching and remained highly effective for structured or technical information retrieval tasks. However, its reliance on term frequency and inverse document frequency limited its ability to capture semantic equivalence.

In complex organizational environments, this limitation became evident when relevant documents used paraphrased language or domain-specific variations. Thus, while BM25 remained essential for precision retrieval, it was insufficient as a standalone mechanism for semantic understanding [5].

2.1.7 Semantic Retrieval

Dense embedding-based retrieval improved conceptual matching by encoding meaning beyond lexical overlap. However, this strength introduced a new limitation: semantic similarity did not guarantee evidential correctness.

Research consistently showed that embedding-based systems could retrieve contextually related but non-answer-bearing passages, particularly in multi-topic or densely structured documents. This limitation reduced precision in domains requiring factual exactness.

2.1.8 Reciprocal Rank Fusion

Reciprocal Rank Fusion (RRF) addressed retrieval fusion challenges by combining ranked outputs from multiple retrieval systems without relying on score normalization. While this improved robustness in hybrid retrieval settings, it remained agnostic to query intent and document structure.

As a result, RRF improved ranking stability but did not solve deeper issues related to query understanding or contextual relevance.

2.1.9 Reranking Techniques

Reranking mechanisms improved retrieval precision by applying deeper query-document interaction models after initial retrieval. Cross-encoder models demonstrated significant improvements in filtering irrelevant or weakly related passages.

However, reranking introduced additional computational overhead and still depended on the quality of initial retrieval candidates. This created a dependency chain where early-stage retrieval errors could not always be fully corrected downstream [10].

2.1.10 Query Transformation

Query transformation techniques improved retrieval alignment by reformulating or decomposing ambiguous queries before retrieval. Research showed that retrieval systems performed significantly better when queries were clarified or expanded prior to embedding.

However, most existing approaches treated query transformation as optional preprocessing rather than an integrated reasoning stage, limiting its effectiveness in dynamic conversational settings [16], [17].

2.1.11 Semantic and Adaptive Chunking

Traditional chunking strategies introduced structural fragmentation by dividing documents into fixed-size segments without regard to semantic boundaries. This often disrupted contextual continuity and reduced retrieval accuracy.

Adaptive chunking methods addressed this limitation by preserving semantic coherence during segmentation. However, many implementations lacked integration with retrieval strategies, resulting in improvements at indexing level without fully optimizing end-to-end retrieval performance [28].

2.1.12 Temporal Ranking and Freshness-Aware Retrieval

Conventional retrieval systems typically ignored temporal relevance, leading to outdated or obsolete information being retrieved in dynamic environments. This limitation was particularly critical in organizational systems where policies and procedures evolve over time.

Temporal ranking approaches introduced freshness-aware scoring, improving retrieval relevance in time-sensitive contexts. However, most systems treated temporal signals as secondary ranking features rather than core retrieval constraints [27].

2.1.13 Corrective Retrieval-Augmented Generation

Corrective RAG frameworks introduced feedback loops that evaluated retrieval quality before generation. While this approach improved robustness, it also introduced additional system complexity and latency.

More importantly, corrective mechanisms often depended on heuristic thresholds, which limited their adaptability across different domains and query types [8].

2.1.14 Verification-Oriented Generation

Verification-oriented systems attempted to reduce hallucination by evaluating generated outputs against retrieved evidence. Although these systems improved trustworthiness, they remained limited by imperfect evidence alignment and incomplete retrieval coverage.

This indicated that verification alone could not compensate for weak retrieval, reinforcing the dependency between retrieval quality and generation reliability [8], [9].

2.1.15 Structured Output and Source Attribution

Structured output formats and citation mechanisms improved transparency in generated responses by enabling traceability of claims. However, many systems treated attribution as a post-processing step rather than a core architectural requirement.

This reduced the reliability of citations, particularly when retrieval quality was weak or inconsistent.

2.1.16 Agent-Oriented Grounded AI Systems

Agent-based architectures introduced modular intelligence systems capable of handling domain-specific workflows and dataset interactions. While these systems improved flexibility, they often lacked unified retrieval coordination, resulting in fragmented reasoning pipelines.

This fragmentation limited their effectiveness in structured organizational environments requiring consistent grounding and traceability.

2.1.17 Productized Document Intelligence Platforms

Recent industry systems demonstrated that retrieval pipelines alone were insufficient for real-world adoption. Effective solutions required integrated platforms that combined ingestion, retrieval, interaction, monitoring, and access control.

However, most existing platforms prioritized usability over retrieval intelligence or vice versa, creating a gap between research-grade RAG systems and production-ready organizational tools.

2.2. Identifying Milestones and Finding the Gaps

2.2.1 Emergence of Standalone LLM Interfaces

The first major milestone in the evolution of language-based AI systems was the introduction of standalone Large Language Model (LLM) interfaces capable of natural conversational interaction. These systems demonstrated a significant shift in human–computer interaction by enabling users to access computational capabilities through natural language rather than structured commands.

However, despite their fluency and general-purpose reasoning ability, standalone LLMs remained fundamentally limited in terms of factual grounding and external knowledge access. Since these systems relied exclusively on parametric memory, they were unable to incorporate private organizational data or reflect recent updates in external environments. This limitation resulted in hallucinated or outdated responses, particularly in knowledge-intensive and domain-specific tasks, thereby constraining their applicability in organizational contexts.

2.2.2 Emergence of Retrieval-Augmented Generation

The introduction of Retrieval-Augmented Generation (RAG) represented a significant architectural shift aimed at addressing the grounding limitations of standalone LLMs. By integrating external retrieval mechanisms into the generation process, RAG systems improved factual accuracy and enabled models to condition responses on retrieved evidence [1].

Despite this improvement, early RAG implementations still retained structural weaknesses. Retrieval was predominantly based on static similarity search mechanisms, which assumed that semantic proximity equated to factual relevance. This assumption often failed in complex document environments where relevant evidence required exact matching, contextual interpretation, or domain-specific understanding. As a result, early RAG systems improved access to external knowledge but did not fundamentally resolve retrieval unreliability or ambiguity sensitivity.

2.2.3 Hybrid Retrieval and Reranking

A subsequent milestone involved the introduction of hybrid retrieval and reranking mechanisms, which aimed to improve evidence quality by combining lexical precision with semantic understanding. Hybrid retrieval reduced dependence on a single retrieval paradigm, while reranking techniques refined candidate selection by re-evaluating retrieved passages using more expressive models [5], [10].

However, while these approaches improved retrieval robustness, they largely operated as independent optimization layers rather than as part of a unified adaptive reasoning system. This resulted in improved ranking quality without addressing deeper issues such as query misinterpretation, retrieval strategy selection, or contextual adaptation.

2.2.4 Adaptive and Corrective Retrieval Architectures

More recent developments introduced adaptive retrieval paradigms that incorporated query transformation, corrective retrieval loops, temporal awareness, and self-evaluation mechanisms [8], [27]. These systems marked an important transition from static retrieval pipelines toward more dynamic and feedback-driven architectures.

Although these approaches improved system resilience under weak or ambiguous retrieval conditions, most implementations remained partially decoupled, with adaptive components functioning as add-on mechanisms rather than fully integrated pipeline behaviors. This limited their ability to consistently coordinate retrieval, ranking, and verification in a unified decision process.

2.2.5 Trustworthy and Productized Grounded AI Systems

The most recent milestone reflects a shift from research-focused retrieval systems toward productized grounded AI platforms that emphasize transparency, traceability, and usability in real-world environments. These systems integrate retrieval pipelines with user-facing interfaces, dataset management, and inspection workflows to support organizational adoption.

However, despite these advancements, a clear gap persists between experimental RAG architectures and fully deployable systems. Many implementations prioritize either retrieval performance or user experience, but rarely achieve both within a unified, scalable, and verifiable architecture.

2.2.6 Research Gaps Identified from the Literature

A synthesis of existing literature and system implementations reveals several persistent gaps that remain insufficiently addressed.

First, many existing systems still rely on naive or partially optimized RAG pipelines without fully resolving pipeline coordination challenges across retrieval, ranking, and generation stages. This leads to fragmented architectures where improvements in one stage do not reliably propagate across the system.

Second, most retrieval frameworks continue to depend excessively on a single retrieval strategy, limiting robustness in heterogeneous document environments where lexical precision and semantic similarity must be jointly considered.

Third, query understanding remains underdeveloped in many systems, as ambiguous, multi-intent, or underspecified queries are often passed directly into retrieval modules without sufficient transformation or decomposition.

Fourth, although verification and citation mechanisms have been introduced in recent research, they are not consistently integrated into end-to-end generation pipelines, resulting in partial transparency rather than full traceability of generated outputs.

Fifth, a significant portion of research remains focused on improving isolated components of RAG systems rather than addressing the translation of these improvements into usable, production-grade organizational platforms.

Finally, a gap exists between high-performing experimental systems and practical grounded AI platforms that combine adaptive retrieval, verification mechanisms, user interaction workflows, and system transparency within a unified architecture.

Collectively, these gaps highlight the need for a more integrated, adaptive, and verification-aware retrieval architecture. This study addressed these limitations through the development of the Advanced Adaptive Retrieval-Augmented Generation (RAG) Pipeline and its implementation within the Grounded AI Platform, which combined retrieval intelligence with practical system usability for organizational deployment.

2.3. Lessons Learned from the Literature

The analysis of existing literature revealed several interconnected insights that collectively shaped the design decisions of this study and highlighted the limitations of current Retrieval-Augmented Generation systems in organizational contexts.

A primary insight was that basic Retrieval-Augmented Generation architectures remained insufficient for reliable organizational deployment. Although RAG improved factual grounding compared to standalone language models, most naive implementations continued to exhibit structural weaknesses, including unstable retrieval behavior, hallucinated outputs, and limited verification capability. This indicated that grounding alone was not sufficient unless supported by stronger retrieval control and validation mechanisms.

A second key insight was that retrieval quality directly constrained the overall quality of generated responses. Across multiple studies, poor retrieval was consistently identified as the root cause of incomplete, irrelevant, or misleading outputs, regardless of the strength of the underlying language model. This dependency emphasized the importance of improving retrieval mechanisms through hybrid retrieval strategies, reranking models, semantic chunking, and temporal awareness, rather than relying solely on generative improvements.

A third important finding was that query ambiguity represented a critical failure point in most retrieval pipelines. Research demonstrated that vague, underspecified, or multi-intent queries frequently resulted in misaligned retrieval outcomes when processed without transformation or clarification. This limitation highlighted the necessity of incorporating query transformation and retrieval planning mechanisms as integral components of the system rather than optional preprocessing steps.

A fourth insight was that generation-focused architectures without verification mechanisms were inherently unreliable in high-stakes environments. The literature consistently showed that hallucination could not be fully eliminated through retrieval alone. Instead, trustworthiness required additional mechanisms such as verification loops, evidence alignment, critique-based evaluation, and explicit source attribution. These components collectively improved transparency and reduced unsupported generation.

A fifth lesson was that technical retrieval improvements alone were insufficient for real-world adoption. Organizational environments required complete systems that integrated retrieval intelligence with usable software workflows, including document ingestion, dataset management, conversational interfaces, inspection tools, and access control mechanisms. This demonstrated that usability and system integration were as important as algorithmic performance.

Overall, the literature indicated that effective grounded AI systems required the convergence of multiple research directions, including retrieval optimization, adaptive query processing, verification-driven generation, and practical system engineering. These combined insights directly informed the design of the Advanced Adaptive Retrieval-Augmented Generation (RAG) Pipeline and its implementation within the Grounded AI Platform, which addressed both the technical and operational limitations identified in existing work.

Chapter Three: Problem Analysis and Modeling

Chapter Three focuses on the systematic analysis and modeling of the problem addressed by this study. While the previous chapters established the motivation and objectives of the project, this chapter translates those motivations into a concrete technical understanding of the existing system and its deficiencies. The purpose of this chapter is to examine how current Retrieval-Augmented Generation (RAG) systems operate in practice, identify their limitations, and analyze the underlying causes of these limitations from both technical and stakeholder perspectives. Through this analysis, the chapter establishes a clear rationale for the proposed solution and defines the requirements that the new system must satisfy. [1]

Problem analysis and modeling play a critical role in bridging the gap between abstract challenges, such as hallucinations and unreliable AI outputs, and practical system-level improvements. By decomposing the existing RAG workflow into its functional components, it becomes possible to understand how design decisions related to document processing, retrieval strategies, and response generation directly contribute to errors, inconsistencies, and loss of trust. This chapter therefore provides the analytical foundation upon which the proposed Grounded framework is designed.

3.1 Existing System and Its Problems

The existing system examined in this study is the commonly deployed form of Retrieval-Augmented Generation that is widely used in academic prototypes and industry applications. In its current form, this system follows a largely static and linear pipeline in which documents are ingested, transformed into vector embeddings, retrieved based on similarity to a user query, and passed directly to a large language model for answer generation. Although this approach represents a significant improvement over standalone language models, it remains insufficient for professional and high-stakes environments. [1]

In the existing system, documents are typically preprocessed by converting them into plain text and dividing them into fixed-size segments. These segments are embedded into a vector space using a general-purpose embedding model and stored in a vector database. When a user submits a query, the system retrieves a small number of segments that are most similar to the query based on a distance metric such as cosine similarity. The retrieved segments are then provided to the language model as contextual input, and the model generates a response without further validation or verification.

While this workflow is computationally efficient and easy to implement, it suffers from several fundamental problems. One major limitation is the loss of semantic structure during document preprocessing. Fixed-size segmentation often breaks logical units such as definitions, conditions, or conclusions across multiple segments. As a result, retrieved content may lack critical context, leading the language model to generate incomplete or misleading answers. This problem is particularly severe in domains such as medicine and law, where meaning is highly dependent on precise phrasing and contextual qualifiers.

Another significant issue lies in the retrieval mechanism itself. Similarity-based retrieval assumes that semantic proximity in vector space corresponds directly to factual relevance. In practice, this assumption

does not always hold. Documents that share surface-level terminology with a query may be retrieved even when they do not contain the information required to answer the user’s question. Conversely, highly relevant documents may be overlooked if they use different terminology or are phrased in a more technical or concise manner. This leads to low precision in retrieved contexts and increases the cognitive burden on the language model.

Temporal relevance further complicates the behavior of the existing system. In rapidly evolving fields, older documents often remain highly similar to new queries due to shared terminology, even though their content may be outdated or superseded. Because most current RAG systems do not explicitly account for document recency or versioning, they may retrieve obsolete regulations, deprecated technical documentation, or outdated clinical guidelines. When such information is presented to a language model without verification, it can result in authoritative-sounding but incorrect outputs.

Query ambiguity represents another persistent challenge. Users frequently submit vague or underspecified queries that require interpretation before meaningful retrieval can occur. Existing systems typically treat the initial user query as final and perform retrieval without attempting to clarify intent or reformulate the query. As a result, the system retrieves a broad and unfocused set of documents, leading to generic responses that fail to meet user expectations. This issue undermines user trust and limits the usefulness of RAG systems in professional workflows.

The impact of these limitations is felt by multiple stakeholders. Developers often struggle to improve system performance through manual parameter tuning, such as adjusting chunk sizes or retrieval thresholds, without addressing the root causes of failure. Organizations face risks related to data reliability and compliance when incorrect or outdated information is surfaced by the system. End-users experience frustration when responses appear confident but lack factual accuracy or practical relevance. Together, these issues highlight the inadequacy of existing RAG systems for environments where correctness, transparency, and accountability are essential.

3.2 Specifying the Requirements of the Proposed Solution

The analysis of the existing system reveals the need for a more adaptive, reliable, and transparent approach to Retrieval-Augmented Generation. In order to address the identified shortcomings, the proposed solution must satisfy a set of clearly defined requirements derived from both stakeholder needs and technical constraints. These requirements were identified through a combination of literature review, analysis of existing RAG deployments, and examination of use cases in knowledge-intensive domains. [1]

From a functional perspective, the proposed system must support document ingestion mechanisms that preserve semantic structure and contextual integrity. Rather than relying on rigid, fixed-size segmentation, the system should be capable of dividing documents along meaningful semantic boundaries to ensure that retrieved information remains coherent and complete. This requirement directly addresses the issue of contextual fragmentation observed in the existing system.

The retrieval component of the proposed solution must go beyond single-stage similarity search. It should incorporate mechanisms for refining and reranking retrieved content to improve precision and relevance.

This includes the ability to evaluate retrieved documents at a deeper contextual level, ensuring that the most useful and trustworthy information is prioritized before being passed to the language model. Such capabilities are essential for reducing noise and mitigating the risk of hallucinations.

Temporal awareness is another critical requirement. The system must be capable of accounting for the recency and validity of documents during retrieval, particularly in domains where information evolves over time. By incorporating temporal considerations into the retrieval process, the proposed solution can reduce the likelihood of outdated or superseded information influencing the generated response.

In addition, the proposed system must address query ambiguity through adaptive query handling. This involves interpreting user intent, reformulating or expanding queries when necessary, and ensuring that retrieval strategies are aligned with the underlying information needed. Such functionality improves response specificity and enhances user satisfaction.

Non-functional requirements are equally important. The system must be transparent in its operation, allowing users and developers to understand how retrieved information contributes to the final output. It should also be modular and extensible, enabling future enhancements without requiring a complete redesign. Finally, the solution must be deployable in a practical manner, supporting local or controlled environments to ensure data privacy and organizational compliance.

By clearly defining these requirements, this chapter establishes the foundation for the design and implementation of the proposed Grounded framework. The following chapter builds upon this analysis to present the system architecture and modeling approach used to realize these requirements in a coherent and scalable solution.

3.3. System Modeling

System modeling is the process of developing abstract representations of the **Grounded** framework to visualize, specify, and document its architecture and behavior. In this study, modeling is utilized as a formal validation tool to ensure that the 11-stage pipeline is structurally sound and that the complex interactions between autonomous agents are logically consistent. By employing a variety of **Unified Modeling Language (UML)** and architectural diagrams, this section provides a comprehensive roadmap of the system's internal mechanics, ensuring that every functional requirement from recursive chunking to agentic verification is accurately mapped to a technical component.

3.3.1. Functional Requirements (FR)

The functional requirements define the core behaviors and technical processes the Grounded system must perform to bridge the "Expertise-Reliability Gap." These requirements are mapped directly to the 11-stage pipeline architecture.

FR-1: Agentic Intent Decomposition and Query Transformation

The system shall not process user input as a static search string. It must implement an autonomous "Intent Agent" to analyze the query for complexity. If a query contains multiple facets (e.g., "Analyze the legal precedents of X vs. Y in relation to Z"), the system must decompose it into N distinct sub-queries, execute parallel searches, and synthesize the results to eliminate the "Intent Gap."

FR-2: Recursive Semantic Chunking Engine

The ingestion engine must replace arbitrary character-limit splitting with context-aware partitioning. The system shall utilize embedding similarity thresholds to identify "Topic Shifts." A document chunk will only be "broken" when the semantic meaning changes, ensuring that dense logical statements, such as medical dosages or legal clauses, remain contextually intact.

FR-3: Hybrid Retrieval and Reciprocal Rank Fusion (RRF)

To ensure high recall of both technical jargon and conceptual themes, the system shall implement a dual-path retrieval strategy. It must simultaneously execute a **BM25 (Sparse)** search for exact keyword matches and a **Vector (Dense)** search for semantic meaning. These results must be unified using the RRF algorithm to generate a single, high-confidence candidate list.

FR-4: Neural Reranking and Distillation Layer

The system must act as a precision filter to remove "noise" from initial retrieval. The top candidate documents must be passed through a **Cross-Encoder (BGE Reranker)** to evaluate the deep query-document relationship. The system shall distill the results down to the most factually salient passages (typically the top 5) before passing them to the generator.

FR-5: Temporal Decay and Metadata Scoring

The system must prioritize the most recent information in fast-moving industries. It shall apply an **Exponential Decay Function** to document relevance scores based on "Created_At" metadata. This ensures that current regulations or recent clinical findings are ranked significantly higher than superseded historical data.

FR-6: Corrective RAG (CRAG) and Web-Search Fallback

The system shall maintain "self-awareness" of its internal knowledge limitations. If the retrieval evaluator assigns a confidence score below a predefined threshold (e.g., 0.6), the system must trigger an automated web-search fallback (via APIs like Tavily) to supplement the internal knowledge base and prevent "I don't know" responses or hallucinations.

FR-7: Agentic Verification (The Critic Agent)

Every generated response must undergo a formal audit phase. A secondary "Critic Agent" shall cross-reference the draft claims against the source metadata. If a claim is not explicitly supported by the retrieved context, the response must be rejected and sent back through a "refinement loop" until 100% grounding is achieved.

FR-8: Structured Citation and Pydantic Enforcement

The system shall physically constrain the LLM to return data in a strict JSON format using Pydantic models. Every response must contain three mandatory components: the synthesized answer, a verbatim "source_quote," and the specific metadata (document UUID and page number) to ensure the output is fully auditable by human experts.

3.3.2. Non-Functional Requirements (NFR)

Non-functional requirements specify the quality attributes and operational constraints necessary to make the system safe and performant for professional deployment.

NFR-1: Data Sovereignty and Privacy (Docker-Native Execution)

To safeguard sensitive medical and legal information, the system must support full on-premise deployment. It shall be capable of running entirely within a local **Docker** container, ensuring that no user-uploaded data or internal document embeddings are transmitted to external servers, satisfying strict data residency regulations.

NFR-2: Adaptive Latency and Tiered Performance

The system must balance complex 11-stage processing with user experience. It shall implement an **Adaptive Router** that assigns processing tiers based on query risk. "Standard" tier queries (e.g., simple FAQs) must return results in under 5 seconds, while "Critical" tier queries involving full agentic verification must complete within 15 seconds.

NFR-3: Multi-Tenant Security (Namespace Isolation)

The system must provide mathematical isolation of data between different users or organizations. All vector database operations (indexing and retrieval) must be strictly partitioned using **Namespace Metadata**. The system must automatically apply a user_id filter to every query to prevent cross-tenant data leakage.

NFR-4: Scalability and Throughput The underlying vector infrastructure (Qdrant or Pinecone) must support horizontal scaling. The system shall maintain sub-second retrieval latency even when the knowledge base scales to 1,000,000 document chunks, ensuring it can grow with the organization's data needs.

NFR-5: Explainability and Auditability (The "Trace View")

The management dashboard must provide a transparent audit trail for every query. Administrators must be able to view the "Thought Chain" of the agent, including which documents were retrieved, their reranking scores, and the specific reason the **Critic Agent** approved the final response.

NFR-6: Fault Tolerance and Model Redundancy

The system must ensure high availability by managing API dependencies gracefully. If the primary LLM provider (e.g., OpenAI) is unreachable, the system must support an automatic failover to a secondary provider or a local open-source model (e.g., Llama-3 via Ollama) to maintain service continuity.

Summary of Requirements Mapping

Failure in Existing System	Required Feature in Grounded	Requirement ID
Hallucinations	Agentic Verification Loop	FR-7
Outdated Info	Temporal Decay Scoring	FR-5
Vague Queries	Intent Decomposition	FR-1
Split Sentences	Recursive Semantic Chunking	FR-2
Data Privacy Risk	Local Docker Deployment	NFR-1
Irrelevant Search	Hybrid Search + Reranking	FR-3, FR-4

Table 3.1 Requirement Mapping

3.3.3. Usecase

Use case modeling describes how the proposed Grounded platform interacts with its users at a high level. The aim is to capture what the system should do from the perspective of external actors, without exposing internal technical components such as retrievers, rerankers, or verification models.

In this project, the primary human actor is the Platform User, which includes Developers, Enterprise Clients, and Legal or Medical Operators who interact with the system for data upload, querying, monitoring, and integration. Two supporting actors are also considered: the Authentication Service that manages secure access, and an optional External Web Source that is used when the system needs to supplement internal knowledge with fresh web information.

Identified Actors

- **Actor A1: Platform User**
Represents developers and professional end users who interact with the system for data upload, query, monitoring, and integration.
- **Actor A2: Authentication Service**
Handles user login, credential validation, and session management.

Use Case Identification Table

Use Case ID	Use Case Name	Primary Actor	Supporting Actor(s)	Brief Description
UC01	Submit Query	End User	Developer / API Integrator	Submits a query to initiate the RAG pipeline.
UC02	Agentic Query Expansion and Multi-Step Decomposition	External LLM Model	Vector Database	Expands or decomposes complex queries into sub-queries.
UC03	Execute Recursive Semantic Chunking	System (Internal)	Vector Database	Splits documents into semantically coherent segments.
UC04	Hybrid Retrieval (BM25 + Dense Search)	System (Internal)	Vector Database	Performs combined sparse and dense retrieval.
UC05	BGE Neural Reranking	System (Internal)	Vector Database	Reranks retrieved candidates using neural cross-encoder.
UC06	Apply Temporal Scoring and Time-Decay Logic	System (Internal)	Vector Database	Scores documents based on temporal relevance.
UC07	Agentic Verification (Generator-Critic Loop)	External LLM Model	System (Internal)	Verifies groundedness of retrieved information.

UC08	Generate Grounded Response	External LLM Model	System (Internal)	Produces grounded answer using verified context.
UC09	Provide Final Output to User	End User	Developer / API Integrator	Displays grounded answer to the user.
UC10	Manage Data Sources (Upload PDFs, DOCX, TXT)	Admin	None	Admin uploads structured or unstructured documents for indexing.
UC11	Configure System Parameters	Admin	System (Internal)	Admin configures retrieval and pipeline parameters.
UC12	Monitor System Logs & Pipeline Performance	Admin	System (Internal)	Admin reviews performance metrics and logs.
UC13	Obtain API Key & Integrate RAG API	Developer / API Integrator	System (Internal)	Developer generates API key and integrates Grounded through API.

Use Case 01: Submit Query

Element	Description
Use Case ID	UC1
Use Case Name	Submit Query
Actor	End User
Supporting Actor	Developer / API Integrator (when the query is sent via API instead of UI)

Description	The End User submits a query to the Grounded Adaptive RAG System to initiate the retrieval augmented reasoning process.
Precondition	The system is running and reachable. At least one data source has been registered and indexed.
Flow of Events	1. End User opens the application or client interface. 2. End User types or provides a query. 3. End User submits the query. 4. System receives the query and forwards it to the internal pipeline.
Alternate Flows	If the query is empty or invalid, the system displays an error message and prompts the user to provide a valid query.
Post Condition	A valid query is registered in the system and passed to the next stage (UC2).

Use Case 02: Agentic Query Expansion and Multi-Step Decomposition

Element	Description
Use Case ID	UC2
Use Case Name	Agentic Query Expansion and Multi-Step Decomposition
Actor	External LLM Model
Description	The External LLM Model expands or decomposes the original query into refined or multiple sub-queries to improve retrieval coverage.
Precondition	A valid query has been submitted in UC1 and passed to the planning layer.

Flow of Events	1. System sends the user query to the External LLM Model. 2. Model analyzes intent and complexity. 3. Model generates expanded or decomposed sub-queries. 4. System collects the sub-queries and forwards them to retrieval stages.
Alternate Flows	If the query is simple and does not require decomposition, the system can skip this step and move directly to retrieval (UC3/UC4).
Post Condition	A set of one or more refined sub-queries is available for downstream semantic processing and retrieval.

Use Case 03: Execute Recursive Semantic Chunking

Element	Description
Use Case ID	UC3
Use Case Name	Execute Recursive Semantic Chunking
Actor	Vector Database
Description	The system prepares documents by splitting them into semantically coherent chunks before they are stored in the Vector Database.
Precondition	Data sources have been uploaded by Admin and are available for preprocessing.
Flow of Events	1. System loads raw documents from storage. 2. System applies semantic similarity based chunking to identify logical boundaries. 3. Chunks are created and associated with metadata. 4. Chunks are stored in the Vector Database for later retrieval.
Alternate Flows	If semantic chunking fails, the system falls back to rule based or fixed size chunking.

Post Condition	All relevant documents are stored as semantically meaningful chunks in the Vector Database.
----------------	---

Use Case 04: Hybrid Retrieval (BM25 + Dense Search)

Element	Description
Use Case ID	UC4
Use Case Name	Hybrid Retrieval (BM25 + Dense Search)
Actor	Vector Database
Description	The system uses both keyword based (BM25) and dense vector based search to retrieve candidate chunks relevant to a query or sub-query.
Precondition	Queries or sub-queries have been prepared (UC1 and UC2). Documents have been indexed in the Vector Database (UC3).
Flow of Events	1. System sends query or sub-queries to the Vector Database. 2. Database performs BM25 keyword search. 3. Database performs dense vector similarity search. 4. Database returns combined result sets to the system.
Alternate Flows	If one retrieval path is temporarily unavailable, the system may rely on the other path and log the incident for later review.
Post Condition	A merged list of candidate chunks is returned from the Vector Database and forwarded to reranking (UC5).

Use Case 05: BGE Neural Reranking

Element	Description
Use Case ID	UC5

Use Case Name	BGE Neural Reranking
Actor	Vector Database
Description	Retrieved candidate chunks are reranked using a BGE based neural cross encoder to prioritize the most relevant passages.
Precondition	A list of candidate chunks has been retrieved in UC4.
Flow of Events	1. System collects candidate chunks from the Vector Database. 2. System feeds query and chunks into the BGE reranker. 3. Reranker assigns relevance scores to each chunk. 4. System sorts and selects the top ranked chunks.
Alternate Flows	If the reranking model is unavailable, the system uses the original hybrid ranking and logs the event.
Post Condition	A refined list of top ranked chunks is available for temporal scoring and verification (UC6 and UC7).

Use Case 06: Apply Temporal Scoring and Time-Decay Logic

Element	Description
Use Case ID	UC6
Use Case Name	Apply Temporal Scoring and Time-Decay Logic
Actor	Vector Database
Description	The system adjusts the relevance of candidate chunks using time based scoring to favor more recent and authoritative documents.
Precondition	Candidate chunks with base relevance scores exist from UC5. Metadata such as timestamps are available.

Flow of Events	1. System reads timestamps from chunk metadata. 2. System applies time decay formulas to compute adjusted scores. 3. Chunks are reranked according to combined semantic and temporal scores. 4. Top chunks are selected and passed to verification (UC7).
Alternate Flows	If timestamp metadata is missing or incomplete, the system uses only semantic scores and flags affected chunks.
Post Condition	A final time aware set of evidence chunks is ready for agentic verification (UC7).

Use Case 07:Agentic Verification (Generator-Critic Loop)

Element	Description
Use Case ID	UC7
Use Case Name	Agentic Verification (Generator-Critic Loop)
Actor	External LLM Model
Description	The External LLM Model acts as a critic to verify whether the candidate evidence supports the potential answer and detects hallucinations.
Precondition	A set of high ranking evidence chunks is available from UC5 and UC6.
Flow of Events	1. System sends evidence and draft reasoning to the External LLM Model acting as a critic. 2. Model checks groundedness, consistency, and conflicts. 3. Model flags unsupported claims and suggests corrections or additional retrieval if needed. 4. System either accepts the verification or loops back to retrieval if evidence is insufficient.
Alternate Flows	If verification confidence is low, the system may trigger additional retrieval or escalate to web fallback logic before proceeding.

Post Condition	Verified and approved evidence is available for final answer generation (UC8).
----------------	--

Use Case 08: Generate Grounded Response

Element	Description
Use Case ID	UC8
Use Case Name	Generate Grounded Response
Actor	External LLM Model
Description	The External LLM Model generates a natural language answer that is explicitly grounded in the verified evidence chunks.
Precondition	Evidence chunks have been verified in UC7 and deemed sufficient.
Flow of Events	1. System sends verified evidence and the original query to the External LLM Model. 2. Model generates an answer that cites or references the evidence. 3. System structures the output (for example JSON with citations and confidence scores).
Alternate Flows	If the model is unable to generate a confident answer, the system informs the user that more specific input or additional data is required.
Post Condition	A grounded and structured response is produced and ready to be returned to the user in UC9.

Use Case 09: Provide Final Output to User

Element	Description
---------	-------------

Use Case ID	UC9
Use Case Name	Provide Final Output to User
Actor	End User
Supporting Actor	Developer / API Integrator (when the output is consumed via an external client application)
Description	The system returns the grounded response to the End User, either through the user interface or through an integrated external application.
Precondition	A grounded response has been generated in UC8.
Flow of Events	1. System formats the answer according to the chosen channel (UI or API). 2. System sends or displays the response to the user or client. 3. User reviews the answer and may decide to submit a follow up query.
Alternate Flows	If message delivery fails (for example network error), the system logs the error and may retry or notify the user.
Post Condition	The End User has received the final grounded response for the submitted query.

Use Case 10: Manage Data Sources

Element	Description
Use Case ID	UC10
Use Case Name	Manage Data Sources
Actor	Admin

Description	The Admin manages data sources such as PDFs, DOCX, and TXT files that will be ingested into the RAG system.
Precondition	Admin is authenticated and authorized to manage data sources.
Flow of Events	1. Admin logs into the admin interface. 2. Admin navigates to the "Data Sources" section. 3. Admin uploads, updates, or removes documents. 4. System processes changes and updates the index.
Alternate Flows	If upload or deletion fails, the system displays a message and logs the failure.
Post Condition	The set of active data sources is updated and will be used in future preprocessing and retrieval.

Use Case 11: Configure System Parameters

Element	Description
Use Case ID	UC11
Use Case Name	Configure System Parameters
Actor	Admin
Description	The Admin configures system parameters such as retrieval depth, reranking options, temporal decay settings, and verification thresholds.
Precondition	Admin is authenticated and has configuration privileges.
Flow of Events	1. Admin opens the "Settings" or "Configuration" section. 2. Admin views current pipeline parameters. 3. Admin modifies values such as top-k retrieval size, decay constants, or verification thresholds. 4. Admin saves the new configuration.

Alternate Flows	If configuration changes are invalid, the system rejects the update and shows validation messages.
Post Condition	New system parameters are stored and will be applied in subsequent RAG pipeline executions.

Use Case 12: Monitor System Logs and Pipeline Performance

Element	Description
Use Case ID	UC12
Use Case Name	Monitor System Logs and Pipeline Performance
Actor	Admin
Description	The Admin monitors system health, logs, and performance metrics to understand how well the RAG pipeline is functioning.
Precondition	Admin is authenticated and monitoring features are enabled.
Flow of Events	1. Admin navigates to the "Logs" or "Monitoring" section. 2. System displays metrics such as query volume, latency, error rates, and verification statistics. 3. Admin may filter or drill down into specific time ranges or events.
Alternate Flows	If metrics or logs fail to load, the system shows an error and suggests retrying or checking system status.
Post Condition	Admin has an updated understanding of pipeline performance, which can inform tuning or troubleshooting.

Use Case 13: Obtain API Key and Integrate RAG API

Element	Description
---------	-------------

Use Case ID	UC13
Use Case Name	Obtain API Key and Integrate RAG API
Actor	Developer / API Integrator
Description	The Developer or API Integrator obtains an API key and configures an external application to communicate with the Grounded Adaptive RAG API.
Precondition	Developer has an account with the required permissions.
Flow of Events	1. Developer logs into the platform. 2. Developer navigates to the "API Keys" or "Developer" section. 3. Developer generates or retrieves an API key. 4. Developer configures the client application with the base URL and API key. 5. Client application sends test requests to confirm integration.
Alternate Flows	If key generation fails or integration tests fail, the system shows error details so the Developer can correct configuration.
Post Condition	The external application is integrated with the RAG API and can submit queries and receive grounded responses.

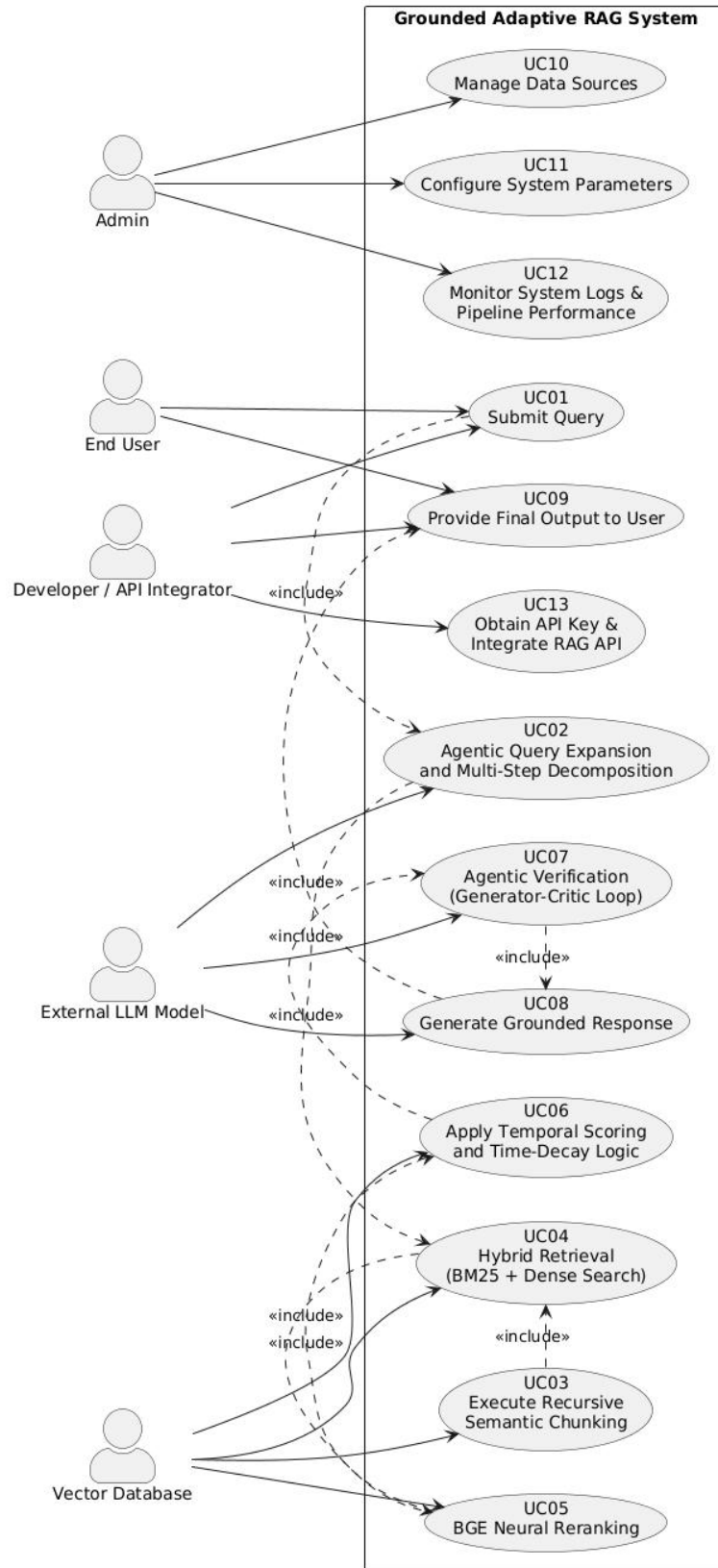


Fig 3.3.1. Use Case Diagram

3.3.4. Dynamic Models of a System

Dynamic models describe how different system components interact during runtime.

3.3.4.1. Sequence Diagram: Document Upload and Processing Flow

This diagram shows the interaction between user, API, queue, worker, vector database, and file storage when a document is uploaded.

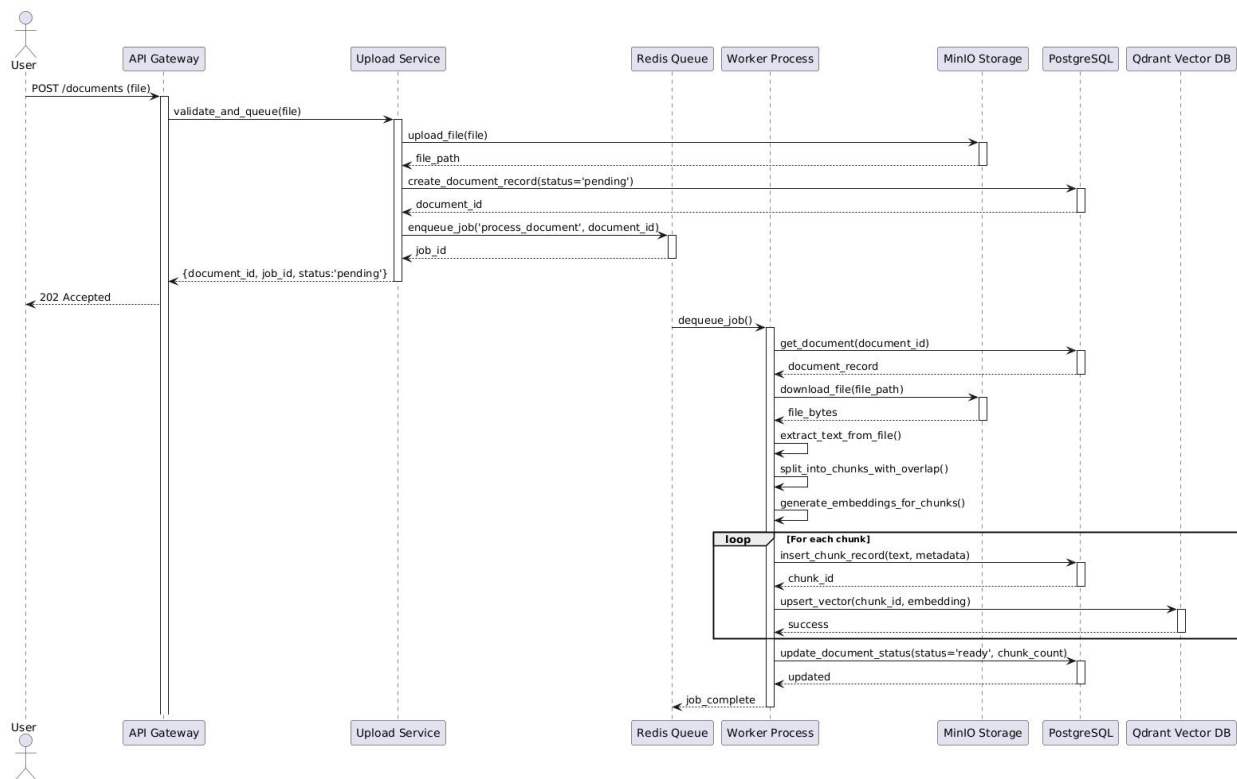


Fig 3.3.4.1. Sequence Diagram: Document Upload and Processing Flow

3.3.4.2. Sequence Diagram: Query Execution Flow

This diagram shows the complete query flow including routing, hybrid search, reranking, generation, and logging.

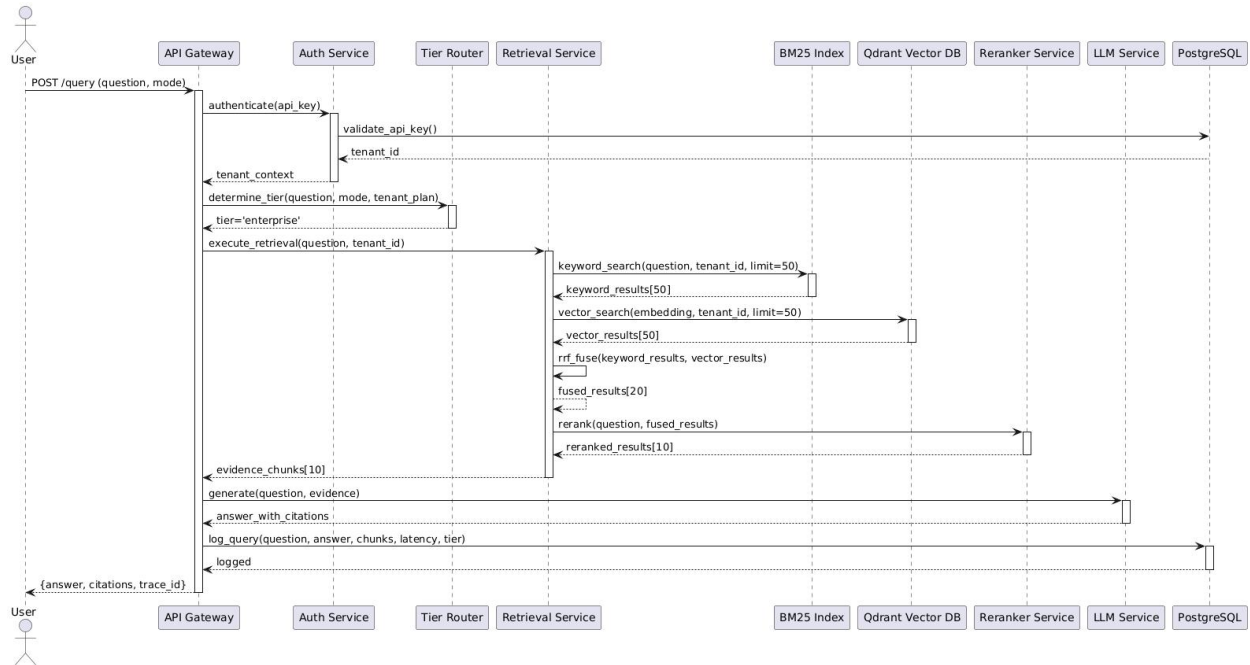


Fig 3.3.4.2. Sequence Diagram: Query Execution Flow

3.3.4.3. State Machine Diagram: Document Processing States

This diagram shows how a document transitions through states from upload to ready for querying, including failure states and retry logic.

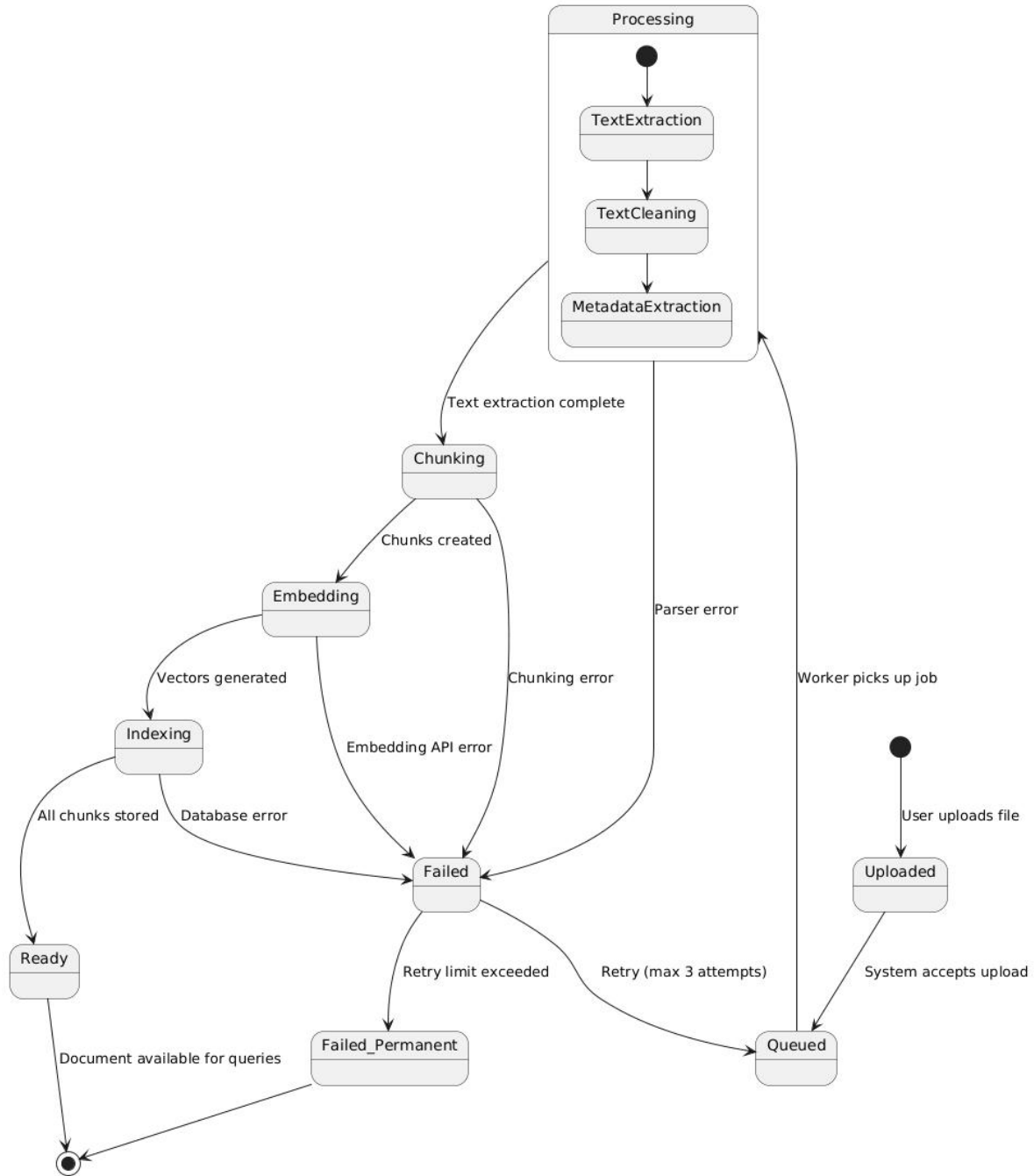


Fig 3.3.4.3 State Machine Diagram: Document Processing States

3.3.4.4. State Machine Diagram: Query Execution States

This diagram shows how a query moves through the system from arrival to response delivery, including tier selection and verification loops.

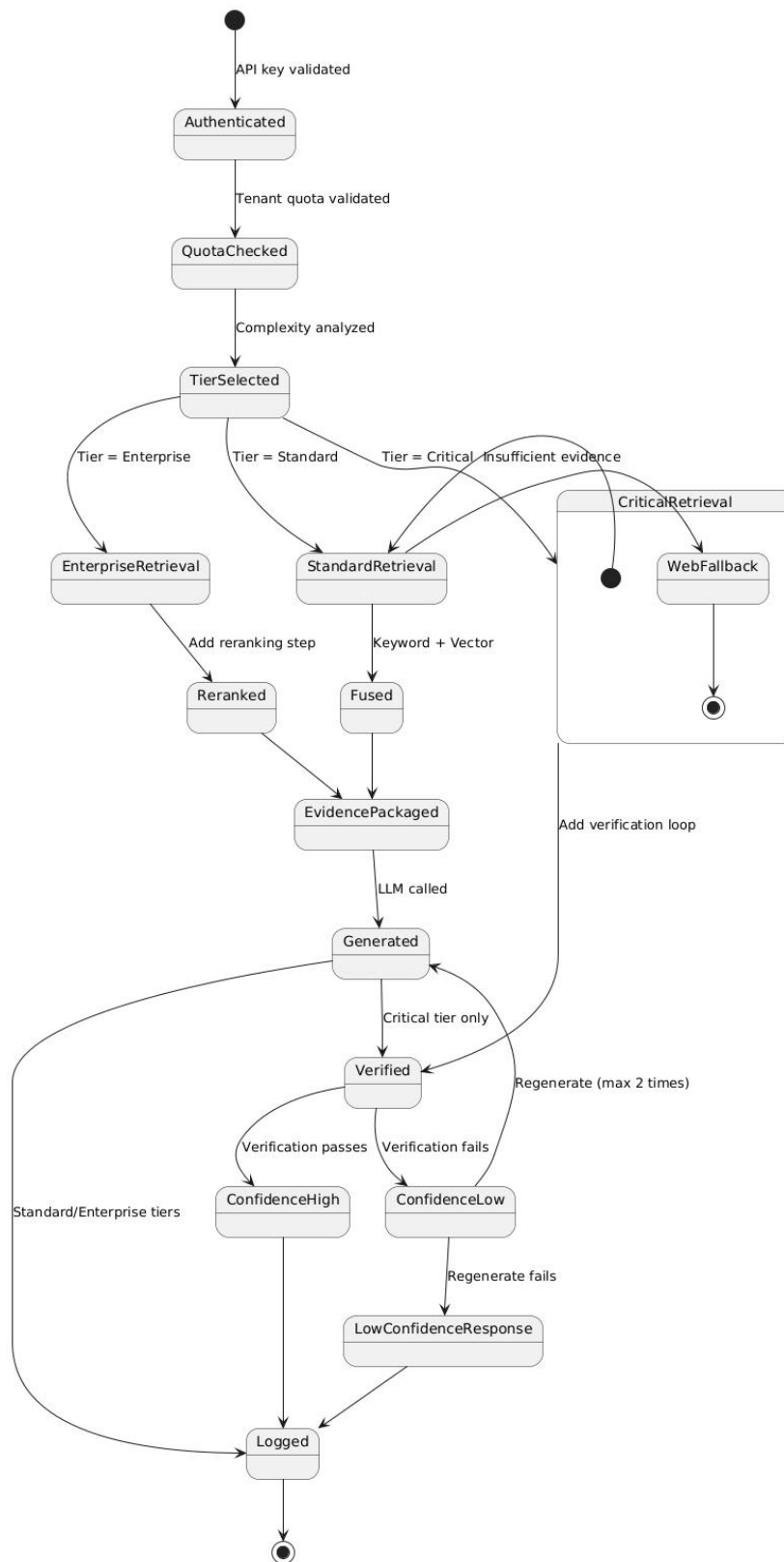


Fig 3.3.4.4 State Machine Diagram: Query Execution States

3.3.4.5. Activity Diagram: Complete Query Processing Workflow

This diagram shows the decision points and parallel processes in the query workflow.

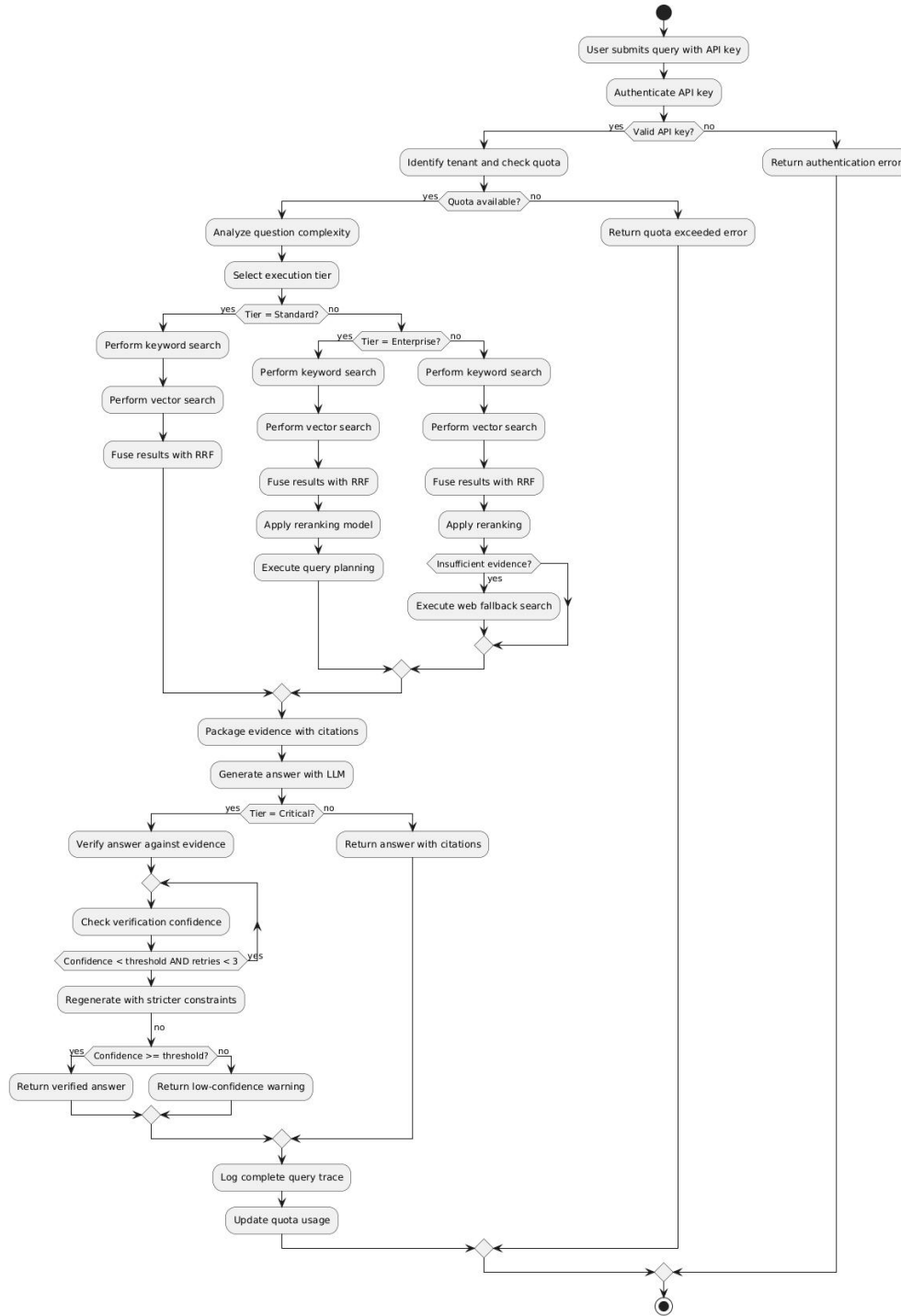


Fig 3.3.4.5. Activity Diagram

3.3.4.6. Collaboration Diagram: Query Processing Components

This diagram shows how objects collaborate during query execution without time ordering.

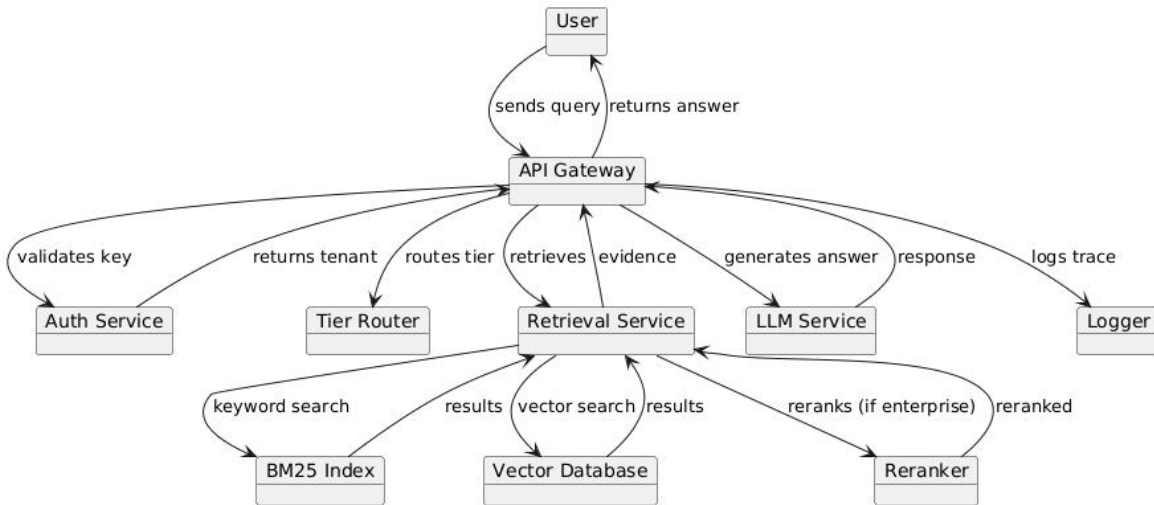


Fig 3.5. Collaboration Diagram

3.4 Model Validation

Model validation serves as the formal verification phase to ensure that the problem analysis and proposed system models for the Grounded framework are technically sound, logically consistent, and capable of fulfilling the project's high-precision objectives. The validation process begins by confirming that the identified "Expertise-Reliability Gap" is systematically addressed by the 11-stage optimization suite, ensuring that the transition from a passive retrieval model to a verifiable reasoning process is mathematically and architecturally justified. This involves a structural walkthrough of the proposed solution to confirm that the architectural design directly mitigates the root causes of hallucination and query ambiguity identified in the existing system.

The structural integrity of the system is validated through a rigorous mapping of functional requirements to the dynamic system models. By examining the Unified Modeling Language (UML) sequence and activity diagrams, the validation process ensures that data flows remain secure and that the Agentic Verification Loop functions as a robust "Double-Check" mechanism. This phase confirms that the interactions between the LLM Planner, the Vector Database, and the Critic Agent are free of logical deadlocks and that the multi-tenant Namespace Isolation is enforced at the earliest point of the retrieval lifecycle, preventing any potential cross-contamination of sensitive data.

Furthermore, the validation ensures that the system satisfies both functional and non-functional requirements by cross-referencing the Pedantic Enforcement schema with the project's goal of medical and legal auditability. This involves confirming that the system models physically constrain the LLM to provide verbatim source quotes and page references, thereby validating the "Proof-of-Source" logic. This step is crucial for professional environments where every AI-generated claim must be traceable to a specific document fragment. By simulating various query scenarios, the model is tested to ensure that the structured output remains consistent regardless of the complexity of the retrieved context.

The dynamic behavior of the system is further validated by evaluating the state transitions within the Corrective RAG (CRAG) fallback logic. This validation proves that the system can autonomously identify low-confidence retrieval results and trigger a web-search recovery or an internal model retrieval without human intervention. By analyzing the decision-making thresholds within the reranking stage, the validation confirms that the system maintains a high signal-to-noise ratio, prioritizing factual saliency over simple keyword similarity. This ensures that the system remains resilient even when the primary knowledge base is incomplete or contains conflicting information.

Finally, the validation phase concludes by assessing the model's completeness against the project's Scope and Limitations. While acknowledging the exclusion of real-time audio and multilingual support, the validation confirms that the core English-text architecture is optimized for performance, reliability, and security. This comprehensive evaluation ensures that the Grounded framework is not merely a collection of tools, but a complete, reliable, and secure ecosystem. The result is a validated blueprint that provides a clear roadmap for implementation, ensuring that the final software product will meet the rigorous standards required for professional, high-stakes AI applications.

Chapter Four: System Design

4.1. Overview

After identifying the functional and non-functional requirements of the proposed Grounded-RAG platform in the previous chapter, this chapter presents the complete system design of the platform. The purpose of the design phase is to translate the identified requirements into a practical and implementable architecture that can support scalable document retrieval, grounded answer generation, tenant isolation, and reliable query processing. The proposed system is designed as a Retrieval-Augmented Generation platform that combines traditional information retrieval techniques with modern large language models. Unlike conventional chatbot systems that generate responses directly from model memory, the Grounded-RAG platform retrieves evidence from uploaded organizational documents before generating answers. This design minimizes hallucinated responses and improves trustworthiness by attaching citations to generated answers.

The design process focused on solving several critical challenges associated with production-level RAG systems. These challenges include handling multiple tenants securely, processing large document collections efficiently, supporting different execution tiers for different query complexities, and ensuring reliable answer generation even under failure conditions. To address these challenges, the system adopts a service-oriented architecture supported by asynchronous document processing, hybrid retrieval mechanisms, vector search infrastructure, and tier-based execution routing.

This chapter presents the architectural structure of the system, subsystem decomposition, database design, deployment strategy, user interface design, integration strategy, and security mechanisms. The chapter concludes by verifying that the proposed design satisfies the requirements identified in Chapter Three.

4.2. Specifying the Design Goals

The design of the Grounded-RAG platform was guided by several major goals derived from the requirements analysis. These goals influenced the selection of technologies, architectural patterns, database structures, deployment strategies, and communication mechanisms throughout the system.

Performance: The system is expected to provide fast and responsive query execution while simultaneously supporting computationally intensive retrieval and generation operations. Users interacting with the platform should receive responses within acceptable time limits depending on the complexity of the query and the execution tier selected. To achieve this objective, the system separates

lightweight operations from heavy background processing tasks. Document ingestion is performed asynchronously so that users receive immediate upload confirmation while processing continues in the background. Query execution paths are also separated into Standard, Enterprise, and Critical tiers. Simple factual questions follow a lightweight execution path, while complex analytical queries use reranking and verification mechanisms only when necessary. This tier-based execution model reduces unnecessary computational overhead and improves overall responsiveness.

Scalability: The proposed platform must support growth in both the number of users and the size of document collections without significant performance degradation. The architecture therefore adopts stateless backend services that can scale horizontally when query volume increases. The system also separates metadata storage, vector storage, caching, and file storage into independent infrastructure components. PostgreSQL manages structured relational data, while Qdrant manages semantic vector embeddings. Redis provides queue management and caching support, and MinIO stores uploaded document files. This separation allows each component to scale independently according to workload demands. Tenant isolation was also considered a scalability concern. The system prevents one tenant's workload from affecting other tenants by using tenant-based filtering and dedicated vector collections for embeddings.

Security: Security was treated as a fundamental architectural requirement throughout the design process. Since the platform stores potentially sensitive organizational documents, the system must ensure strict tenant isolation and secure communication between all components. Authentication is enforced using API keys and user credentials. All API communication occurs through HTTPS using TLS encryption. API keys are hashed before storage using bcrypt to prevent credential exposure if the database becomes compromised. The system also enforces tenant-level authorization throughout the backend architecture. Every database query includes tenant-based filtering, and vector embeddings are stored in tenant-specific collections to prevent accidental cross-tenant retrieval.

Availability and Reliability: The platform must remain operational even when some components experience temporary failure. The architecture therefore assumes that external services such as LLM providers, vector databases, or storage systems may become unavailable at runtime. To address this challenge, the system includes retry mechanisms, asynchronous queues, graceful degradation paths, and verification loops. If a language model request fails temporarily, the system retries the operation before returning an error. If evidence quality is insufficient in Critical tier queries, the system performs regeneration attempts with stricter constraints. Reliability also refers to answer quality. The platform reduces hallucinated responses by forcing the language model to generate answers only from retrieved

evidence. Critical tier queries additionally pass through a verification loop that checks whether generated claims are supported by retrieved document chunks.

Usability: Although the system contains multiple backend services and complex retrieval pipelines, the user experience must remain simple and intuitive. Users should be able to upload documents easily, submit natural language questions, review citations, and understand system responses without technical knowledge of retrieval systems or large language models. The interface therefore adopts a chat-oriented design similar to modern conversational AI systems. Citation previews, upload progress indicators, and execution mode selectors are integrated into the interface to improve transparency and usability.

4.3. System Design

The Grounded-RAG platform follows a service-oriented architecture that separates the system into specialized components responsible for authentication, document ingestion, retrieval, generation, storage, and monitoring. This decomposition improves maintainability, scalability, and fault isolation while enabling independent subsystem development. The overall design integrates traditional keyword retrieval techniques, vector similarity search, asynchronous processing pipelines, and large language model generation into a unified platform capable of handling grounded question answering over organizational documents.

4.3.1. Proposed Software Architecture

The Grounded-RAG platform adopts a service-oriented architecture that separates major system responsibilities into independent but cooperating components. The architecture was designed to improve scalability, maintainability, reliability, and extensibility while supporting the complete Retrieval-Augmented Generation workflow. Instead of implementing all functionality inside a single monolithic application, the system distributes responsibilities across specialized services. This architectural approach allows individual components to scale independently based on workload requirements while reducing tight coupling between modules. It also simplifies future system enhancement because services can evolve without affecting the entire platform.

The proposed architecture consists of six primary architectural layers: the Client Layer, Gateway Layer, Core Processing Layer, Retrieval Layer, Storage Layer, and External Service Layer. Each layer contains components that collaborate to perform specific operations within the platform.

The Client Layer represents the entry point through which users interact with the system. Users access the platform either through the web dashboard or external API clients. The dashboard provides interfaces for document management, querying, citation viewing.

The Gateway Layer contains the API Gateway implemented using FastAPI. This component acts as the central communication point between clients and backend services. The gateway handles request validation, authentication, authorization, tenant identification, rate limiting, routing, and response formatting. The gateway was intentionally designed to remain lightweight by delegating business logic to specialized backend services.

The Core Processing Layer contains the major business services that implement the platform's core functionality. The Query Orchestrator coordinates the complete Retrieval-Augmented Generation pipeline. It determines the execution tier, coordinates retrieval operations, packages evidence, and manages interactions with the language model gateway. The orchestrator additionally manages verification loops for Critical tier queries.

The Ingestion Service manages asynchronous document processing workflows. After document upload, the service extracts text, cleans formatting artifacts, generates chunks with overlap, produces embeddings, and updates retrieval indexes. The asynchronous processing model ensures that large uploads do not block user interaction.

The Retrieval Layer implements hybrid retrieval by combining lexical search and semantic vector search. The Search Service executes BM25 keyword retrieval and vector similarity search in parallel. The results are combined using Reciprocal Rank Fusion (RRF), improving retrieval quality for both exact keyword matches and semantically related queries. Enterprise and Critical execution tiers additionally apply reranking mechanisms to improve evidence relevance.

The Generation Layer contains the LLM Gateway, which manages communication with external language model providers. This component handles prompt delivery, retry policies, timeout handling, model selection, response parsing, and fallback management. The gateway abstracts external model providers from the rest of the system, making it easier to replace or extend language model integrations in the future.

The Storage Layer provides persistent and distributed data management. PostgreSQL stores structured metadata including tenants, users, documents, chunks, API keys, and query logs. Qdrant stores vector

embeddings required for semantic retrieval. Redis supports asynchronous job queues and caching operations, while MinIO manages uploaded document storage.

The architecture additionally supports multi-tenancy through strict tenant isolation mechanisms implemented across all layers. Every query, retrieval operation, document upload, and indexing process is scoped to a verified tenant context. This design prevents unauthorized cross-tenant data access while supporting scalable enterprise deployment. The architecture also supports fault tolerance and graceful degradation. If a component such as the vector database or external LLM provider becomes unavailable, the orchestrator can apply fallback strategies such as keyword-only retrieval or retry mechanisms. This ensures that the platform remains operational even during partial service failures.

4.3.2. Subsystem Decomposition

To simplify development, maintenance, scalability, and future system extension, the Grounded-RAG platform was decomposed into several cohesive subsystems. Each subsystem is responsible for a specific set of functionalities and communicates with other subsystems through clearly defined interfaces. This decomposition approach reduces coupling between components while improving modularity and maintainability.

The system was divided into seven major subsystems: the User Interface Subsystem, Authentication and Authorization Subsystem, Document Processing Subsystem, Retrieval Subsystem, Query Orchestration Subsystem, Generation and Verification Subsystem, and Monitoring and Logging Subsystem.

4.3.2.1. User Interface Subsystem

The User Interface subsystem provides the interaction layer between end users and the platform. It allows users to upload documents, manage document collections, submit queries, review generated responses, inspect citations, and monitor processing status.

The subsystem was designed using a modern single-page application architecture to ensure responsiveness and smooth interaction. The interface communicates with backend services through REST APIs exposed by the API Gateway. The design emphasizes usability by presenting conversational interactions similar to modern AI assistants while preserving transparency through visible citations and query metadata. The dashboard interface supports document management operations including upload, deletion, indexing status monitoring, and query history visualization. The query interface allows users to select execution tiers such as Standard, Enterprise, or Critical depending on the complexity and reliability requirements of the query.

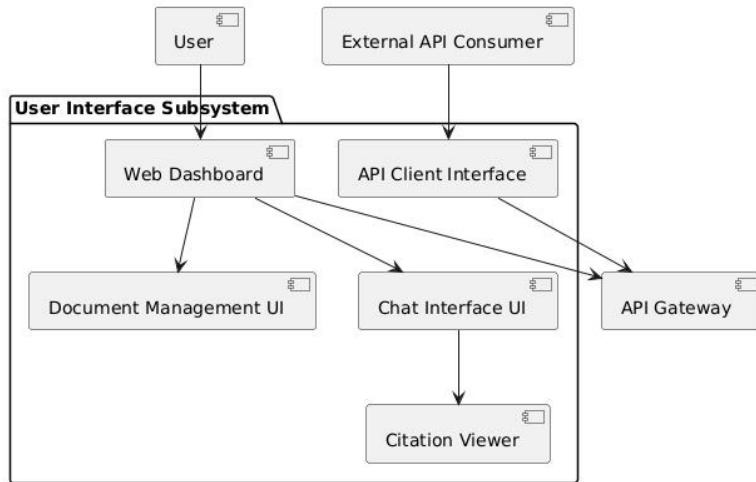


Figure 4.3.2.1. User Interface Subsystem

4.3.2.2. Authentication and Authorization Subsystem

The Authentication and Authorization subsystem is responsible for identity verification, access control, API key validation, and tenant isolation. Since the platform operates in a multi-tenant environment, secure separation of tenant resources is critical. Every incoming request passes through this subsystem before reaching core services. The subsystem validates user credentials or API keys, identifies the tenant associated with the request, verifies quota availability.

API keys are hashed before storage to protect credentials in case of database compromise. The subsystem also implements rate limiting to prevent abuse and denial-of-service attacks. Tenant isolation is enforced throughout the request lifecycle by attaching tenant context to every database query and retrieval operation.

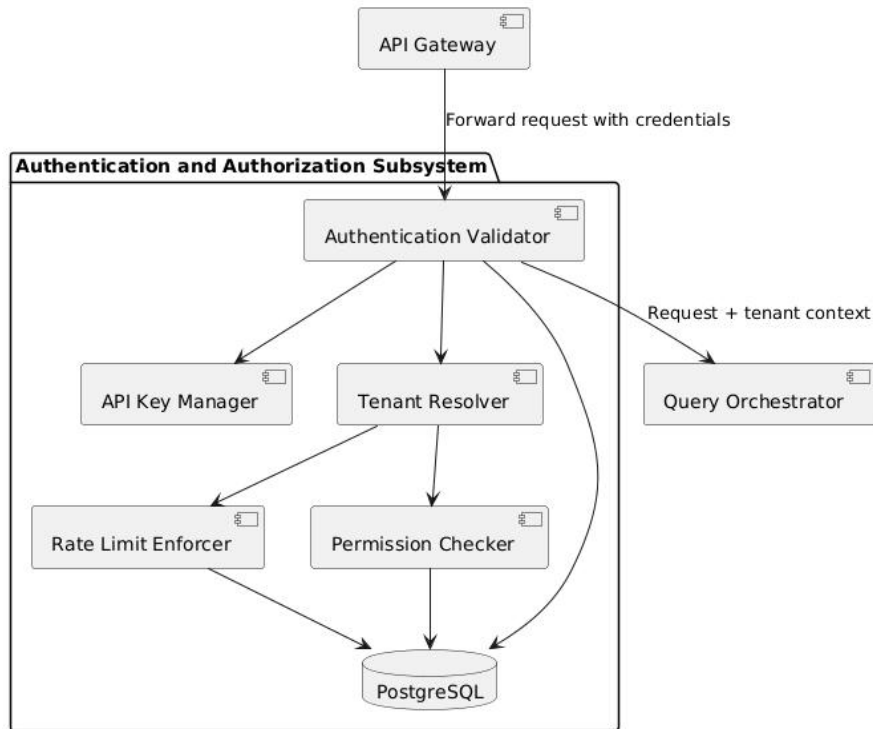


Figure 4.3.2.2. Authentication and Authorization Subsystem

4.3.2.3. Document Processing Subsystem

The Document Processing subsystem handles the ingestion and indexing pipeline of uploaded documents. This subsystem transforms raw uploaded files into searchable chunks and embeddings that can later be retrieved during query execution. The subsystem supports multiple document formats including PDF, DOCX, and TXT files. After upload, documents are stored in MinIO object storage and processing jobs are placed into a Redis queue for asynchronous execution. Worker processes retrieve queued jobs and perform several operations sequentially. First, text is extracted from the uploaded document. The extracted text is then cleaned and normalized to remove unnecessary formatting artifacts. The cleaned text is divided into overlapping chunks to preserve semantic continuity between sections. Embeddings are generated for every chunk and stored inside the vector database. The subsystem was designed asynchronously to prevent large document uploads from blocking user requests. Users receive immediate upload confirmation while processing continues in the background.

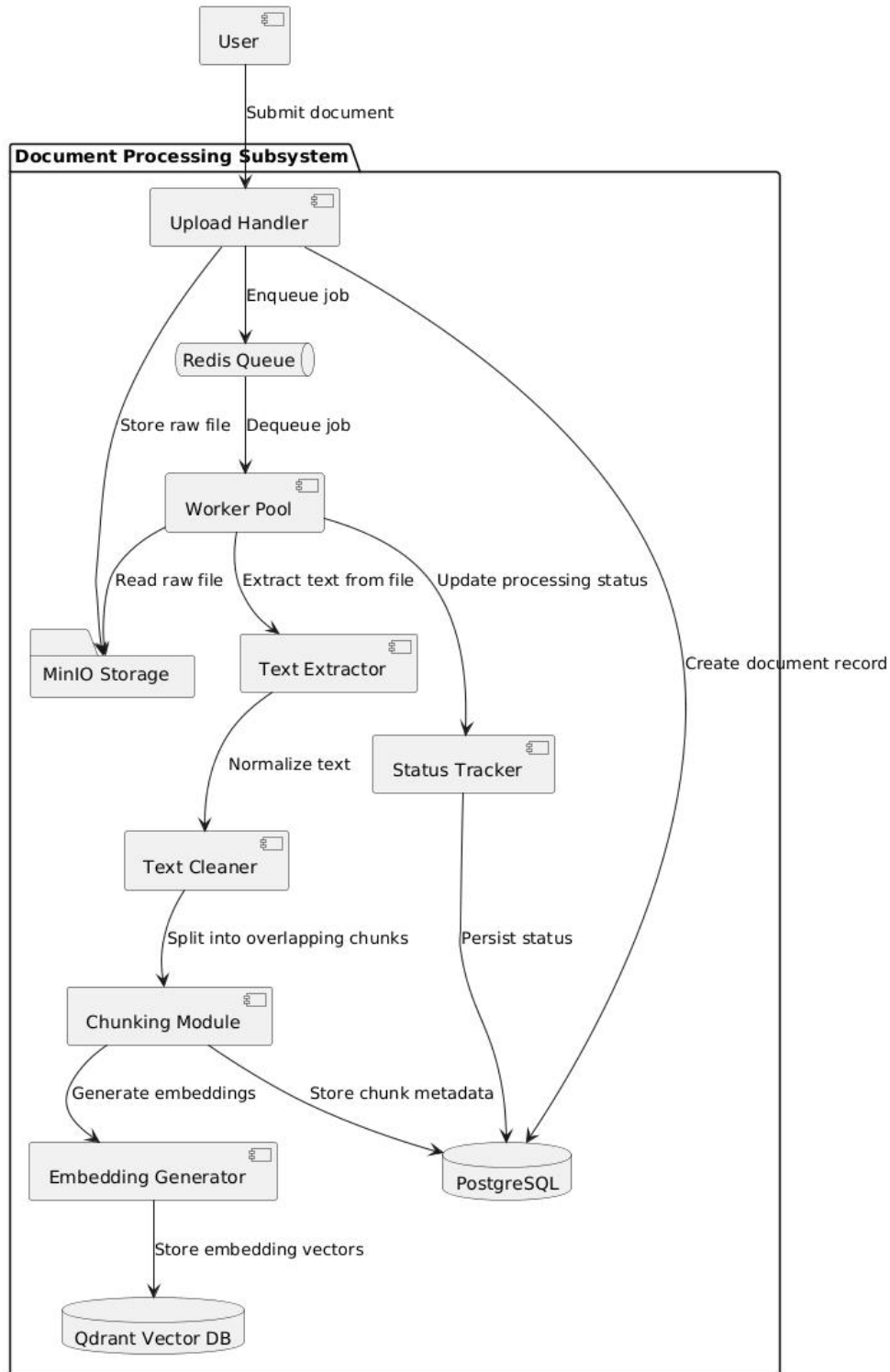


Figure 4.3.2.3. Document Processing Subsystem

4.3.2.4. Retrieval Subsystem

The Retrieval subsystem is responsible for locating the most relevant evidence chunks for a user query. The subsystem implements hybrid retrieval using both keyword-based and vector-based search techniques. When a query arrives, the subsystem performs BM25 keyword retrieval and semantic vector retrieval in parallel. The keyword retrieval mechanism is effective for exact terminology and identifiers, while vector search captures semantic similarity and conceptual meaning.

The retrieved results from both retrieval methods are combined using Reciprocal Rank Fusion (RRF). The RRF formula computes a combined score for each chunk as the sum of reciprocal ranks: $\text{score} = \sum 1/(\text{rank} + k)$, where k is a constant typically set to 60. This approach favors chunks that appear highly ranked in either retrieval method.

For Enterprise and Critical tiers, the subsystem additionally applies cross-encoder reranking models to improve relevance quality before evidence is passed to the generation subsystem. Reranking evaluates each chunk against the original question using a more computationally expensive but more accurate model. The subsystem ensures strict tenant-level filtering by constructing database queries with tenant ID predicates and by using tenant-specific vector collections in Qdrant. Retrieval operations only access documents belonging to the authenticated tenant.

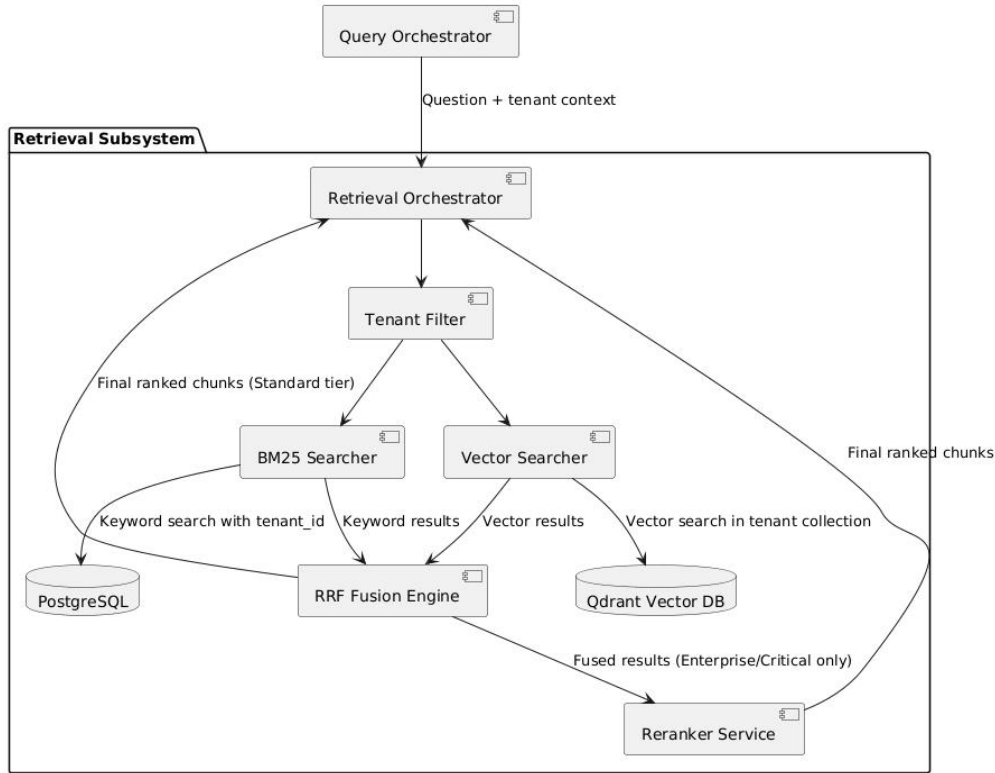


Figure 4.3.2.4. Retrieval Subsystem

4.3.2.5. Query Orchestration Subsystem

The Query Orchestration subsystem coordinates the complete query execution workflow. It acts as the central controller of the RAG pipeline and determines how queries are processed depending on complexity, tenant plan, and execution tier. The subsystem begins by analyzing the incoming question to assess complexity. Complexity analysis examines factors such as the presence of comparison words ("compare", "versus", "difference"), temporal references ("before", "after", "during"), conditional logic ("if", "then"), and multi-step reasoning patterns.

Based on complexity score and tenant subscription plan, the subsystem selects the appropriate execution tier. Standard tier executes basic hybrid retrieval and single-pass generation. Enterprise tier introduces reranking and query planning for multi-hop questions. Critical tier introduces verification loops and evidence validation with optional web fallback for high-stakes queries.

The orchestrator coordinates communication between the retrieval subsystem, generation and verification subsystem, and logging subsystem. It handles timeout management, fallback strategies, and error recovery. If the LLM gateway times out, the orchestrator returns a graceful degradation response rather than failing

completely. Each tier implementation shares the same underlying retrieval and generation services but composes them differently.

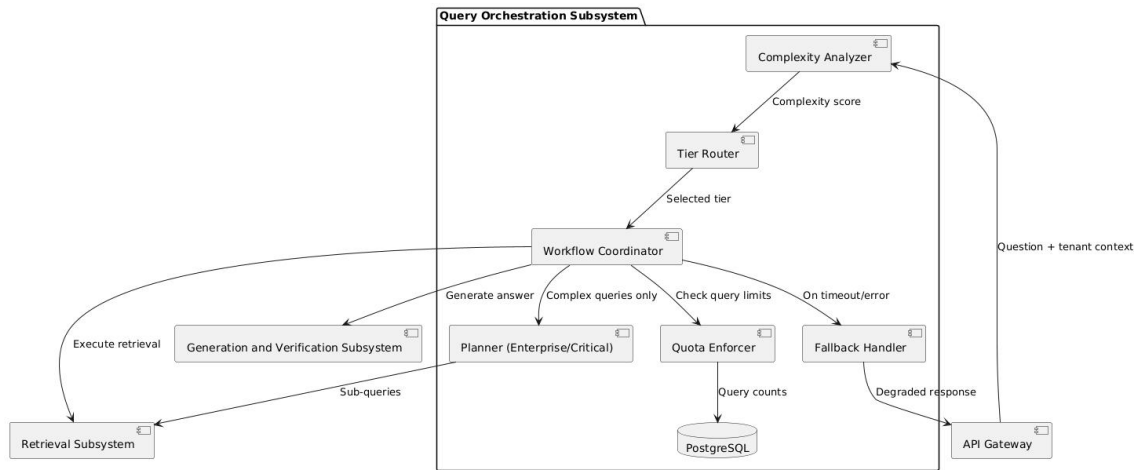


Figure 4.3.2.5. Query Orchestration Subsystem

4.3.2.6. Generation and Verification Subsystem

The Generation and Verification subsystem is responsible for generating grounded responses from retrieved evidence and validating response correctness for higher reliability tiers. The subsystem constructs structured prompts containing the user question, retrieved evidence chunks, citation formatting instructions, and hallucination prevention constraints. The prompt template explicitly instructs the model to cite sources using bracketed numbers corresponding to the evidence list and to respond with "I cannot answer based on available documents" when evidence is insufficient.

These prompts are sent to external language models through the LLM Gateway, which handles API authentication, retry logic with exponential backoff, cost tracking, and fallback to alternative models when primary models are unavailable. For Standard and Enterprise tiers, the subsystem generates responses directly from retrieved evidence. Citations are embedded as inline references.

For Critical tier queries, an additional verification loop executes. The verification engine checks whether each claim in the generated response is supported by the retrieved evidence using a separate language model call. If unsupported claims are detected with low confidence, the system triggers one of three recovery actions: regeneration with stricter constraints, confidence score reduction with disclosure to the user, or web fallback search if configured and allowed. Citation generation is integrated into the response generation process to ensure every factual statement can be traced back to source documents. The

subsystem maintains a mapping between citation numbers and source chunks, which is returned to the client for rendering clickable citations.

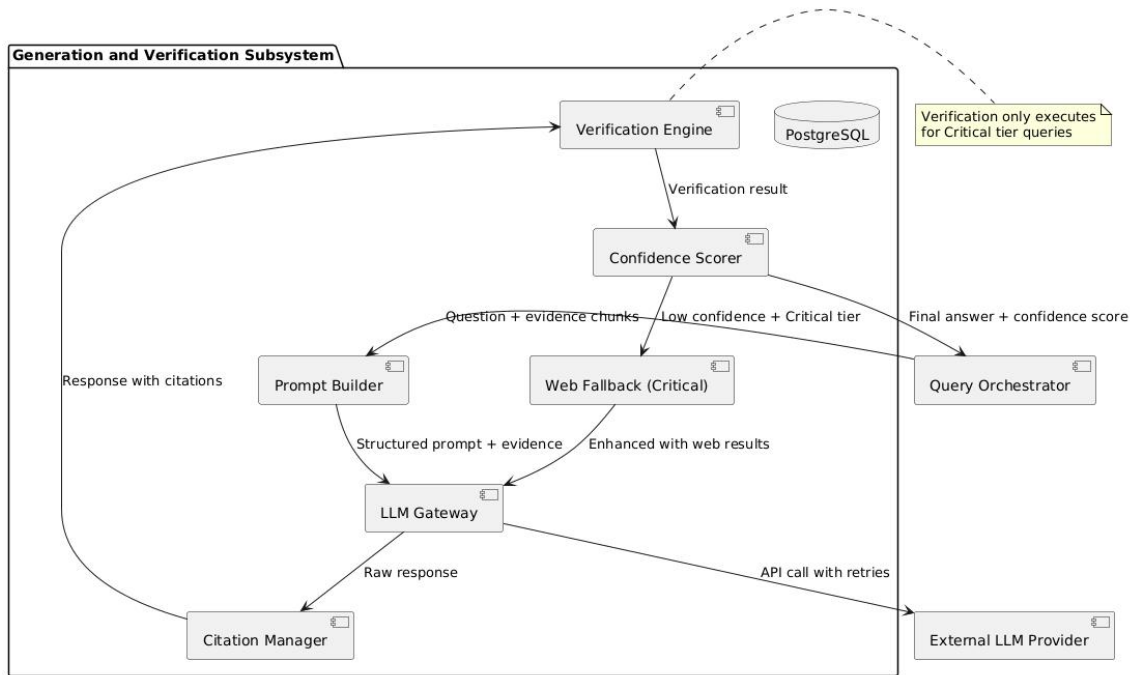


Figure 4.3.2.6. Generation and Verification Subsystem

4.3.2.7. Monitoring and Logging Subsystem

The Monitoring and Logging subsystem provides observability, analytics, system diagnostics, and operational health tracking across the entire platform. Every major operation generates structured JSON logs that include timestamps, request identifiers, tenant IDs, latency values, processing outcomes, and error traces. Query execution logs store the question, selected tier, retrieval latency, generation latency, number of chunks retrieved, citation count, and final answer. These logs are written to PostgreSQL with tenant ID partitioning. The subsystem distinguishes between two categories of observability:

Tenant-Facing Analytics: Tenants can query their own usage metrics including query volume over configurable time windows, average and percentile latency per tier, document count and storage utilization, and query history for audit purposes. These queries are filtered by tenant ID at the database level, ensuring tenants never see other tenants' data.

Platform Health Monitoring: Platform operators monitor infrastructure health through dedicated endpoints that expose service availability, error rates, and resource utilization. These endpoints never expose tenant-

specific content. Metrics collected include CPU and memory usage per service, database connection pool status, vector database health, queue depth for document processing, LLM API latency and error rates, and total query throughput across all tenants.

Health check endpoints at `/health/live` and `/health/ready` allow orchestration systems to verify service availability. The liveness endpoint checks that the process is running, while the readiness endpoint verifies that all dependencies (database, vector store, cache) are accessible.

The subsystem also supports benchmarking operations using the BEIR benchmark suite. These benchmarks run against standardized datasets, not tenant production data, and measure retrieval quality metrics including `nDCG@10`, `Recall@10`, and `MRR` across Standard, Enterprise, and Critical tier configurations. Logs are retained for 90 days to support debugging and compliance requirements. Tenants can export their query history as JSON or CSV for external analysis. Platform operators can access aggregated error logs for system debugging, but these logs contain only error codes and stack traces, never document contents or query answers.

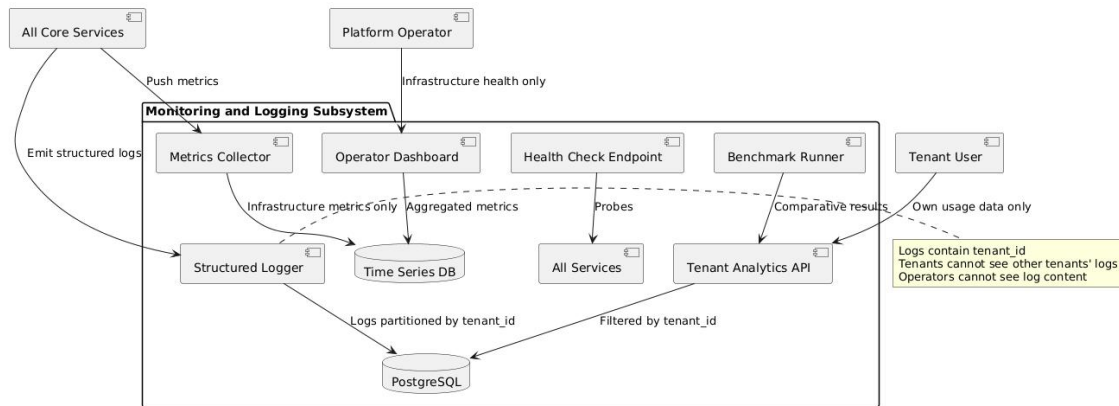


Figure 4.3.2.7. Monitoring and Logging Subsystem

4.3.3. Database Design

The platform uses both relational and vector databases to support structured metadata storage and semantic retrieval operations. PostgreSQL was selected for relational storage because of its transactional consistency and strong support for complex relationships. Qdrant was selected for vector storage because it provides efficient high-dimensional similarity search and tenant-level collection management. The relational schema follows normalized design principles to maintain referential integrity and reduce redundancy.

The tenants entity stores organizational accounts and subscription information. Each tenant owns users, API keys, uploaded documents, and query histories.

Attribute	Type	Description
id	UUID	Primary key
name	VARCHAR	Tenant name
plan	VARCHAR	Subscription tier
quota	INTEGER	Query quota
storage_limit_bytes	BIGINT	Storage limit
created_at	TIMESTAMP	Creation date

The users entity stores user accounts associated with tenants. Each user belongs to exactly one tenant and may have different access roles.

Attribute	Type	Description
id	UUID	Primary key
tenant_id	UUID	Foreign key to tenant
email	VARCHAR	User email
role	VARCHAR	User role
last_login_at	TIMESTAMP	Last login

The documents entity stores uploaded document metadata and processing status information.

Attribute	Type	Description
id	UUID	Primary key
tenant_id	UUID	Owner tenant
name	VARCHAR	Document name
file_path	VARCHAR	Storage path
status	VARCHAR	Processing state
chunk_count	INTEGER	Number of chunks

The chunks entity stores extracted document fragments used during retrieval.

Attribute	Type	Description
id	UUID	Primary key
document_id	UUID	Parent document
text	TEXT	Chunk content
page_number	INTEGER	Source page
token_count	INTEGER	Chunk token size

The queries entity stores query traces and analytics information for monitoring and debugging.

Attribute	Type	Description
id	UUID	Primary key
tenant_id	UUID	Owner tenant
question	TEXT	User query
answer	TEXT	Generated response
tier_used	VARCHAR	Execution tier
latency_ms	INTEGER	Query latency

The relationship structure ensures that documents belong to tenants, chunks belong to documents, and queries reference retrieved chunks through junction relationships.

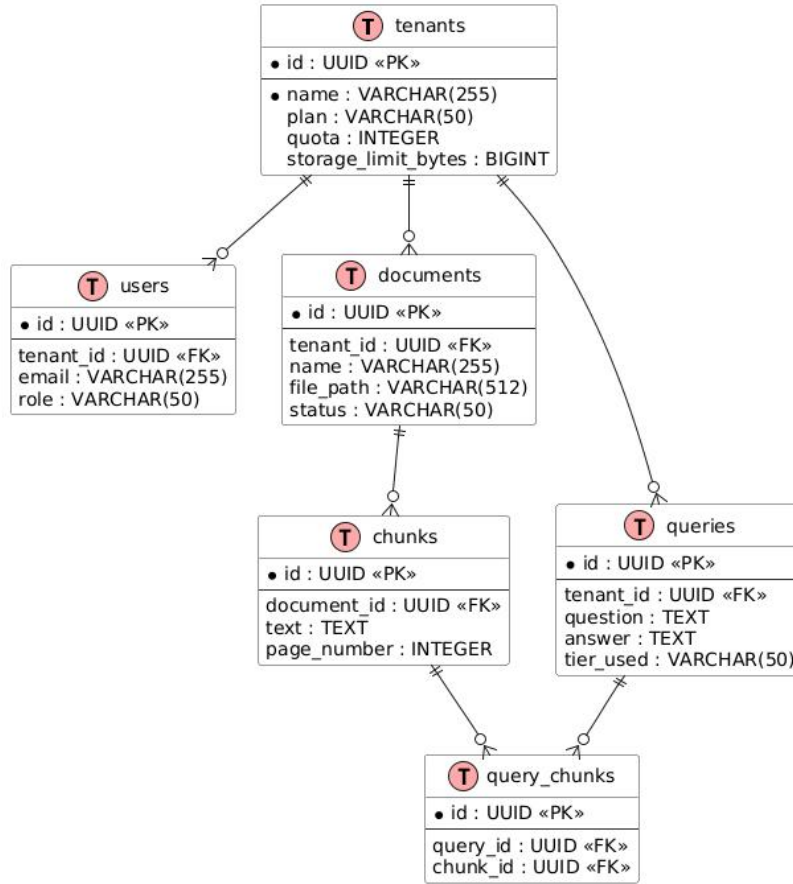


Fig 4.3.3.1 ER Diagram

4.3.4. Deployment Diagram

The deployment architecture was designed to support modularity, maintainability, and simplified service management. The platform is deployed using Docker and Docker Compose, where each major subsystem operates as an isolated containerized service. This approach enables consistent environments across development and testing while simplifying deployment and dependency management. The deployment environment consists of multiple interconnected containers including the API Gateway, Query Orchestrator, Ingestion Service, Search Service, PostgreSQL database, Qdrant vector database, Redis cache, and MinIO object storage service. Docker Compose manages service orchestration, internal networking, and startup dependencies between containers.

All services communicate through Docker's internal virtual network, allowing secure inter-service communication between backend components. Persistent Docker volumes are used to preserve database records, vector embeddings, cached data, and uploaded documents across container restarts. The API

Gateway serves as the primary entry point for client requests and forwards requests to the appropriate backend services. The Query Orchestrator coordinates retrieval, reranking, verification, and response generation workflows. The Search Service interacts with PostgreSQL, Qdrant, and Redis to support hybrid retrieval and caching operations. The Ingestion Service processes uploaded documents and stores raw files within MinIO object storage. The containerized deployment architecture improves maintainability by separating system responsibilities into independent services. It also supports future scalability by allowing individual services to be extended, replicated, or migrated into distributed orchestration platforms if required.

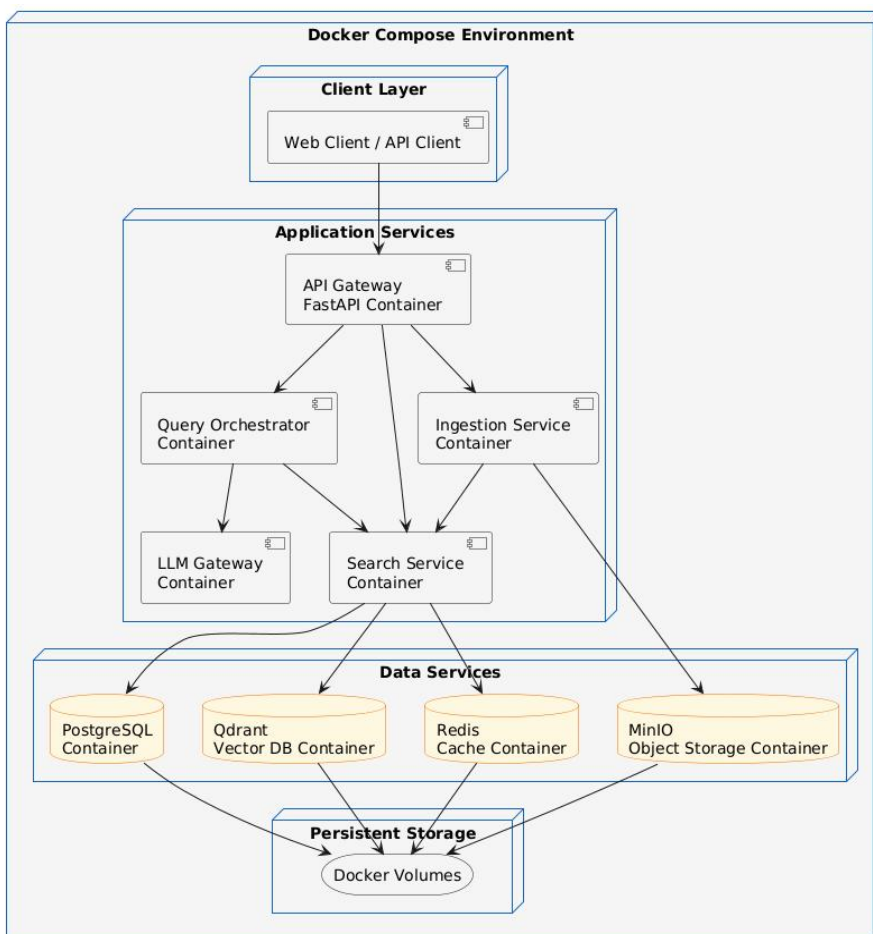


Fig 4.3.4. Deployment Diagram

4.3.5. User Interface Design

The user interface was designed to provide a simple and intuitive interaction model despite the complexity of the backend architecture. The interface follows a modern dashboard-oriented layout with a

navigation sidebar and a central workspace area. The document management interface allows users to upload, monitor, and manage organizational documents. Uploaded files display processing status indicators that show whether the document is pending, processing, ready, or failed. Drag-and-drop upload functionality improves usability and simplifies the ingestion process for non-technical users.

The query interface adopts a conversational chat-based design. Users interact with the platform by asking natural language questions in a message input area. Responses appear within a threaded conversation view together with inline citations referencing supporting evidence. The interface also exposes execution tier selection options. Users may choose between Standard, Enterprise, and Critical execution modes depending on their accuracy and latency requirements. The system displays retrieval metadata, query duration, and verification status to improve transparency and trustworthiness.

Citation previews were included to improve evidence traceability. When users hover over a citation, the interface displays a preview of the referenced text chunk. Selecting the citation opens the original document with the relevant passage highlighted.

4.3.6. System Integration

The Grounded-RAG platform integrates multiple internal services and external providers into a unified workflow. Communication between backend services primarily occurs through REST APIs and asynchronous message queues. When a document is uploaded, the API gateway forwards the request to the ingestion subsystem. The document is stored in MinIO while a background processing job is added to the Redis queue. Worker processes retrieve queued jobs, process the documents, generate embeddings, and update both PostgreSQL and Qdrant indexes.

During query execution, the query orchestrator communicates with the retrieval subsystem to perform both keyword and vector searches. Retrieved chunks are fused and optionally reranked before being forwarded to the language model gateway. The LLM gateway communicates with external language model providers through secured HTTPS APIs. Generated answers are returned to the orchestrator together with citation mappings and verification metadata. The integration architecture also supports observability services such as monitoring dashboards, centralized logging systems, and health check endpoints. This integration strategy ensures interoperability while preserving subsystem independence.

4.3.7. Security Design

Security mechanisms were integrated throughout the architecture to protect sensitive organizational data and maintain tenant isolation. Authentication is enforced using API keys and user credentials. API keys are hashed before storage using bcrypt to prevent credential leakage. All communication between clients and backend services occurs over HTTPS using TLS encryption.

Authorization is implemented using tenant-based access control. Every database query includes tenant filtering to prevent unauthorized cross-tenant access. Vector embeddings are also isolated through tenant-specific collections within Qdrant. Input validation mechanisms protect the platform from malicious uploads and malformed requests. Uploaded files are validated for file type, size, and content structure before processing begins. Query inputs are sanitized before being forwarded to retrieval or generation components.

Rate limiting prevents abuse by restricting excessive API usage. Quota enforcement mechanisms ensure fair resource allocation between tenants. Sensitive configuration values such as API secrets and database credentials are stored as environment variables rather than hardcoded within application code. Logging systems avoid storing sensitive user information or raw credentials. The Critical execution tier further improves reliability by introducing verification loops that validate generated claims against retrieved evidence before returning responses to users.

4.4. Verifying the Requirements in the Design

The proposed design was evaluated against both the functional and non-functional requirements identified in Chapter Three to ensure completeness and consistency. The functional requirements are fully represented within the architecture. Document upload and processing requirements are implemented through the asynchronous ingestion subsystem. Query submission and grounded answer generation are implemented through the orchestrator, retrieval subsystem, and language model gateway. Citation support is integrated into both backend processing and user interface design. Multi-tenant management is implemented through tenant-aware database structures and authorization mechanisms.

The non-functional requirements are also reflected throughout the design. Performance requirements are addressed through asynchronous processing, hybrid retrieval, and tier-based execution routing. Scalability requirements are supported through stateless backend services, distributed infrastructure components, and horizontal scaling mechanisms.

Security requirements are satisfied through authentication, authorization, encryption, input validation, and tenant isolation. Availability requirements are supported through retries, replication strategies, background queues, and graceful degradation mechanisms. Reliability requirements are addressed through evidence-based generation, reranking, and verification loops in Critical tier execution.

The deployment architecture, subsystem decomposition, database structure, and integration mechanisms collectively provide a technically consistent foundation for implementing the Grounded-RAG platform. The completed design therefore satisfies the identified project objectives and provides a clear blueprint for implementation and future system expansion.

Chapter Five: System Implementation

5.1 Reviewing the Design Solution

When we began implementation, we had already committed to two linked outcomes rather than a single technical demonstration.

The first was the research contribution. We implemented mechanisms intended to improve over basic Retrieval-Augmented Generation (RAG) [1]. In practice, that meant building more than a retrieve-and-generate shortcut: the backend applied query analysis, hybrid retrieval, evidence packaging, response shaping, and trace persistence so that answers are tied more deliberately to stored knowledge.

The second was the product contribution. The same pipeline had to be reachable through normal product workflows. Organizations and users could upload documents, organize them into datasets, attach those datasets to agents, chat over grounded answers, and inspect runs without managing embeddings, indexing jobs, or retrieval orchestration themselves.

Before writing substantial code, the design was checked against five practical questions:

1. Does the implementation add mechanisms intended to improve over basic RAG, rather than only reproducing one?
2. Are those mechanisms exposed through a usable product, not hidden inside internal services?
3. Can knowledge be uploaded and organized into datasets without manual pipeline management?
4. Can interaction happen through workspaces, datasets, agents, and chat instead of low-level API calls alone?
5. Do answers remain grounded, cited, and inspectable through persisted run traces?

These questions shaped the implementation plan. Work therefore proceeded on two tracks in parallel: the adaptive grounding pipeline in backend workers and services, and the organization-facing product layer in the frontend and /v1 API.

5.1.1 RAG Context and the Implemented Response

Basic RAG often follows a short path: embed a question, retrieve top-k chunks, and generate an answer. That pattern is useful, but the literature shows that RAG systems still depend heavily on retrieval quality, passage relevance, and the faithfulness of the final answer [1], [2], [23]. In practical terms, weak retrieval, vague questions, or unsupported generated claims can reduce user trust [26]. We did not claim to solve

every RAG limitation. We implemented mechanisms intended to improve over basic RAG in a bounded scope: structure-aware ingestion, hybrid retrieval, evidence packaging, embedded citation checks, and persisted run traces. These mechanisms are delivered through a product interface that users can operate without manually managing embeddings, indexing jobs, or retrieval orchestration.

5.1.2 Rival Approaches Considered

We positioned the implementation against three rival approaches. The first rival approach was a prompt-only chatbot, which is simple to build but does not connect answers to an organization’s uploaded documents. The second was a basic vector-only RAG prototype, which retrieves semantically similar chunks but can miss exact wording, policy terms, or structured document evidence. The third was a full enterprise knowledge platform, which would include connectors, governance, RBAC, audit logs, and administration features; those items were outside the verified scope of this final-year project and were kept as future work.

Grounded AI therefore took a middle path. It did not attempt to deliver a complete enterprise suite, but it also avoided being only a thin RAG demo. The implemented artefact combined direct file upload, dataset organization, structure-aware chunking, sparse and dense retrieval, grounded answers, citations, and run inspection. This choice directly responded to the quality bottleneck in RAG: the answer is only as trustworthy as the retrieved and selected evidence that supports it [2], [23], [26].

5.1.3 Implementation Story and Roadmap

We developed the project in phases rather than as a single monolithic drop. Early work established persistence, the/v1 API shell, and ingestion workers. Mid-phase work completed indexing and `execute_standard_query`. Later work added agents, chat, dashboard visibility, and run inspection.

5.1.4 Requirement Prioritization Using MoSCoW

We used MoSCoW prioritization to separate what had to be implemented from what could remain future work. MoSCoW classifies requirements into Must Have, Should Have, Could Have, and Won’t Have this time. For this project, the Must Have group included authentication, datasets, upload, ingestion status, grounded chat, run inspection, and API-key/tenant-scoped access. Should Have items included dashboard visibility, agents, and dataset attachment. Could Have items included richer UI polish and expanded inspection details. Won’t Have this time items included enterprise connectors, RBAC, SSO, billing, audit logs, and governance.

We added this prioritization to the methodology because it protected the documentation from over-claiming. It also made the implementation story clearer: the delivered system was a verified product foundation, while enterprise governance and stronger verification remained for future development.

MoSCoW Prioritization Summary

MoSCoW Category	Items in This Project	Final Status
Must Have	Authentication, datasets, upload, ingestion status, grounded chat, Agents, dataset attachment, dashboard visibility, structured answers with citations	Implemented and evidence-supported
Should Have	run inspection, API-key authentication and tenant scoping	Implemented with documented caveats
Could Have	Expanded UI polish, deeper raw trace exploration, more frontend automated tests	Partially supported or future improvement
Won't Have This Time	RBAC, SSO, billing, enterprise governance, audit logs, external connectors, standalone verifier	Future work only

Table 5.1 MoSCoW Prioritization

Note: The **Enterprise execution tier** (Thinking mode, deeper retrieval, reranking, and auto-routing) is implemented and evaluated in this project. Items listed under Won't Have refer to platform governance and connector products, not to the Enterprise tier pipeline itself.

5.1.4 Three-Tier Execution Model (Standard, Enterprise, Critical)

Grounded AI was implemented as a single orchestrated pipeline (`execute_standard_query` in `backend/app/services/query.py`) with three execution tiers. Tiers are not separate applications; they are routing and behaviour profiles applied to the same ingestion, retrieval, generation, and trace persistence path.

User mode (product)	Backing tier	Typical use
Instant	Standard	Fast everyday grounded Q&A

Auto	Router decides	Default; may stay on Standard or route up
Thinking	Enterprise	Harder questions, comparison/action queries, richer retrieval
Verified	Critical	Highest assurance; claim verification and corrective retrieval when enabled

Table 5.2 User-facing modes and backing tiers

Standard tier flow

- Upload and ingestion (extraction, structure-aware chunking, sparse + dense indexing).
- Query profiling and retrieval planning (build_query_profile, build_query_plan).
- Hybrid retrieval and evidence packaging (retrieve_hybrid_candidates, package_evidence).
- Grounded generation with citation-contract validation (generate_answer_from_evidence, _validate_citation_contract).
- Response shaping and trace persistence (shape_grounded_response, query_traces).
- Standard is the default production path for Instant and for Auto when no higher tier is selected.

Enterprise tier flow (additions on top of Standard)

When enterprise_enabled is true (default in configuration), Enterprise adds:

- **Auto-routing** for hard query shapes (comparison, action, multi-attribute, document-reference, and related triggers) via _enterprise_route_reasons and _resolve_execution_routing.
- **Query-plan refinement** with extra retrieval variants (refine_query_plan_for_execution_tier, _build_enterprise_retrieval_query_variants in backend/app/core/query_analysis.py).
- **Enterprise retrieval scoring** including freshness boosts (_apply_enterprise_freshness_scoring in backend/app/services/retrival.py)
- **Reranking** through resolve_retrieval_reranker (Gemini reranker backend by default).
- **Broader evidence bundles** in backend/app/services/evidence.py (higher limits for structured multi-chunk support).

Thinking mode maps directly to Enterprise.

Critical tier flow (additions on top of Enterprise)

Critical is the highest-assurance profile behind Verified mode. Implementation on feature/critical-tier includes:

- **Claim-level verification** (`verify_critical_response` in `backend/app/services/verification.py`).
- **Bounded corrective retrieval** (`_run_critical_corrective_loop` in `backend/app/services/query.py`).
- **Policy-gated internal evidence expansion** (`backend/app/services/internal_model_retrieval.py`).
- **Policy-gated external fallback** (`backend/app/services/external_fallback.py`).
- **Async Verified queries** when requested (`queue_async_verified_query`, `process_async_verified_query`).

By default, `critical_enabled` is `False` in `backend/app/config.py`, so Verified mode is hidden in the product until explicitly enabled. When Critical is requested but disabled, routing records `critical_requested_fallback_standard` and executes the Standard path. This behaviour is test-supported in agent chat tests.

Tier gating in data

- **Tenant cap:** `tenants.max_execution_tier`
- **Dataset floor:** `namespaces.min_execution_tier`, plus `freshness_profile` and `sensitivity_level`
- Every run stores `requested_tier`, `router_recommendation`, and `effective_tier` on `query_traces`

Figure 5.2 (Section 5.3) shows how these tiers sit on top of the shared pipeline stages.

5.1.5 User Journey

The verified user journey begins when a user creates an account or signs in, selects a workspace, and prepares a dataset for organizational knowledge. The user then uploads documents, waits for ingestion status to confirm processing, creates a grounded agent, attaches the dataset to that agent, asks questions through chat, reviews cited answers, and finally inspects the saved run for confidence and supporting evidence.

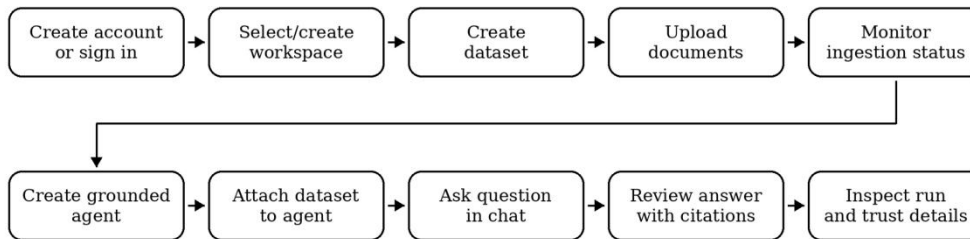


Figure 5.1. User journey of the Grounded AI platform.

Figure 5.1 shows the user-facing flow of the platform rather than the internal system architecture. It highlights how the implemented features are experienced by a normal user: authentication, workspace selection, dataset preparation, document ingestion, agent configuration, grounded chat, citation review, and run inspection.

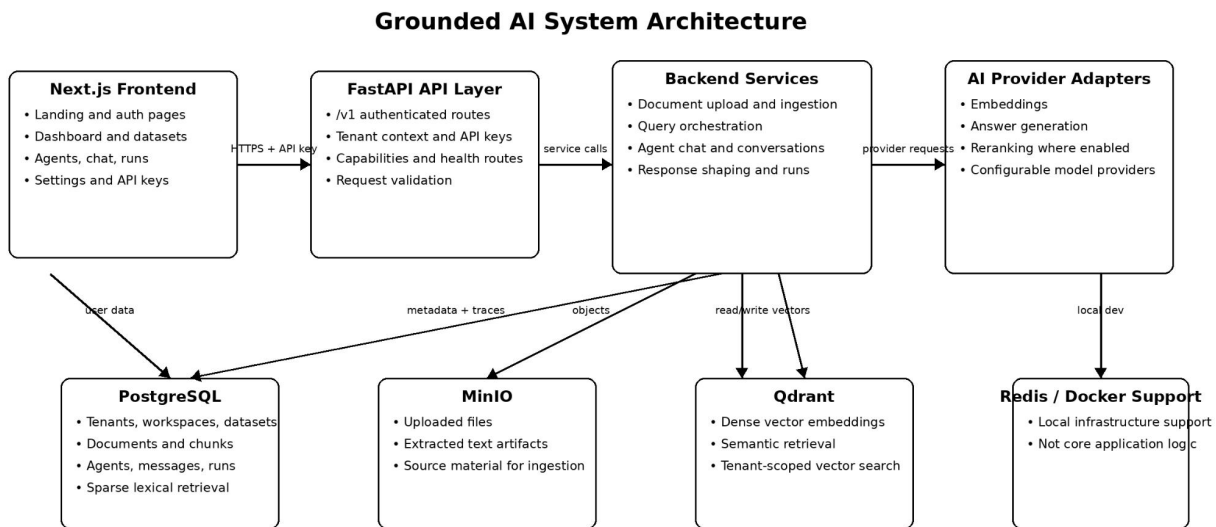


Figure 5.2. System architecture of Grounded AI.

Figure 5.2 shows runtime components: the product frontend, FastAPI /v1 routes, ingestion workers, query orchestration, PostgreSQL, MinIO, Qdrant, and provider adapters. Health routes are exposed separately at /health/live and /health/ready.

5.1.6 User Manual

This section explains how a normal user interacts with the implemented Grounded AI platform. The user manual follows the practical workflow of the system, beginning from workspace review and agent creation, and ending with grounded chat interaction, citation review, and evidence inspection. The screenshots in this section are taken from the implemented product interface and are used to demonstrate the main user-facing features of the platform.

Step 1: Review the Workspace Overview

After signing in, the user is directed to the workspace overview page. This page provides a summary of the current workspace status, including workspace readiness, indexed documents, active agents, available processing modes, and recent query activity. The sidebar allows the user to navigate to agents, datasets, runs, settings, and API key management. From this page, the user can either add data to the workspace or create a new grounded agent.

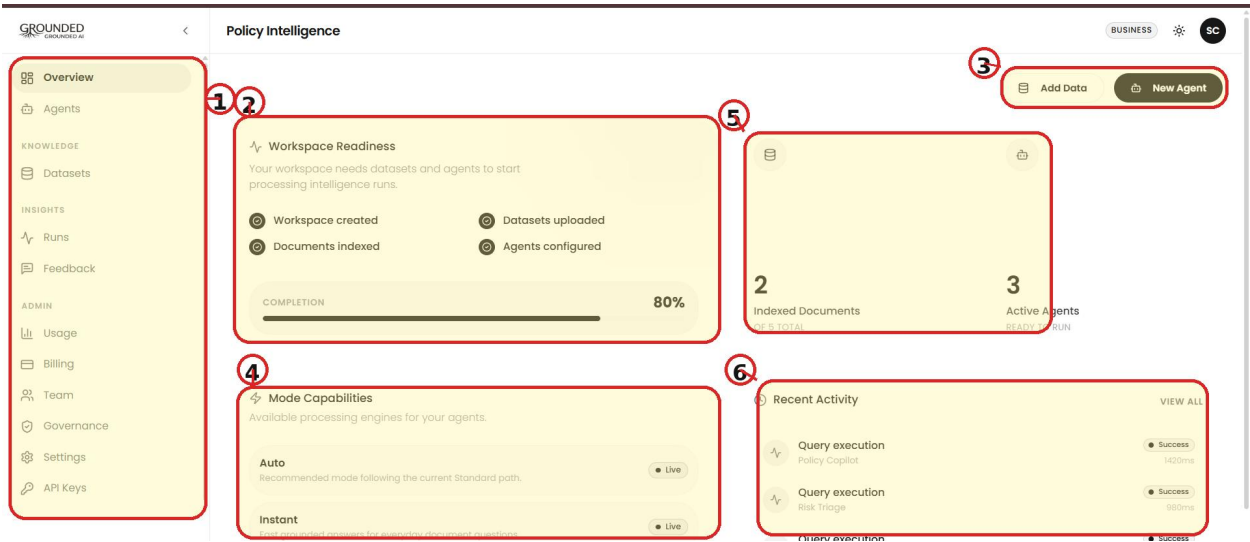


Figure 5.10: Workspace overview and readiness dashboard of the Grounded AI platform.

Callout guide for Figure 5.10:

Callout	Description
1	Sidebar navigation for overview, agents, datasets, runs, settings, and API keys
2	Workspace readiness card showing preparation status
3	Main action buttons for adding data and creating a new agent
4	Mode capabilities showing available processing modes
5	Workspace summary metrics such as indexed documents and active agents
6	Recent activity list showing previous query executions

Step 2: Choose an Agent Setup Path

To create a new agent, the user selects the **New Agent** option and is taken to the agent setup page. The platform provides different setup paths, including using a pre-built template, describing a goal through a prompt-based setup, or starting from a blank canvas. In the shown workflow, the user selects a pre-built agent template to reduce setup effort and quickly configure a grounded assistant.

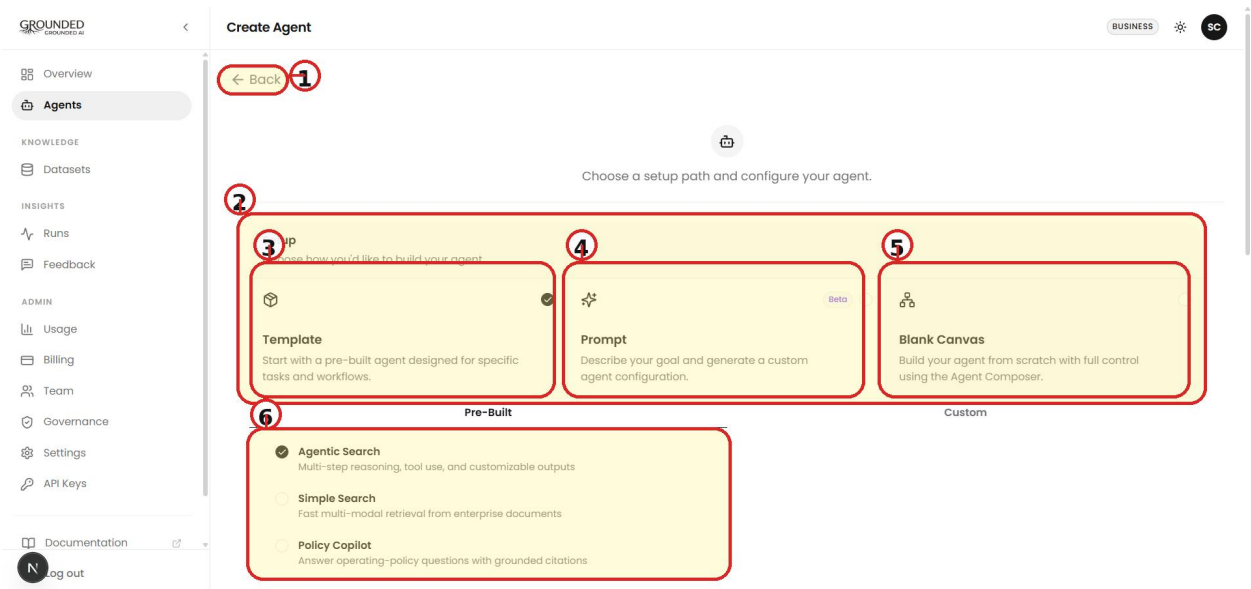


Figure 5.11: Agent setup path selection interface.

Callout guide for Figure 5.11:

Callout	Description
1	Back button for returning to the previous page
2	Agent setup section where the user chooses how to build the agent
3	Template setup option for selecting a pre-built agent
4	Prompt setup option for generating an agent from user instructions
5	Blank canvas option for manual agent configuration

6	Pre-built agent templates such as Agentic Search, Simple Search, and Policy Copilot
---	---

Step 3: Configure the Agent

After selecting the setup path, the user configures the agent in the Agent Composer. The user provides an agent name, chooses the default processing mode, writes instructions, selects the response policy, and attaches one or more knowledge sources. The interface also shows a summary and readiness checklist to confirm that the agent has the required information before it is used.

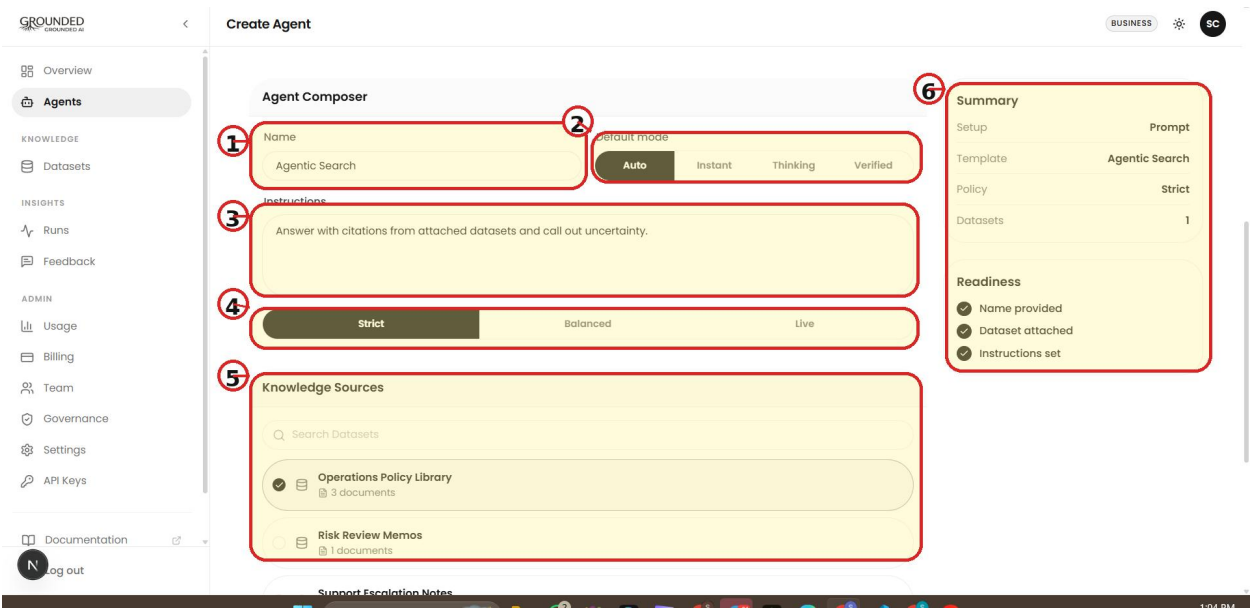


Figure 5.12: Agent composer with mode, policy, instruction, and dataset configuration.

Callout guide for Figure 5.12:

Callout	Description
1	Agent name field
2	Default mode selector: Auto, Instant, Thinking, and Verified
3	Instruction field used to guide the agent’s response behavior

4	Response policy selector, such as Strict, Balanced, or Live
5	Knowledge source section for attaching datasets
6	Summary and readiness panel showing setup status

Step 4: Select Knowledge Sources and Launch the Agent

The user then selects the dataset that the agent should use as its knowledge source. The dataset list displays available knowledge collections and the number of documents inside each dataset. The user may also create a new dataset if the required knowledge source is not already available. After selecting the appropriate dataset, the user can either continue customizing the agent or start chatting directly.

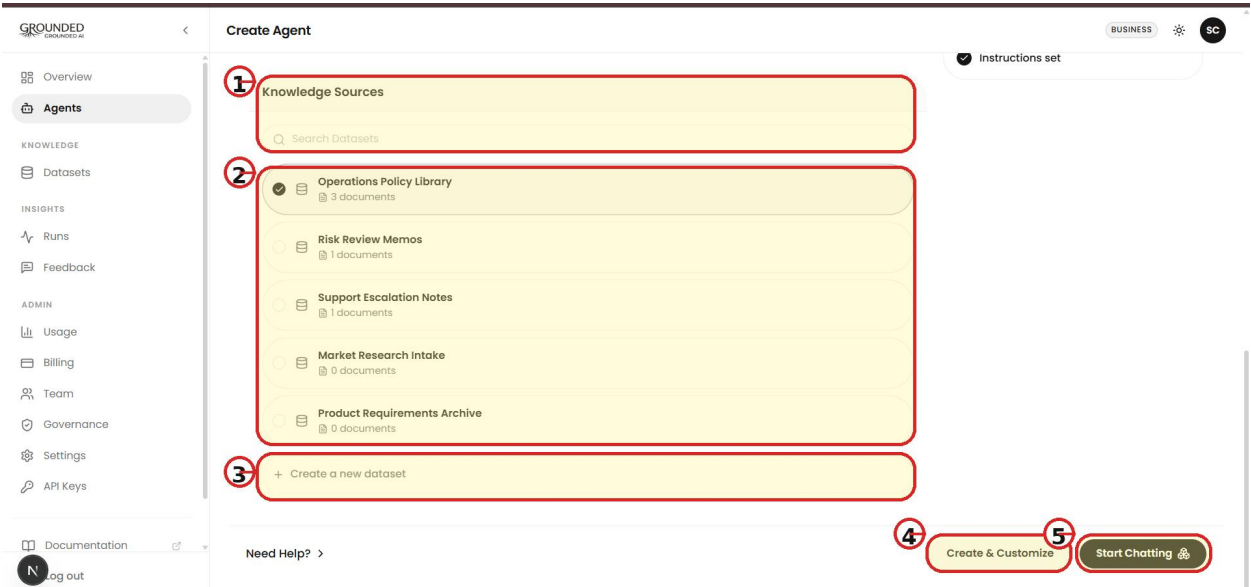


Figure 5.13: Dataset selection and agent launch controls.

Callout guide for Figure 5.13:

Callout	Description
1	Knowledge source search and selection area

2	Available datasets that can be attached to the agent
3	Option to create a new dataset
4	Create and Customize button for continuing configuration
5	Start Chatting button for opening the agent chat interface

Step 5: Ask Questions and Review Grounded Answers

Once the agent is configured, the user can ask questions through the chat interface. The system generates an answer based on the attached datasets and displays confidence information, citation count, and supporting evidence. The right-side evidence panel allows the user to review source text from the referenced document, which improves transparency and helps users verify the answer.

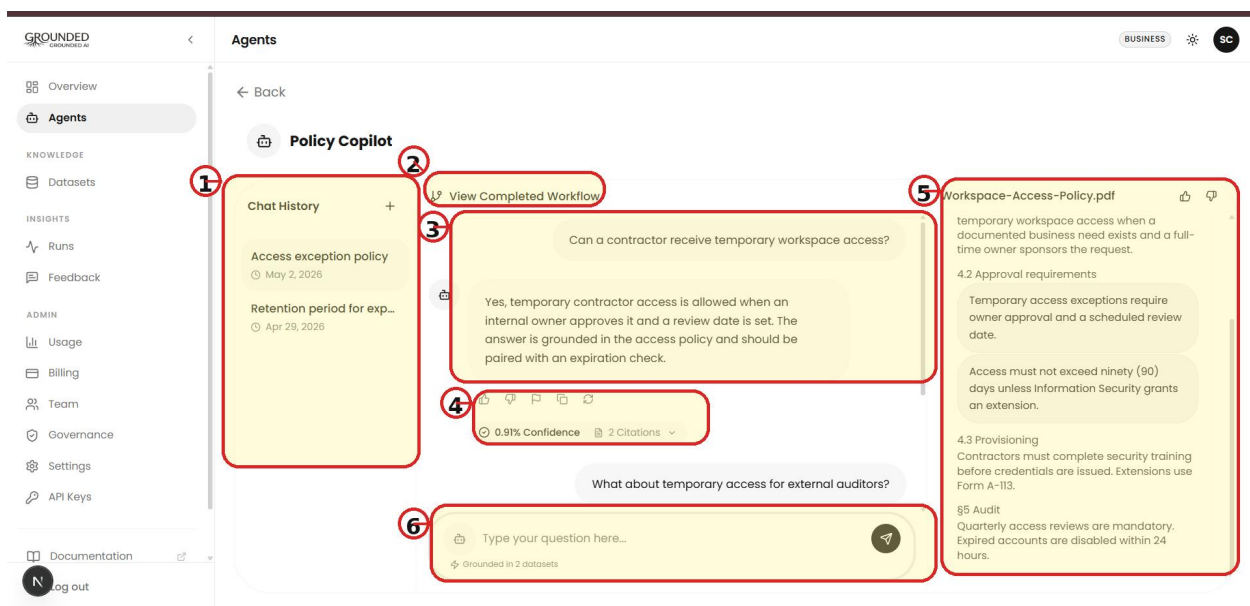


Figure 5.14: Grounded chat interface with citations and supporting evidence viewer.

Callout guide for Figure 5.14:

Callout	Description
1	Chat history panel showing previous conversations
2	Completed workflow link for reviewing the agent workflow
3	User question and generated grounded response
4	Confidence score, citation count, and answer action controls
5	Evidence viewer showing the source document and cited passages
6	Message input area for submitting follow-up questions

The user manual shows that the implemented platform supports the main workflow required for grounded organizational document intelligence. A user can review workspace readiness, create an agent, select processing behavior, attach datasets, start a grounded chat session, and inspect citations and supporting evidence. This confirms that the system is not only a backend retrieval pipeline, but also a usable product interface for dataset-based grounded interaction.

5.2 Deciding on the Development Tools

The toolchain was selected to support two connected needs: a multi-stage grounding backend and a product interface that ordinary users could follow without understanding retrieval engineering. The final stack therefore combines a FastAPI backend, a Next.js frontend, relational and vector storage, object storage, and container-based local infrastructure.

5.2.1 Programming Languages

Python was used for backend API services, document processing, ingestion workers, retrieval orchestration, and provider integration. TypeScript was used in the Next.js frontend so that page interactions, API responses, and shared UI state could be maintained more safely as the product grew.

Python was a practical choice because the backend depends on asynchronous APIs, document handling, indexing jobs, and model-provider adapters. TypeScript supported the frontend by making product flows

such as authentication, datasets, agents, chat, runs, and settings easier to organize and validate during development.

5.2.2 Backend Framework and Tools

The backend stack was built around FastAPI, SQLAlchemy Async ORM, and Alembic migrations. FastAPI provided the versioned REST API layer, SQLAlchemy managed the persistent product and pipeline models, and Alembic supported controlled schema evolution as the project expanded.

FastAPI was suitable for asynchronous request handling, request validation, and modular service organization. SQLAlchemy provided a common persistence layer for tenants, workspaces, datasets, documents, ingestion jobs, agents, conversations, messages, API keys, and query traces. Alembic was necessary because the database schema changed gradually as the product features matured.

Provider-specific behaviour was isolated inside adapter modules for embeddings and answer generation. This separation allowed the core query flow to remain stable while model providers could be configured, replaced, or disabled during development.

Gemini and OpenAI were used as model/provider adapters only. They were not used as the project database. The implemented storage responsibilities were separated into PostgreSQL for operational records, Qdrant for dense vectors, and MinIO for uploaded and extracted artefacts.

Domain routes were grouped under a versioned API structure for authentication, workspaces, datasets, documents, agents, conversations, grounded query execution, runs, dashboard views, API keys, capabilities, and health checks.

5.2.3 Frontend Framework and Tools

The frontend was implemented with Next.js and TypeScript. Next.js was used to organize the product into route-based pages for landing, authentication, onboarding, dashboard, datasets, dataset detail, agents, grounded chat, runs, settings, and protected workspace flows. Server data was handled through typed frontend utilities and page-level state, with TanStack Query used where cached request state was useful.

This frontend structure supported the full product loop: sign-up, sign-in, onboarding, dataset creation, file upload, agent creation, dataset attachment, grounded chat, run inspection, dashboard review, and API key management. Automated frontend regression coverage remained limited, so the strongest verification evidence in Chapter Six comes from backend unit and integration tests plus implemented page validation.

5.2.4 Storage and Retrieval Tools

The storage and retrieval architecture was divided deliberately so that each type of data had a clear responsibility. The system does not treat storage as a single database problem; instead, operational records, vector search, uploaded files, and local infrastructure support are handled separately.

PostgreSQL manages the structured product and pipeline records of the platform. It stores tenants, workspaces, datasets, documents, ingestion jobs, agents, conversations, messages, API keys, and query traces. It also supports sparse lexical retrieval over stored document chunks, which helps the system find exact terms, identifiers, and policy-style wording that may not be captured reliably by semantic similarity alone.

Qdrant provides the dense vector retrieval layer. After documents are processed into chunks, embeddings are stored in Qdrant so that the system can retrieve semantically related passages during grounded question answering. This complements PostgreSQL-based lexical search and supports the hybrid retrieval strategy used by the pipeline.

MinIO stores uploaded documents and extracted text artifacts. Keeping binary files and text artifacts in object storage prevents the relational database from becoming overloaded with large file content while still allowing the ingestion pipeline to retrieve source material when needed.

Redis was included only as part of the local Docker infrastructure support. In the delivered application logic, Redis was not used as the application queue, cache, or session store. Its role should therefore be described as development and deployment support rather than as a core application component.

5.2.5 Development Environment and Infrastructure Tools

Development was carried out using Visual Studio Code, Git, Docker Compose, and Pytest. Docker Compose made the local environment repeatable by running PostgreSQL, Qdrant, MinIO, and related services together, while Pytest provided the main automated verification framework for backend unit and integration behaviour. Together, these tools made the project repeatable enough to develop as both a pipeline and a product.

5.2.6 Version Control Practice

The project was maintained in a Git repository with separate areas for backend implementation, frontend implementation, and documentation. Feature work, database migrations, and evidence notes were developed incrementally so that the final report could remain aligned with the actual implementation.

5.2.7 Documented Product Capabilities

The following capabilities were implemented:

1. **Authentication:** preview email/password sign-in and API-key session use under tenant scope.
2. **Datasets:** dataset create, list, retrieve, and update with file upload into a dataset.
3. **Ingestion:** asynchronous pipeline with ingestion status visibility (not numeric percentage progress).
4. **Agents:** agent create, list, retrieve, update, and dataset attach/detach (no verified delete route).
5. **Chat:** grounded agent chat linked to persisted runs and messages.
6. **Runs:** inspectable traces with citations and trust-related fields.
7. **Access control:** API-key authentication and tenant scoping on protected routes.

Capabilities listed on Settings roadmap cards (connectors, enterprise governance, billing) are future work, not delivered features.

Tool / Technology	Area Used	Purpose in the System	Codebase Evidence
Python	Backend	Implements API services, ingestion workers, retrieval, query orchestration, and grounding logic	backend/app/main.py, backend/pyproject.toml, requirements.txt
TypeScript	Frontend	Provides typed frontend workflows and safer API-response handling in the Next.js interface	TypeScript configuration and Next.js frontend source

FastAPI	Backend	Exposes REST API routes, request validation, and async service orchestration	FastAPI application entry and versioned API router
SQLAlchemy Async ORM	Backend	Persists tenants, workspaces, datasets, documents, jobs, traces, agents, and conversations	backend/app/core/database.py, backend/app/models/*
Alembic	Backend	Manages schema migrations for product and pipeline tables	backend/alembic/env.py, backend/alembic/versions/.py, backend/alembic.ini
Next.js	Frontend	Builds the route-based product interface and page workflows	Next.js application structure and package configuration
Next.js routing	Frontend	Organizes landing, authentication, onboarding, workspace, and protected application routes	Next.js routing and protected page structure
TanStack Query	Frontend	Fetches and caches backend data for dashboard, datasets, agents, chat, and runs	frontend project configuration and Next.js pages

Frontend test/build tooling	Frontend	Supports frontend development and limited automated UI smoke testing	Frontend package and test configuration
PostgreSQL	Backend / retrieval	Stores operational product state and supports sparse lexical retrieval over chunk records	Database models, migrations, and sparse indexing services
Qdrant	Backend / retrieval	Stores and searches dense vector embeddings for semantic retrieval	Vector client, dense indexing service, and Docker Compose configuration
MinIO	Backend / storage	Stores uploaded documents and normalized extracted-text artifacts	Object storage, document upload, and extraction services
Redis	Infrastructure	Provides local Docker infrastructure support only; not used as core application logic	Local Docker Compose configuration
OpenAI / Gemini provider adapters	Backend / AI	Supplies embeddings and answer generation through provider-	backend/app/core/openai_embeddings.py, backend/app/core/gemini_embeddings.py, backend/app/core/openai_generator.py, backend/app/core/gemini_generator.py

		specific adapters	
Docker Compose	Infrastructure	Runs PostgreSQL, Qdrant, MinIO, Redis, and related local services together	docker-compose.yml, docker-compose.dev.yml

Table 5.3 Development Tools and Their Purpose

5.3 Developing the Solution

We followed the same two-part structure used throughout the project: first the grounding pipeline, then the product built around it.

Backend Module	Main Responsibility	Key File Paths	Routes / Services / Models
Application entry and API routing	Creates the FastAPI app and mounts product routes	backend/app/main.py, backend/app/api/v1/router.py	create_app, api_router; routes under /v1
Authentication and tenant context	Email sign-up/sign-in, API key validation, tenant resolution	backend/app/api/v1/auth.py, backend/app/api/deps.py, backend/app/services/auth.py, backend/app/core/security.py	email_sign_up, email_sign_in, get_tenant_context, TenantContext; models Tenant, APIKey
Workspaces	Workspace creation, listing, retrieval, and update	backend/app/api/v1/workspaces.py, backend/app/services/workspaces.py	create_workspace_route, list_workspaces_route, create_workspace; model Workspace

Datasets	Dataset create, list, retrieve, and update; document listing; dataset-scoped upload; ingestion-job listing	backend/app/api/v1/datasets.py, backend/app/services/datasets.py	create_dataset_route, list_dataset_documents_route, upload_dataset_document_route; model Namespace
Documents and upload	File validation, object storage, duplicate detection, document and job creation	backend/app/api/v1/documents.py, backend/app/services/documents.py, backend/app/core/storage.py	upload_document, create_document_upload, get_ingestion_job_status; models Document, IngestionJob
Ingestion orchestration	Claims ingestion jobs and runs the standard ingestion pipeline	backend/app/services/ingestion.py, backend/app/workers/ingestion.py, backend/app/workers/pipeline.py	claim_ingestion_job, run_standard_ingestion_pipeline, run_ingestion_job
Extraction and normalization	Extracts source files into normalized text artifacts	backend/app/services/extraction.py, backend/app/workers/extraction.py	extract_document_artifact, _normalize_text, run_extraction_job
Chunking	Produces structure-aware chunk manifests for indexing	backend/app/services/chunking.py, backend/app/pipeline/orchestrator.py, backend/app/pipeline/contracts.py	chunk_extracted_document, build_chunk_manifest, _build_structure_aware_chunks; model DocumentChunkRecord

Sparse indexing	Indexes chunks for PostgreSQL-backed lexical retrieval	backend/app/services/sparse_indexing.py, backend/app/workers/sparse_indexing.py	sparse_index_document, run_sparse_indexing_job
Dense indexing	Embeds chunks and upserts vectors into Qdrant	backend/app/services/dense_indexing.py, backend/app/workers/dense_indexing.py, backend/app/core/embeddings.py	dense_index_document, embed_texts, run_dense_indexing_job
Query analysis	Builds query profiles and retrieval plans before search	backend/app/core/query_analysis.py, backend/app/services/query.py	build_query_profile, build_query_plan, build_retrieval_query_text
Hybrid retrieval and reranking	Combines sparse and dense hits, fuses scores, and reranks candidates	backend/app/services/retrieval.py, backend/app/core/reranker.py	sparse_retrieve_chunks, dense_retrieve_chunks, retrieve_hybrid_candidates, fuse_retrieval_hits
Evidence packaging	Ranks and packages retrieved chunks for generation	backend/app/services/evidence.py	package_evidence, _rank_hits, _select_hits_for_query
Query orchestration	Coordinates retrieval, generation, shaping, and trace persistence	backend/app/services/query.py, backend/app/api/v1/query.py	execute_standard_query, query_documents; model QueryTrace
Answering and generation	Generates grounded drafts from packaged evidence	backend/app/services/answering.py, backend/app/services/generation.py,	answer_from_evidence, generate_answer_from_evidence, _validate_citation_contract

		backend/app/core/llm_client.py	
Response shaping and trust metadata	Shapes final API responses with citations, confidence, and support summaries	backend/app/services/response_shaping.py, backend/app/services/trust.py, backend/app/schemas/query.py	shape_grounded_response, confidence_label_for_score, GroundedAnswerResponse, CitationResponse
Agents	Creates and updates agents; attaches or detaches datasets	backend/app/api/v1/agents.py, backend/app/services/agents.py	create_agent_route, attach_dataset_to_agent_route, detach_dataset_from_agent_route; models Agent, AgentDataset
Conversations and messages	Manages conversations and persists chat messages	backend/app/api/v1/conversations.py, backend/app/services/conversations.py, backend/app/services/messages.py	create_conversation_route, list_conversation_messages_route, create_message; models Conversation, Message
Agent chat	Executes grounded chat turns and links responses to runs	backend/app/services/agent_chat.py, backend/app/api/v1/agents.py	execute_agent_chat_turn, chat_with_agent_route
Runs / traces	Lists and returns persisted grounded execution traces	backend/app/api/v1/runs.py, backend/app/services/runs.py, backend/app/models/query_trace.py	list_runs_route, get_run_route, _build_run_response; model QueryTrace

Dashboard	Aggregates counts, recent runs, and recent ingestion jobs	backend/app/api/v1/dashboard.py, backend/app/services/dashboard.py	get_dashboard_summary_route, list_dashboard_recent_runs_route, list_dashboard_recent_jobs_route
API keys	Lists, creates, and revokes tenant API keys	backend/app/api/v1/api_keys.py, backend/app/services/api_keys.py	list_api_keys_route, create_api_key_route, revoke_api_key_route; model APIKey

Table 5.4 Backend Implementation Summary

Page / Component	User function	Key backend interaction
HomePage	Public landing	None
LoginPage	Sign-in; attach workspace	POST /v1/auth/email/sign-in, GET /v1/workspaces
SignUpPage	Registration	POST /v1/auth/email/sign-up
OnboardingPage	First workspace	POST /v1/workspaces
DashboardPage	Overview; recent runs and jobs	/v1/dashboard/*
DatasetsPage	List and create datasets	GET /v1/datasets, POST /v1/datasets
DatasetDetailPage	Upload; documents and jobs	dataset upload, documents, ingestion-jobs routes
AgentsPage	List and create agents	GET /v1/agents, POST /v1/agents
AgentChatPage	Chat; attach dataset; inspect	agent chat, conversations, messages, runs

	answer	
RunsPage	Run inspection	GET /v1/runs, GET /v1/runs/{run_id}
SettingsPage	API key management	/v1/api-keys routes

Table 5.3. Frontend Mapping (Page → API → Function)

Pipeline Stage	Purpose	Implementation Evidence	Related File Paths	Verification Status
Document upload and validation	Accepts supported files into a dataset and validates upload type	create_document_upload, _SUPPORTED_UPLOAD_TYPES, upload_dataset_document_route	backend/app/services/documents.py, backend/app/api/v1/datasets.py, Dataset detail page	Verified
Canonical storage and duplicate detection	Stores uploaded bytes in object storage and prevents duplicate re-ingestion in the same dataset	upload_bytes, create_document_upload, SHA-256 checksum, already_exists	backend/app/core/storage.py, backend/app/services/documents.py	Verified
Ingestion job tracking	Creates and tracks ingestion jobs from upload through	create_document_upload, claim_ingestion_job, get_ingestion_job_status, StatusBadge	backend/app/services/ingestion.py, backend/app/api/v1/documents.py, Dataset detail page, Next.js page layerages/DashboardPage.tsx	Verified

	indexing			
Extraction and normalization	Converts uploaded files into normalized text artifacts for downstream processing	extract_document_artifact, _normalize_text, derive_extracted_text_key	backend/app/services/extraction.py, backend/app/workers/extraction.py	Verified
Structure-aware chunking	Splits normalized text into retrieval-oriented chunks using structure_aware_v1	chunk_extracted_document, build_chunk_manifest, _build_structure_aware_chunks, ChunkingConfig	backend/app/services/chunking.py, backend/app/pipeline/orchestrator.py, backend/app/pipeline/contracts.py	Partially Verified
Sparse indexing	Builds PostgreSQL-backed lexical indexes over chunk records	sparse_index_document, run_sparse_indexing_job, sparse_retrieve_chunks	backend/app/services/sparse_indexing.py, backend/app/workers/sparse_indexing.py, backend/app/services/retrieval.py	Verified
Dense indexing	Embeds chunks and stores vectors in Qdrant	dense_index_document, embed_texts, run_dense_indexing_job, search_dense_points	backend/app/services/dense_indexing.py, backend/app/workers/dense_indexing.py, backend/app/core/qdrant_client.py	Verified

Query analysis	Interprets the user question and prepares a retrieval plan before search	build_query_profile, build_query_plan, refine_query_plan_for_execution_tier, build_retrieval_query_text	backend/app/core/query_analysis.py, backend/app/services/query.py	Verified
Hybrid retrieval	Retrieves evidence through sparse and dense paths and fuses the results	sparse_retrieve_chunks, dense_retrieve_chunks, fuse_retrieval_hits, retrieve_hybrid_candidates	backend/app/services/retrieval.py	Verified
Ranking and evidence selection	Reranks fused hits and packages the best support material for generation	package_evidence, _rank_hits, _rerank_fused_hits_for_query, resolve_retrieval_reranker	backend/app/services/evidence.py, backend/app/services/retrieval.py, backend/app/core/reranker.py	Verified
Grounded answer generation	Generates an answer draft constrained to selected evidence	execute_standard_query, generate_answer_from_evidence, answer_from_evidence	backend/app/services/query.py, backend/app/services/generation.py, backend/app/services/answering.py	Verified
Response shaping and trust metadata	Returns structured answers with citations, confidence, and support summaries	shape_grounded_response, CitationResponse, confidence_label_for_score, support_summary_for_response	backend/app/services/response_shaping.py, backend/app/services/trust.py, Runs page, Agent chat page	Verified

Embedded verification (citation-contract)	Applies citation-contract checks and maps verification status into traces and UI	<code>_validate_citation_contract</code> , <code>shape_grounded_response</code> , <code>_verification_status_for_trace</code>	<code>backend/app/services/generation.py</code> , <code>backend/app/services/response_shaping.py</code> , <code>backend/app/services/runs.py</code>	Partially Verified
Run trace persistence	Persists each grounded execution for later inspection	<code>_persist_query_trace</code> , <code>list_runs_route</code> , <code>get_run_route</code> , <code>RunResponse</code>	<code>backend/app/services/query.py</code> , <code>backend/app/services/runs.py</code> , <code>backend/app/models/query_trace.py</code> , <code>Runs</code> page	Verified
Agent chat execution path	Delivers the pipeline through conversational product use and links messages to runs	<code>execute_agent_chat_turn</code> , <code>chat_with_agent_route</code> , <code>Message.run_id</code>	<code>backend/app/services/agent_chat.py</code> , <code>backend/app/api/v1/agents.py</code> , <code>Agent chat</code> page	Verified

Table 5.4. Grounded AI Pipeline Implementation

Implemented Grounding Pipeline

Verified document-ingestion and grounded-query stages

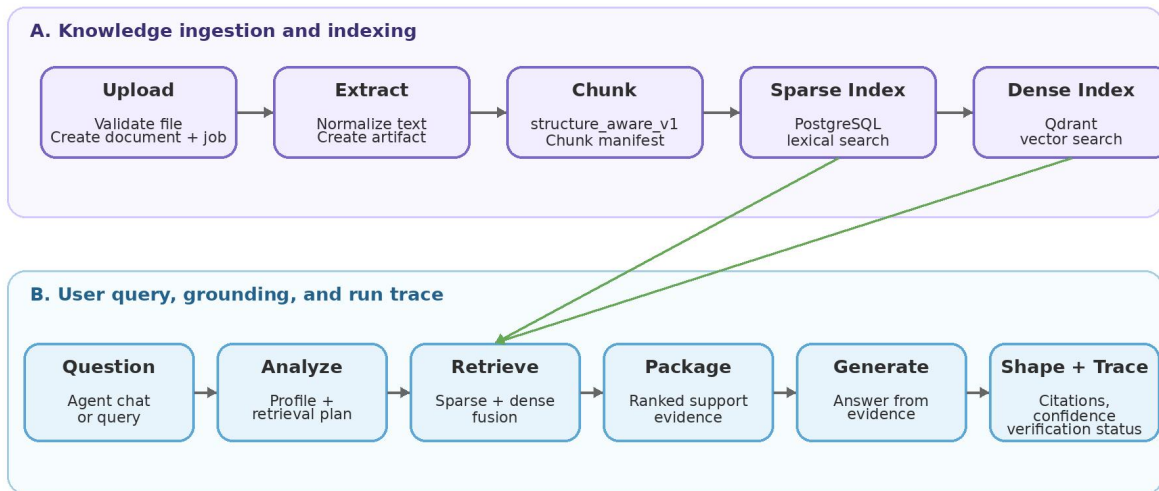


Figure 5.3. Implemented grounding pipeline (shared across tiers).

Dataset-to-Agent-to-Grounded-Chat Workflow

Verified product journey from sign-in to run inspection

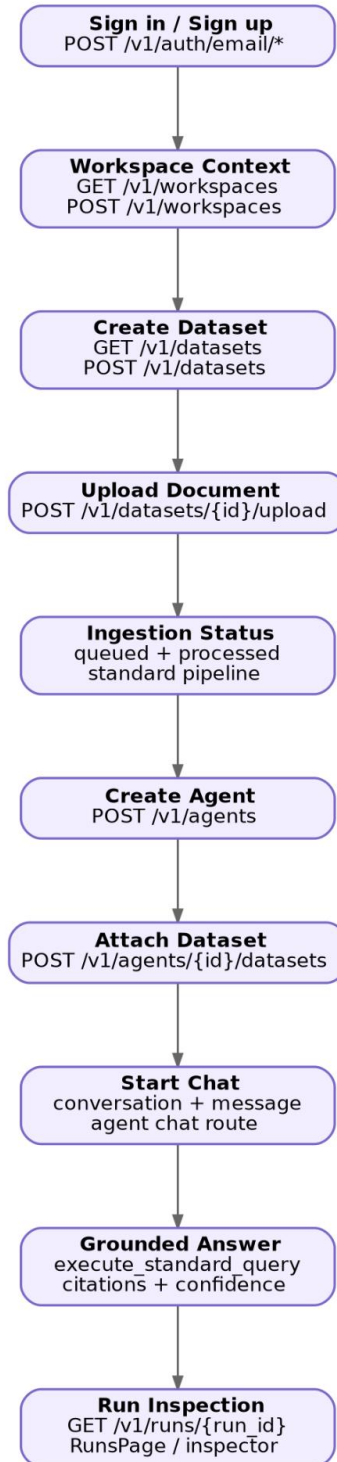


Figure 5.4. Dataset-to-agent-to-grounded-chat workflow.

Entity-Relationship Model (Implemented Scope)

Core operational tables and verified relationships

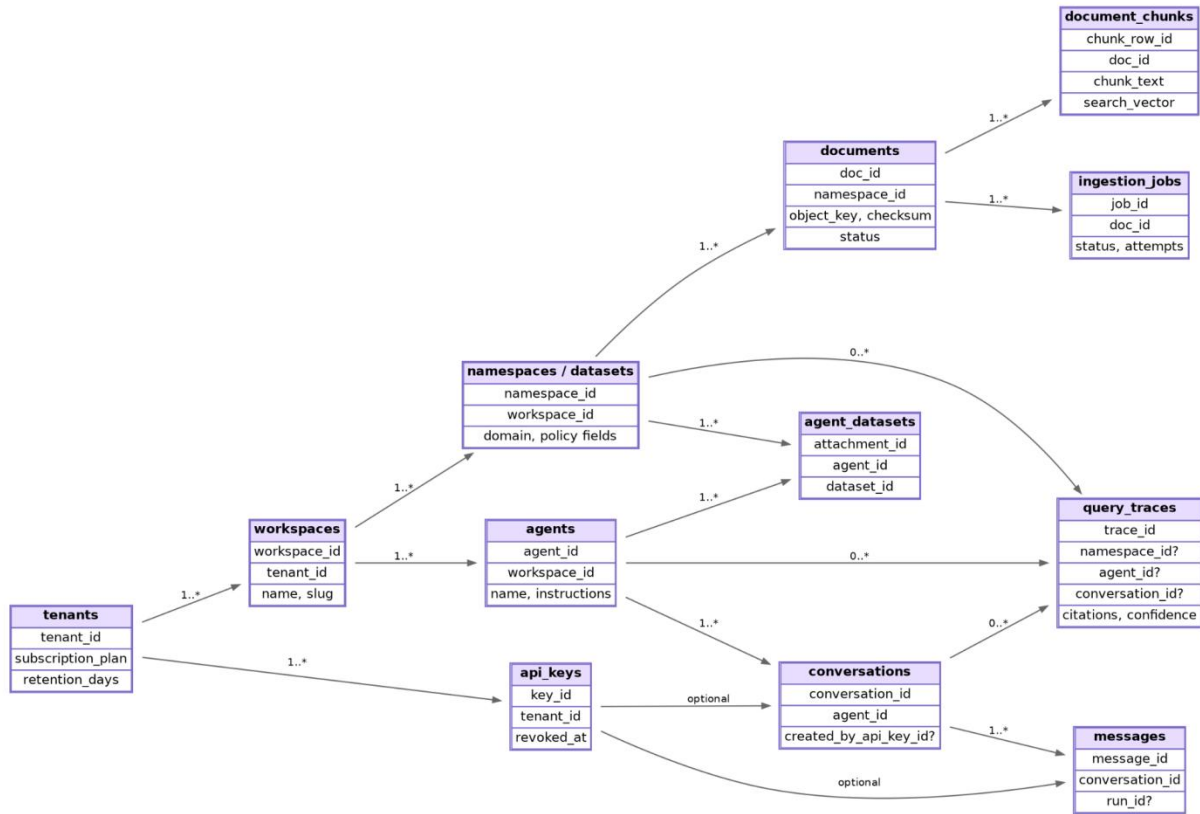


Figure 5.5. Entity-relationship model for the implemented data schema.

Database schema documentation is provided by the entity model (Figure 5.5), which lists product and pipeline tables including namespaces, documents, document chunks, agents, agent-dataset attachments, conversations, messages, and query traces. Optional relationships in the trace records are treated as nullable because not every run is produced through the same product path.

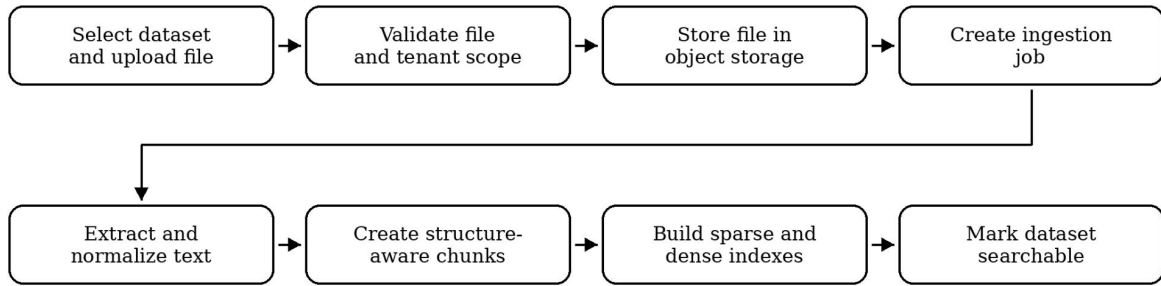


Figure 5.6. Activity diagram - document upload and ingestion (decision points: file type valid, duplicate checksum, job status).

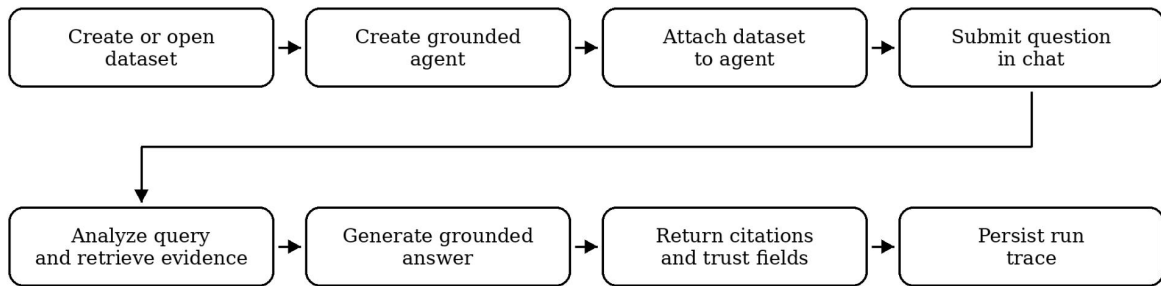


Figure 5.7. Activity diagram - dataset, agent, and grounded chat (decision points: dataset attached, mode/tier routing, clarification vs answer).

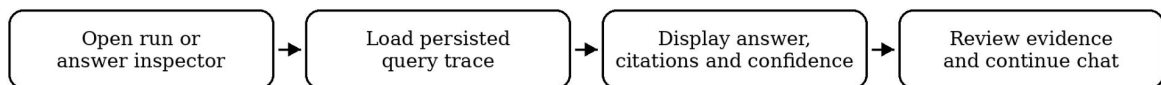
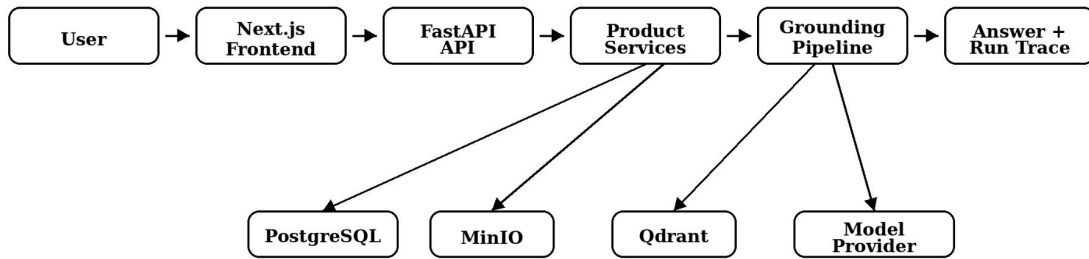


Figure 5.8. Activity diagram - run inspection and trust review (decision points: verification status, partial vs supported answer).



The frontend presents the workflow, the API applies tenant context, product services manage datasets and agents, and the grounding pipeline retrieves evidence before returning citations and trace details.

Figure 5.9. Collaboration diagram - frontend, API, services, stores, and retrieval indexes.

The subsections below walk through how each part was built in code.

5.3.1 Backend Application Entry and API Routing

The application starts from the FastAPI application factory, which mounts the versioned API router. This entry point matters because the system is implemented as a product API.

The main API areas are summarised below.

Domain	API module	Typical route prefix
Authentication	backend/app/api/v1/auth.py	/v1/auth/
Workspaces	backend/app/api/v1/workspaces.py	/v1/workspaces
Datasets	backend/app/api/v1/datasets.py	/v1/datasets
Documents and ingestion jobs	backend/app/api/v1/documents.py	/v1/documents, /v1/ingestion-jobs/{job_id}
Agents and agent chat	backend/app/api/v1/agents.py	/v1/agents, /v1/agents/{agent_id}/chat
Conversations	backend/app/api/v1/conversations.py	/v1/conversations, /v1/agents/{agent_id}/conversations

Grounded query	backend/app/api/v1/query.py	POST /v1/query
Runs	backend/app/api/v1/runs.py	/v1/runs, /v1/runs/{run_id}
Dashboard	backend/app/api/v1/dashboard.py	/v1/dashboard/
API keys	backend/app/api/v1/api_keys.py	/v1/api-keys
Capabilities	backend/app/api/v1/capabilities.py	/v1/capabilities
Health	backend/app/api/v1/health.py	/health/live, /health/ready

The frontend API utility layer wraps these routes for the Next.js pages. Splitting the backend by domain made it easier to grow the product incrementally while keeping each feature area testable on its own.

5.3.2 Authentication and Tenant Context Implementation

Access is handled through API-key authentication and tenant scoping, not through a full enterprise identity platform.

Protected routes resolve a tenant context before allowing access to product resources. API keys are validated securely, and major resources such as workspaces, datasets, agents, conversations, runs, and API keys are loaded through tenant-aware service methods.

For the product demo path, preview email and password sign-up/sign-in are supported. After authentication, the client stores the returned key and continues into workspace selection or onboarding.

Authenticated product routes are protected before users can reach workspace-level pages. Role-based access control, audit logging, billing, and enterprise administration were not implemented as delivered features in the verified codebase.

5.3.3 Workspace and Dataset Implementation

A core product decision was to avoid one undifferentiated document pile. Knowledge is grouped under workspaces, then datasets.

Workspaces are created, listed, and persisted as the top-level organizational context for users. In the interface, workspace context is established during onboarding or when a user signs in and selects an available workspace. A full workspace administration screen remains a future product improvement.

Datasets are exposed to users as named knowledge collections and are persisted internally as namespace records. The implemented API supports dataset creation, listing, retrieval, update, dataset-scoped upload, document listing, and ingestion-job listing. Users interact with these capabilities through the Next.js dataset pages.

5.3.4 Document Upload and Ingestion Implementation

Uploads enter the system through dataset-scoped and general document upload endpoints. The backend validates the file, stores the object, records metadata, and prepares the document for asynchronous ingestion.

The upload path validates file types through `_SUPPORTED_UPLOAD_TYPES`, stores the raw object with `upload_bytes`, records metadata on documents, and creates an `ingestion_jobs` row. Duplicate files are detected with a SHA-256 checksum and an `already_exists` flag scoped per dataset, which prevents accidental double-ingestion during repeated uploads.

Ingestion does not finish inside the upload request itself. The API returns a queued job, and the standard ingestion pipeline processes the document asynchronously. This keeps uploads responsive while extraction, chunking, sparse indexing, and dense indexing continue in the background. The dataset detail page shows job state rather than pretending indexing is instant.

5.3.5 Extraction and Normalization Implementation

Uploaded files are not chunked directly in their original binary form. They are first converted into normalized text artifacts so that later processing stages can work with consistent textual input.

Extracted text is stored in MinIO, while document metadata stays in PostgreSQL. There is no separate normalization microservice; normalization is simply the cleanup and artifact-writing step needed before chunking can run reliably across PDF, text, and similar supported inputs.

5.3.6 Structure-Aware Chunking Implementation

We implemented a structure-aware chunking strategy that produces retrieval-oriented units from normalized text. Rather than splitting documents at arbitrary character limits, our approach identifies structural boundaries in the text so that logical units like definitions, conditions, and conclusions remain intact within a single chunk. The ingestion pipeline coordinates this process and records chunk metadata so that retrieved evidence can be linked back to its source document.

The chunking stage produces a manifest of chunks and persists them with structure-related metadata. These chunk records become the foundation for both sparse and dense indexing.

Our chunking algorithm (`structure_aware_v1`) works as follows:

1. The pipeline reads normalized extracted text from object storage.
2. A chunk manifest is created using the structure-aware chunking configuration.
3. Chunk records are persisted with source and structure metadata.
4. The resulting chunks are then passed into sparse and dense indexing.

The practical goal was to produce evidence units that are more coherent than fixed-size splits, so that retrieval and citations have cleaner boundaries to work with. This directly addressed one of the key weaknesses we identified in conventional RAG systems, where rigid segmentation frequently breaks contextual meaning across chunk boundaries and degrades retrieval quality.

5.3.7 Sparse and Dense Indexing Implementation

Retrieval is intentionally split across two indexes. Sparse indexing writes searchable chunk records in PostgreSQL so that exact words, identifiers, and domain-specific terminology can be retrieved reliably. Dense indexing embeds chunks and stores vectors in Qdrant so that semantic similarity search can retrieve conceptually related evidence.

Using both paths was a direct response to a common weakness in basic RAG setups that rely on only one retrieval mode. The implementation is intended to improve over that single-path design, even though benchmark results in the repository are mixed and should not be overstated as universal superiority.

5.3.8 Query Analysis and Transformation Implementation

Before retrieval begins, the question is interpreted by the query analysis component. The resulting profile and retrieval plan help the pipeline choose cleaner retrieval text and a more appropriate search strategy.

This step exists because vague user questions are one of the main failure modes in basic RAG. Passing the raw question straight to vector search often retrieves the wrong evidence. The implemented pipeline at least attempts to shape retrieval text and control rewrites before search begins.

Query analysis is internal to orchestration. There is no separate public endpoint whose only job is to analyse a query outside the main grounded flow.

5.3.9 Hybrid Retrieval Implementation

The retrieval service combines sparse and dense candidates through fusion before evidence is selected for generation. Instead of trusting a single dense top-k ranking, the service merges candidate sets and prepares a narrower evidence pool for the generator.

Hybrid retrieval is one of the clearest mechanisms intended to improve over basic RAG in this codebase. It is implemented and covered by unit tests. The supplementary benchmark evidence is mixed, however, so measured improvement must be tied to specific reported results rather than stated as universal superiority.

5.3.10 Ranking and Evidence Selection Implementation

Initial retrieval can still return more material than the generator should receive. The evidence selection stage therefore ranks and packages the strongest supporting chunks before answer generation.

The point of this stage is straightforward: the model should receive the best available support chunks, not simply the first chunks that happened to score highly in one retrieval channel.

5.3.11 Grounded Query Orchestration Implementation

The full grounded query flow is coordinated by the standard query execution service. It resolves tenant and dataset context, builds the retrieval plan, retrieves and packages evidence, generates a grounded answer, shapes the response, and persists a query trace.

Developers can call the pipeline through the grounded query API. In normal product use, the more important entry point is agent chat, which runs the same orchestration path, persists messages, and returns run metadata to the client.

5.3.12 Response Shaping and Structured Output Implementation

Raw model text alone would be weak for organizational use. Response shaping converts the generated answer into a structured result that includes citations, confidence labels, support summaries, and verification-related status fields.

These values are persisted in query traces and rendered in the run and chat inspection interfaces. That structure makes answers inspectable in the UI instead of appearing as unsupported prose.

5.3.13 Verification Implementation

Verification is embedded behaviour, not a standalone verifier service. Citation-contract checks are embedded in the generation and response-shaping flow. The system checks whether generated citations follow the expected contract and maps the outcome into stored traces and run responses.

This improves traceability compared with a minimal RAG answer, but it does not amount to a separate hallucination-prevention engine or contradiction checker. Those stronger forms of verification were left as future work.

5.3.14 Agent Implementation

Agents are reusable assistants scoped to a workspace. The backend supports agent creation, listing, retrieval, update, and dataset attach/detach operations.

Links between agents and datasets are stored as attachment records. Users create agents on the agent page and attach datasets from the grounded chat experience.

Two limitations are worth noting here. No agent delete route was found in the repository, and although backend detach exists, no frontend detach control was found in the UI review. Agent management is therefore create, list, retrieve, update, and dataset attach/detach, not full CRUD including delete.

5.3.15 Conversation and Chat Implementation

Chat is the main way many users experience the system. A typical turn creates or reuses a conversation, stores the user message, runs the grounded query flow, stores the assistant response, and links the result to a persisted run.

Messages are created through the chat workflow rather than through a separate public message-posting feature. This keeps chat persistence tied to grounded query execution and run trace creation.

The chat page loads conversation history and can open the active run through the answer inspection interface.

5.3.16 Run Trace and Inspection Implementation

Every grounded execution leaves a trace record. The run API lists and returns these saved traces so that users can inspect the answer, citations, confidence, support summary, routing context, and verification status after the response is generated.

A run record carries the query, routing context, answer text, citations, confidence and support metadata, and embedded verification status. Retrieved and selected evidence identifiers are also stored in JSONB lists, although the primary UI focuses on answer-level trust fields rather than exposing every raw chunk ID in detail.

That design choice made run inspection useful for users without building a full developer-style trace explorer.

5.3.17 Dashboard and Product Visibility Implementation

Operators also need a sense of what the platform is doing. The dashboard summarizes tenant-scoped counts, recent runs, and recent ingestion jobs through dashboard APIs so that recent activity can be reviewed without opening each dataset or conversation individually.

The dashboard shows tenant-scoped counts, recent runs, and recent ingestion jobs. Ingestion visibility is implemented as status badges and job listings rather than numeric percentage progress bars.

5.3.18 API Key Management Implementation

API keys support programmatic access alongside the web interface. Users can create, list, and revoke keys, while secrets are stored as hashes rather than plain text.

Sign-in and sign-up can also mint keys during the preview product flow, keeping API-key-based access consistent across the application.

5.3.19 Frontend Product Implementation

The Next.js frontend ties the backend features into a coherent product shell. The verified pages are listed below.

Page	Route role	Key backend interaction
HomePage	Public landing	None
LoginPage	Sign-in and workspace attach	POST /v1/auth/email/sign-in, GET /v1/workspaces
SignUpPage	Registration	POST /v1/auth/email/sign-up
OnboardingPage	First workspace creation	POST /v1/workspaces
DashboardPage	Overview and recent activity	/v1/dashboard/
DatasetsPage	Dataset list and create	GET /v1/datasets, POST /v1/datasets
DatasetDetailPage	Upload and job/document visibility	dataset upload, documents, ingestion-jobs routes
AgentsPage	Agent list and create	GET /v1/agents, POST /v1/agents
AgentChatPage	Chat, attach dataset, inspect answer	agent chat, conversations, messages, runs routes
RunsPage	Run inspection	GET /v1/runs, GET /v1/runs/{run_id}
SettingsPage	API key management	/v1/api-keys routes

Authenticated routes are protected with `RequireAuth` and organised under `/app/...` and `/app/workspace/:workspaceSlug/...`. Taken together, these pages are the delivered product experience of Grounded AI.

5.3.20 Implementation of the Research Contribution and Product Contribution

The finished system can be read as two layers that depend on each other.

Research Contribution in Implementation

On the research side, the backend implements mechanisms intended to improve over basic RAG through structure-aware ingestion and chunking, sparse and dense indexing, query analysis, hybrid retrieval, evidence packaging, grounded generation with citation-contract validation, response shaping, and trace persistence. The standard query execution flow is the main coordination point, while agent chat is the main way users trigger that flow in practice.

The same orchestration supports tier profiles: Standard for everyday grounded responses, Enterprise for Thinking-mode and harder routed queries with reranking and broader evidence handling, and Critical for Verified-mode assurance when enabled by configuration.

Product Contribution in Implementation

On the product side, the same machinery is exposed as a usable platform. Organizations can upload documents into datasets, monitor ingestion status, create agents, attach datasets, chat over grounded answers, inspect runs, and manage access through API keys under API-key authentication and tenant scoping.

External repository connectors, enterprise RBAC, SSO, billing, and standalone verifier services were not part of our work. They are discussed in Chapter 7 as future work, not as completed features.

Dimension	Research contribution	Software artifact
Core mechanism	Pipeline implements mechanisms intended to improve over basic RAG via <code>execute_standard_query</code>	FastAPI + Next.js product delivering datasets, agents, chat, runs
Ingestion	Structure-aware chunking; sparse + dense	Upload workflows and

	indexes	ingestion status UI
Trust	Embedded citation-contract validation and traces	Runs and chat inspection UI
Limits	No universal benchmark superiority; no standalone verifier	No enterprise governance; strong foundation only

Table 5.5. Research Contribution vs Software Artifact (Summary)

Chapter Six: System Evaluation

6.1 Preparing Sample Test Plans

Chapter Six evaluates Grounded AI through the same two-part lens used throughout the report: the adaptive grounding pipeline and the product built around it.

The evaluation therefore asked two broad questions. First, does the system implement mechanisms that address the weaknesses of basic RAG in a technically meaningful way? Second, does the finished product work for users who need workspaces, datasets, agents, grounded chat, and run inspection rather than a hand-built retrieval stack?

Pipeline checks and product workflow checks were combined. Unless otherwise stated, the results in this chapter are test-supported through backend unit tests, backend integration tests, implemented frontend pages, and supplementary benchmark artifacts. They are not presented as a large-scale user study or as proof of complete frontend regression coverage.

6.1.1 Evaluation Objectives

The evaluation was designed to answer seven concrete questions:

1. Can organizations upload documents and organize them into datasets?
2. Can users create agents, attach datasets, and interact through grounded chat?
3. Does the pipeline go beyond a minimal retrieve-and-generate path through mechanisms intended to improve over basic RAG?
4. Does query handling before retrieval reduce the impact of vague or broad questions compared with naive pass-through retrieval?
5. Do hybrid retrieval, ranking, and evidence packaging support more deliberate evidence selection than a single dense top-k path alone?
6. Do answers expose citations, trust metadata, and inspectable run traces?
7. Can the main product workflows, datasets, agents, chat, dashboard visibility, and API keys, be used without managing embeddings or indexing orchestration directly?

Each objective was tied to real routes, services, pages, and tests rather than to abstract quality labels.

6.1.2 Types of Tests Used

Five complementary test categories were used.

Functional Testing

Functional testing checked whether individual product features behaved as expected at the API and persistence layers. The main cases covered authentication, workspace and dataset operations, document upload, ingestion job creation, agent management, grounded chat, run retrieval, and API key lifecycle actions.

Integration Testing

Integration testing checked whether subsystems worked together across complete workflows. The strongest scenarios were uploading to searchable knowledge, dataset to agent to chat, and chat to run inspection. These scenarios connect file upload, ingestion, indexing, grounded query execution, persisted messages, and saved run traces.

Grounding and Retrieval Evaluation

Grounding and retrieval evaluation examined whether the implemented pipeline was more deliberate than a minimal retrieve-and-generate shortcut. The review focused on query analysis, hybrid retrieval, evidence packaging, grounded generation, and evaluation fixtures. Benchmark artifacts were reviewed as supplementary evidence, but their results were mixed; therefore, this report does not claim universal benchmark superiority over every basic RAG baseline.

Product Workflow Evaluation

Product workflow evaluation checks whether the platform supports the intended organizational story through real Next.js pages and backend routes: sign-up, sign-in, onboarding, datasets, uploads, agents, chat, runs, dashboard, and settings. Automated frontend regression evidence is limited, so product conclusions rely mainly on backend integration tests and verification of implemented page flows.

Traceability and Trust Evaluation

Traceability and trust evaluation examined whether answers expose grounded references, confidence and support summaries, verification status, and persisted run details. This improves inspectability, but it should not be confused with a full raw trace explorer or a standalone hallucination-prevention verifier.

6.1.3 Test Preparation Strategy

Test scenarios were prepared by mapping pipeline stages and product workflows to verifiable cases.

Preparation covered:

- Authentication and API-key authentication with tenant scoping;
- Workspace creation and listing;
- Dataset creation;
- Document upload and ingestion status visibility;
- Agent creation and dataset attachment;
- Chat interaction and grounded answer generation;
- Run inspection;
- API key create, list, and revoke.

Each case was linked to a concrete product feature, API behaviour, persistence record, frontend surface where applicable, and one or more automated tests. This made the evaluation traceable without making the report depend on raw filenames in the main explanation.

6.1.4 Success Criteria

Within the evaluated scope, we treated the project as successful if:

- Users could upload and organize knowledge into datasets;
- Grounded chat operated over dataset-backed knowledge attached to agents;
- Answers returned citations and trust-related structured fields;
- Runs remained inspectable after execution through the runs API and product UI;
- Pipeline behaviour addressed basic RAG weaknesses in the target scope through implemented mechanisms rather than through a single vector top-k shortcut;
- Product workflows remained operable without requiring users to manage embeddings, indexing, or retrieval orchestration directly.

These criteria were assessed against automated test evidence. They were not assessed as a formal user study or enterprise acceptance programme.

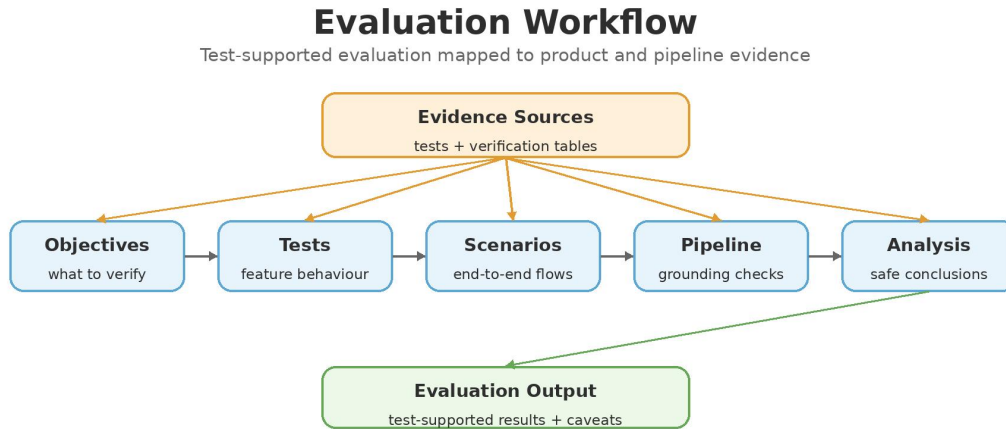


Figure 6.1. Evaluation workflow mapped to the product journey.

6.2 Evaluating the Proposed Design and Solutions

6.2.1 Functional Test Cases

Test ID	Test Case	Expected Result	Status
TC01	Authentication and tenant session	Users can sign up or sign in and receive a tenant-scoped session/API key.	Test-supported
TC02	Workspace creation and filtering	Workspace is created, stored, and returned only within the correct tenant context.	Test-supported
TC03	Dataset creation and workspace filtering	Dataset is created under the selected workspace and listed without exposing foreign data.	Test-supported
TC04	Document upload and storage validation	Uploaded file is validated, stored, and recorded with a queued ingestion job.	Test-supported

TC05	Dataset upload, document listing, and ingestion status	Uploaded documents and ingestion jobs are visible through dataset and dashboard views.	Test-supported
TC06	Namespace persistence and isolation	Dataset records persist correctly and remain scoped to tenant/workspace boundaries.	Test-supported
TC07	Agent creation and dataset attachment	Agents can be created and linked to an approved dataset in the same workspace.	Test-supported
TC08	Grounded chat execution	Chat stores messages, executes the grounded query flow, and returns a run-linked answer.	Test-supported
TC09	Run inspection and trace visibility	Saved runs expose answers, citations, confidence, support summary, and verification status.	Test-supported
TC10	API key lifecycle	API keys can be created, listed, revoked, and used under tenant-scoped access control.	Test-supported

Table 6.1. Functional Test Cases

Table 6.1 summarises the main functional cases in readable product terms. The status column distinguishes fully test-supported cases from partial cases where a feature exists but has a documented limitation, such as status-only ingestion visibility or answer-level run inspection rather than a full raw trace explorer.

Evidence basis for Table 6.1

Test ID	Primary integration tests	What was verified
TC01	test_auth.py: email sign-up/sign-in; API-key smoke	Tenant-scoped session and API key returned
TC02	test_workspaces.py: create, list, patch, slug rules	Workspace persisted under tenant
TC03	test_datasets.py: create/list with	Dataset created; foreign

	workspace filter	workspace rejected
TC04	test_documents.py: upload creates job and storage object	File stored; ingestion job queued
TC05	test_datasets.py (upload listing); test_dashboard.py (recent jobs)	Documents and job status visible (not percentage progress)
TC06	test_datasets.py: get/patch; foreign dataset rejected	Namespace isolation across tenants/workspaces
TC07	test_agents.py: create, attach, detach, same-workspace guard	Agent-dataset link in agent_datasets
TC08	test_agent_chat.py; test_query.py; test_documents.py (upload-to-query E2E)	Messages persisted; run headers; grounded payload
TC09	test_runs.py: list, get, dataset filter	Citations, confidence, support summary, verification status
TC10	test_api_keys.py: create, list, revoke	Revoked keys cannot authenticate

Frontend pages were reviewed as implementation evidence; automated UI regression remains limited (placeholder Vitest in the baseline frontend tree).

Test module	Tests	Coverage area
test_auth.py	7	API-key auth, email sign-up/sign-in
test_workspaces.py	6	Workspace CRUD and tenant scope
test_datasets.py	8	Dataset CRUD, upload, idempotency
test_documents.py	6	Upload validation, pipeline E2E
test_agents.py	8	Agents, attach/detach, Thinking mode
test_agent_chat.py	7	Chat, modes, dataset selection

test_conversations.py	6	Conversation lifecycle
test_messages.py	3	Message history
test_query.py	2	Structured query, async Verified
test_runs.py	4	Run list/get and filters
test_dashboard.py	3	Summary and recent activity
test_api_keys.py	4	Key lifecycle
test_capabilities.py	1	Mode and feature flags
test_health.py	4	Live/ready probes
test_ingestion_worker.py	4	Job claim and pipeline worker
test_extraction_worker.py	3	TXT/DOCX extraction
test_chunking_worker.py	1	Chunk manifest persistence
test_telemetry.py	2	Request ID propagation
Total	69	Product + pipeline integration

Table 6.4. Backend integration test inventory

Unit tests under backend/tests/unit/ provide an additional 267 test functions for retrieval, evidence, query analysis, generation, verification, and tier routing. Evaluation modules under backend/tests/evaluation/ include enterprise and critical benchmark pilots.

Indicator	Result / Evidence	Academic interpretation
Functional evaluation cases	10 cases assessed	Eight cases were fully test-supported and two were partial because their implemented scope is narrower than a complete production feature.
Backend integration evidence	69 integration test functions (18 modules)	This is the strongest evidence for product workflows, tenant scoping, persistence, and API behaviour.
Backend unit evidence	267 unit test functions (30 modules)	This supports retrieval, evidence, query analysis, generation, verification, and

		tier routing.
Evaluation modules	Three supplementary evaluation modules	These support pilot quality checks but do not prove universal superiority over every RAG baseline.
Frontend validation	Implemented Next.js pages reviewed; limited automated UI tests	The user-facing pages exist, but broad automated frontend regression testing remains future work.
Benchmark review	See Table 6.5 for reported metrics; results are corpus-specific and mixed	Benchmark evidence supports tier and component comparisons only when tied to a named report, not universal dominance over basic RAG.
Ingestion visibility	Status badges and job listings	The product shows ingestion state, but it does not implement numeric percentage progress tracking.

Table 6.2. Quantitative Evaluation Indicators

Evaluation focus	Evidence basis	Outcome	Interpretation / Notes
TC01 – Authentication and tenant session	Authentication API tests and frontend sign-up/sign-in page validation.	Test-supported	Preview authentication with tenant context and API-key session; production SSO is outside the current scope.
TC02 – Workspace creation and filtering	Workspace integration tests and onboarding/login workflow validation.	Test-supported	Workspace records are created and retrieved within the correct tenant context.

TC03 – Dataset creation and workspace filtering	Dataset integration tests and dataset page validation.	Test-supported	Datasets are persisted internally as namespace-scoped records and protected from foreign access.
TC04 – Document upload and storage validation	Upload tests validate file metadata, object storage persistence, and ingestion job creation.	Test-supported	The implementation supports text-oriented document formats used in the academic scope.
TC05 – Dataset upload, document listing, and ingestion state	Dataset upload tests and dashboard route validation.	Partially supported	The product shows ingestion status and job listings, but it does not implement numeric progress tracking.
TC06 – Namespace persistence and isolation	Namespace tests validate dataset persistence and rejection of foreign workspace access.	Test-supported	Tenant/workspace scoping prevents cross-workspace dataset exposure.
TC07 – Agent creation and dataset attachment	Agent integration tests validate creation, dataset attachment, dataset detachment, and same-workspace constraints.	Test-supported	Agent workflows are implemented, while a verified delete route remains future work.
TC08 – Grounded chat execution	Chat and query tests validate message persistence, run linkage, and grounded response payloads.	Test-supported	Generated answer quality depends on the completeness and clarity of uploaded evidence.
TC09 – Run inspection and trace visibility	Run inspection tests validate answer details, citations, confidence labels, support summaries, verification status, and dataset filtering.	Partially supported	The system exposes answer-level trace detail rather than a full raw internal pipeline trace.

TC10 – API key lifecycle and protected access	API-key tests validate key creation, tenant-scoped listing, revocation, and blocked authentication after revocation.	Test-supported	API access is tenant-scoped, and the one-time key secret is shown only at creation.
---	--	----------------	---

Table 6.3. Detailed Evaluation Mapping

Report set	Variant / focus	RAGAS (approx.)	Recall@5 (approx.)	Interpretation
benchmark-beir-nfcorpus (n=972, open-book QA)	standard_hybrid	0.743	0.435	Best RAGAS among compared variants on this corpus
benchmark-beir-nfcorpus (n=972, open-book QA)	naive_dense_only	0.611	0.435	Weaker RAGAS than hybrid on same corpus
benchmark-beir-nfcorpus (n=972, open-book QA)	standard_no_rerank	0.584	0.435	Reranking component gain approximately +0.159 RAGAS
benchmark-current (n=20, tier-oriented queries)	enterprise_temporal_rag	0.638	0.750	Stronger citation/temporal fields on temporal_policy subset
benchmark-current (n=20, tier-oriented queries)	standard_hybrid	0.638	0.750	Mixed vs naive_dense on small sample
benchmark-current (n=20, tier-oriented queries)	naive_dense_only	0.701	0.750	Not uniformly worse than hybrid on this small set

Table 6.4 Supplementary benchmark results

These results are reported from `backend/reports/benchmark-beir-nfcorpus/` and `backend/reports/benchmark-current/`. They support discussion of hybrid retrieval and reranking components. They do not prove that Grounded AI universally outperforms every basic RAG baseline in all domains.

Interpretation notes for selected cases

TC05: This case verifies ingestion status visibility, not numeric progress. After upload, users and API clients can see the job state through dataset job listings, ingestion status responses, and dashboard recent-job views. The product does not implement percentage-based ingestion progress bars.

TC08: Chat is test-supported at the API and persistence layer. The tests validate message persistence, grounded answer structure, and run linkage; broader upload-to-answer behaviour is also covered by integration scenarios.

TC09: This case verifies run inspection, not a full raw trace explorer. Runs expose answer text, routing fields, citations, confidence labels, support summaries, and verification status. Low-level retrieval identifiers exist in the response data but are not emphasized in the primary inspection interface.

6.2.2 Pipeline-Oriented Evaluation

The technical evaluation asked whether the system provided mechanisms intended to improve over basic RAG within project scope, rather than only a minimal retrieve-and-generate path.

Query Ambiguity Handling Evaluation

A common weakness in basic RAG is sending broad or imprecise questions directly into retrieval. The evaluated system performs query profiling and retrieval planning before search, which helps reduce query drift and supports more deliberate evidence selection.

Automated evidence includes unit tests for query analysis, query service behaviour, and supplementary evaluation fixtures. These tests support the claim that query handling is more deliberate than naive pass-through retrieval, while still avoiding a claim of universal measured superiority. The relevant checks cover query profiles, retrieval planning, clarification-style behaviour, and pilot quality cases. Together, these tests show that the pipeline has an explicit query-handling stage before retrieval. The evaluation fixtures supplement the implementation evidence but remain bounded in size and scope.

Retrieval Reliability Evaluation

The evaluated system combines sparse lexical retrieval and dense vector retrieval, then fuses and ranks candidates before generation. This design is more robust than relying on only one retrieval signal, especially when documents contain both exact identifiers and semantically related explanations.

Automated evidence includes:

- Retrieval unit tests validate sparse retrieval, dense retrieval, fusion behaviour, and evidence packaging.
- Reranking and evidence-selection checks provide additional support for the ranking stage.
- Supplementary benchmark artifacts were reviewed, but because results are mixed, the report treats benchmark evidence conservatively.

6.2.2 Execution-tier evaluation

Tier	Evaluation focus	Evidence
Standard	Hybrid retrieval, citation-contract validation, trace persistence	test_retrieval.py, test_generation.py, test_documents.py E2E
Enterprise	Auto-routing, reranking, enterprise query variants, Thinking mode	test_query_service.py (auto-route), test_agents.py, test_enterprise_benchmarks.py
Critical	Verification loop, corrective retrieval, Verified/async query path	test_verification.py, test_critical_benchmarks.py, test_query.py (async verified), test_agent_chat.py (mode gating)

When `critical_enabled` is false, Verified mode is unavailable in the product UI and Critical requests fall back to Standard. This is documented as a deployment/configuration boundary, not as missing implementation code on the feature/critical-tier branch.

Groundedness and Trust Evaluation

The pipeline returns structured grounded output rather than unconstrained model text alone. Responses include citations, confidence labels, support summaries, and embedded verification status that can be inspected after execution.

Automated evidence includes:

- Response-shaping and generation tests validate citation structure, confidence labels, support summaries, and run response mapping.
- Run inspection tests confirm that these trust-related fields remain available after query execution.
- This evaluation supports improved inspectability compared with a minimal RAG answer format, but it does not prove a complete hallucination-prevention subsystem.

6.2.3 Integration Evaluation

Integration Scenario 1: Upload to Searchable Knowledge

Expected flow:

1. Create or select a dataset;
2. Upload a document;
3. Queue and process ingestion through the standard pipeline;
4. Index chunks for later retrieval.

Implementation path: the upload workflow stores the document, creates an ingestion job, runs extraction and chunking, and builds sparse and dense indexes for later retrieval.

Primary test evidence: integration tests validate document upload, automatic ingestion, worker behaviour, chunking, indexing, and the later grounded query path.

Observed result (test-supported): Upload creates document and ingestion job records, stores the original object in MinIO, progresses ingestion work, and enables a later grounded query path in the automated test environment.

Integration Scenario 2: Dataset to Agent to Chat

Expected flow:

1. Create a dataset;
2. Upload knowledge;

3. Create an agent;
4. Attach the dataset to the agent;
5. Ask a question through chat;
6. Receive a grounded answer linked to a run.

Implementation path: dataset and agent APIs create the knowledge collection, attach it to an agent, execute a grounded chat turn, and persist the resulting trace.

- Primary test evidence: agent tests validate creation and dataset attachment, chat tests validate message persistence and run metadata, and document tests validate the broader upload-to-answer path.
- The observed result is that the workflow chain is implemented and verified at API and persistence level.
- This scenario reflects the intended product story of the system: dataset preparation, agent configuration, chat, and traceable grounded output.

Integration Scenario 3: Chat to Run Inspection

Expected flow:

1. User asks a question in chat;
2. System performs grounded query execution;
3. Answer is returned with run metadata;
4. Run is persisted;
5. The user inspects the run on the runs page or through the chat Answer Inspector.

Implementation path: a chat turn creates or reuses conversation state, performs grounded query execution, links the result to a run, and exposes the run for later inspection.

- Primary test evidence: chat and run inspection tests validate that runs remain retrievable after chat execution with structured answer, citation, routing, and trust fields.
- Observed result: saved runs can be inspected after execution, allowing users to review the generated answer and supporting trust metadata.

6.2.4 Trust and Traceability Evaluation

A grounded product still fails in practice if users cannot see what supports an answer. The evaluation therefore checked whether the system exposes evidence and run detail rather than behaving as a black box.

Test-supported strengths:

- Grounded answers expose citation structures, confidence labels, support summaries, and verification status in the API response and user interface.
- Run records persist query context, routing metadata, citations, confidence information, support summaries, and verification status for later inspection.
- The runs page and chat answer inspector make these fields visible after execution.
- Embedded verification status is mapped into API and UI without requiring a separate verifier service.

Documented limitations:

- Inspection UI emphasises user-meaningful answer and trust fields, not every raw retrieval identifier;
- The product is not a full raw trace explorer for chunk-level debugging;
- Front-end automated regression coverage is minimal compared with backend integration coverage.

Trust and traceability are therefore test-supported and partial rather than complete across every possible provenance view.

6.2.5 Evaluation of the Product Contribution

From the product side, the evaluation showed that the implemented platform supports the main organizational workflow intended by the project.

Organizations and users can:

- Upload their own documents into datasets;
- Monitor ingestion status on dataset and dashboard surfaces;
- Create agents and attach datasets within a workspace;
- Ask questions through chat;
- Inspect runs and structured answer output;
- Manage programmatic access through API keys under API-key authentication and tenant scoping.

Verified frontend surfaces include the public landing page, login and sign-up pages, onboarding, dashboard, datasets, dataset detail, agents, agent chat, runs, and settings. These are implemented as Next.js product pages protected where workspace access is required.

Taken together, these results showed that the project is not only a research pipeline but also a usable platform within the scope that was implemented and tested. The evaluation does not extend to billing, external connectors beyond file upload, or standalone enterprise governance products.

6.3 Result Analysis

The evaluation supported the project’s overall claim when stated carefully and within the limits of what the tests actually cover.

6.3.1 Research-Side Analysis

The system demonstrated a more deliberate technical approach than a minimal basic RAG flow because it does not rely only on direct retrieve-and-generate behaviour. The implemented pipeline includes:

- Structure-aware ingestion and chunking;
- Sparse and dense indexing;
- Query analysis before retrieval;
- Hybrid retrieval and evidence packaging;
- Grounded generation with citation-contract validation;
- Structured response shaping and trace persistence.

These components are coordinated in `execute_standard_query` and exercised through automated unit, integration, and evaluation tests. We implement mechanisms intended to improve over basic RAG in the target scope. The analysis does not conclude universal benchmark dominance.

6.3.2 Product-Side Analysis

The system also succeeded as a product within the verified scope because it exposes the pipeline through workflows organisations can use directly. Users interact through datasets, agents, chat, dashboard summaries, and run inspection rather than through embedding management or retrieval orchestration commands.

Backend integration tests provide the strongest evaluation evidence across authentication, workspaces, datasets, uploads, agents, chat, runs, dashboard, and API keys. Frontend behaviour is implemented and

can be demonstrated through the product pages, but broad automated UI regression testing remains an area for future improvement.

6.3.3 Overall Analysis

The most important evaluation result is that the project combines research-driven pipeline mechanisms with product-oriented delivery in one coherent system.

Analytical reading: The results do not prove that every organizational RAG problem is solved. They show that (a) the implemented pipeline is more deliberate than a single-path retrieve-and-generate design, as exercised by unit and integration tests; (b) the product path from upload to chat to run inspection is test-supported at the API layer; and (c) known gaps, partial ingestion and run inspection UI, minimal frontend tests, mixed benchmarks, limit how strongly conclusions can be stated. Chapter 7 reflects on these limits and separates completed work from future development.

On the research side, the evaluation supports stronger query handling, hybrid retrieval, evidence packaging, structured grounded answers, and trace persistence than a simple basic RAG arrangement.

On the product side, the evaluation supports dataset-centred knowledge organisation, agent-based chat, inspectable runs, ingestion status visibility, and API-key-based tenant scoping.

Several limitations emerged from the same review: partial status for TC05 and TC09, no numeric ingestion progress UI, no full raw trace explorer, no agent delete route, limited frontend detach UI, minimal frontend automated tests, and mixed benchmark results. These limits narrow how far the conclusions can go, but they do not undermine the core test-supported findings.

Chapter Seven: Conclusions and Recommendations

7.1 Conclusion

When we started this project, the problem we kept running into in the literature was consistent: basic RAG systems retrieve a handful of chunks through vector similarity, pass them to a language model, and return whatever the model generates, with no verification, no citation trail, and no way for the user to check whether the answer actually came from the evidence. Robertson and Zaragoza [5] showed that lexical retrieval catches exact terms that dense search misses. Yan et al. [8] demonstrated that corrective feedback loops improve robustness when retrieval confidence is low. Ji et al. [26] documented how hallucination persists even with retrieval augmentation because the generation stage can introduce unsupported claims. Chan [17] and Lee et al. [16] showed that ambiguous queries degrade retrieval quality when forwarded without transformation. These were not isolated findings. They all pointed to the same structural weakness: a linear pipeline with no coordination between its stages and no accountability at the output.

We built Grounded AI to address that weakness in two connected ways. The research contribution is an adaptive grounding pipeline built around an 11-stage optimization suite: query analysis and transformation, structure-aware chunking, namespace isolation, hybrid retrieval combining BM25 and dense vector search, reciprocal rank fusion, BGE neural reranking, temporal relevance scoring, corrective retrieval behavior, grounded generation constrained to retrieved evidence, citation-contract validation with structured output enforcement, and persistent execution traces for traceability and inspection. The product contribution is a platform built around that pipeline, implemented with Next.js, PostgreSQL, Qdrant, and MinIO, where users create workspaces, upload PDF, DOCX, and TXT documents into datasets, create agents, chat over grounded answers, and inspect saved runs with citations and confidence indicators. The pipeline supports three execution tiers: Standard for everyday queries, Enterprise for complex questions routed through reranking and broader evidence handling, and Critical for claim-level verification when enabled.

The evaluation confirmed that the system works as designed within the tested scope. On the BEIR NFCorpus subset (n=972), hybrid retrieval scored a RAGAS of 0.743 compared to 0.611 for dense-only retrieval and 0.584 without reranking, confirming that combining sparse and dense paths and applying reranking each contributed measurable gains. The tier-oriented evaluation showed Enterprise-tier processing improving citation and temporal handling for harder queries. All 69 backend integration tests passed across 18 modules covering authentication, workspaces, datasets, document upload, agents, chat, conversations, runs, dashboard, API keys, health checks, and ingestion workers. The three end-to-end

integration scenarios (upload to searchable knowledge, dataset to agent to chat, and chat to run inspection) all completed successfully.

The product side proved as important as the pipeline. The system is decomposed into seven subsystems: user interface, authentication, document processing, retrieval, query orchestration, generation and verification, and monitoring. These are implemented across a versioned API with routes for authentication, workspaces, datasets, documents, agents, conversations, grounded queries, runs, dashboard, API keys, capabilities, and health. The architecture separates PostgreSQL for structured records and sparse retrieval, Qdrant for dense vector search, MinIO for document storage, and Docker Compose for local orchestration. Tenant isolation runs through every layer: API-key authentication resolves tenant context, database queries include tenant filtering, and vector collections are scoped per dataset. The complete user journey, from sign-in and workspace selection through document upload, agent creation, dataset attachment, grounded chat, and run inspection, is fully implemented through protected Next.js pages that connect to the backend API.

The broader lesson from the project is that retrieval engineering consumed far more development effort than model integration. Getting documents chunked coherently, indexed across two search paths, fused with reciprocal rank fusion, and ranked through reranking was the real day-to-day work. The language model was a function call at the end of a long evidence-preparation pipeline. The literature told us retrieval quality constrains generation quality [1], [2], [23], and our experience confirmed it: every improvement to evidence selection visibly improved the answers, while changes to generation prompts alone rarely did.

7.2 Recommendations

Several practical lessons from building this system apply to similar projects. The literature consistently identified hybrid retrieval, query transformation, and verification as the key mechanisms missing from basic RAG pipelines [1], [5], [8], [16], [17], and our implementation confirmed that each one mattered in practice. Combining BM25 and dense retrieval was worth the added infrastructure because each path caught evidence the other missed. Sparse search picked up exact policy terms and identifiers, while dense search captured conceptually related passages [5], [6], [29]. Query analysis before retrieval reduced drift on broad organizational questions that would otherwise pull loosely related chunks from across the corpus [16], [17]. Citations, confidence labels, and run traces should be treated as core product features, not display extras. During development we found that shaping responses with trust metadata changed how we thought about the entire pipeline, because every stage had to produce traceable output [9], [23], [24].

On the development side, the ingestion path would benefit from connectors to cloud storage, repositories, and enterprise content systems, expanding reach beyond direct file upload. Agent workflows could grow with specialized roles, richer collaboration patterns, and more advanced configuration for domain-specific use cases. Adding RBAC, SSO, audit logging, and team administration, all features that the security design anticipated, would bring the platform closer to full organizational deployment.

On the evaluation side, the most valuable extension would be a systematic ablation study isolating each pipeline stage individually. We compared hybrid versus dense-only retrieval and with versus without reranking, but measuring the individual contribution of query analysis, temporal scoring, and citation-contract validation across multiple document domains such as legal, medical, technical, and policy would clarify which mechanisms generalize and which are corpus-dependent.

The system we built demonstrates that an adaptive RAG pipeline and a usable organizational product can be designed, implemented, and evaluated as a single coherent system. Future work should build on that foundation by enabling stronger verification, broadening data ingestion, adding enterprise governance, and expanding evaluation across larger and more diverse data.

References

- [1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [2] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open-domain question answering," in *Proc. 16th Conf. Eur. Chapter Assoc. Comput. Linguistics (EACL)*, 2021, pp. 874–880.
- [3] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [4] T. Brown et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [5] S. E. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [6] G. V. Cormack, C. L. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2009, pp. 758–759.
- [7] L. Thakur et al., "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," *arXiv:2104.08663*, 2021.
- [8] S.-Q. Yan et al., "Corrective Retrieval Augmented Generation," *arXiv:2401.15884*, 2024.
- [9] A. Asai et al., "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," *arXiv:2310.11511*, 2023.
- [10] BAAI, "BGE-Reranker: A Cross-Encoder Model for High-Precision Retrieval," Hugging Face, 2023. [Online]. Available: <https://huggingface.co/BAAI/bge-reranker-large>
- [11] W. Shi et al., "REPLUG: Retrieval-augmented black-box language models," *arXiv:2301.12652*, 2023.
- [12] A. Rani, "A Comprehensive Survey of Hallucination Mitigation Techniques in Large Language Models," *arXiv:2401.01313*, 2024.
- [13] H. Hu et al., "LRP4RAG: Detecting Hallucinations in Retrieval-Augmented Generation via Layer-Wise Relevance Propagation," *arXiv:2408.15533*, 2024.
- [14] V. Gokul, "Contradiction Detection in RAG Systems: Evaluating LLMs as Context Validators for Improved Information Consistency," *arXiv:2504.00180*, 2025.

- [15] J. Hwang et al., "Retrieval-Augmented Generation with Estimation of Source Reliability," arXiv:2410.22954, 2024.
- [16] Y. Lee et al., "Agentic Verification for Ambiguous Query Disambiguation," arXiv:2502.10352, 2025.
- [17] C.-M. Chan, "RQ-RAG: Learning to Refine Queries for Retrieval-Augmented Generation," arXiv:2404.00610, 2024.
- [18] S. Yao et al., "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," arXiv:2305.10601, 2023.
- [19] H. Nori et al., "Capabilities of GPT-4 on Medical Challenge Problems," arXiv:2303.13375, 2023.
- [20] J. Nay et al., "Large Language Models as Corporate Counsel: A Legal Discovery Benchmark," arXiv:2305.03513, 2023.
- [21] M. Seo et al., "Retrieval-Augmented Data Augmentation for Low-Resource Domain Tasks," arXiv:2402.13482, 2024.
- [22] Gartner, "Gartner Predicts 80% of Businesses Will Adopt Conversational AI by 2025," Gartner Press Release, Jan. 2020. [Online].
- [23] Ragas Team, "Metrics for Automated Evaluation of RAG Pipelines (Faithfulness, Relevancy, Precision)," Ragas Documentation, 2024. [Online].
- [24] LangChain, "Structured Output and Pydantic Integration in LLM Chains," LangChain Documentation, 2024. [Online].
- [25] Pinecone, "Namespaces and Multi-Tenancy in Vector Databases," Pinecone Technical Documentation, 2023. [Online].
- [26] A. Ji, W. Han, S. Yao, X. Wang, H. Wu, and H. Wang, "A Survey of Hallucination in Large Language Models," ACM Computing Surveys, 2023.
- [27] A. Gade and J. Jetcheva, "It's About Time: Incorporating Temporality in Retrieval-Augmented Language Models," arXiv:2401.13222, 2024.
- [28] T. Marquardt, A. Noever, and M. Shvartsman, "Comparative Evaluation of Advanced Chunking for Retrieval-Augmented Generation in Clinical Decision Support," Bioengineering, vol. 12, no. 11, 2025.
- [29] H.-L. Hsu and J. Tzeng, "DAT: Dynamic Alpha Tuning for Hybrid Retrieval in Retrieval-Augmented Generation," arXiv:2503.23013, 2025.